



École Polytechnique Fédérale de Lausanne

Exploring AI Assistants for ERP Systems: From Retrieval-Augmented Wrappers to Autonomous Self-Improving Agents

by David Loïck Claude Lacour

Master Thesis

Approved by the Examining Committee:

Dr. Mathieu Salzmann, CVLAB, EPFL
Thesis Advisor

Dr Richard Lengagne
External Expert

Jean-Gabriel Marchetta
Thesis Supervisor

EPFL IC
BC 160 (Bâtiment BC)
CH-1015 Lausanne

February 27, 2026

*"We can only see a short distance ahead,
but we can see plenty there that needs to be done."*
— Alan Turing

Dedicated to my friends and family,
whose unwavering support made this journey possible.

Acknowledgments

I would like to express my sincere gratitude to all those who have supported me throughout this thesis journey.

I am profoundly grateful to EPFL (École Polytechnique Fédérale de Lausanne) for providing an exceptional academic environment and the resources necessary to conduct this research. The institution's commitment to excellence in education and research has been instrumental in shaping this work.

I would like to express my sincere appreciation to the Mediasoft team for their invaluable collaboration and for granting access to the Mediasoft MS.ERP system. Their expertise in ERP software development and their willingness to support this research have been essential to the practical aspects of this thesis.

Special thanks go to the Wiven team for their technical guidance and support throughout this project. Their insights into AI applications for enterprise software have greatly enriched this work.

Finally, I thank all the professors, colleagues, and collaborators who have contributed to this thesis through discussions, feedback, and encouragement. Your collective wisdom has helped shape this research into its final form.

David Loïck Claude Lacour

Abstract

Enterprise Resource Planning (ERP) systems are critical to modern organizations, yet their complexity creates significant barriers for users, particularly in small and medium-sized enterprises lacking dedicated IT support. This thesis explores four progressively autonomous approaches to building intelligent assistants for ERP systems: (1) a RAG-enhanced [1] chat interface using commercial models for immediate deployment, (2) a terminal-using agent capable of autonomous multi-step research and dependency resolution, (3) a fine-tuned vision-language model trained on domain-specific data, and (4) exploring various auto-learning strategies including multi-agent voting, environment-grounded rejection fine-tuning, and autonomous zero-supervision learning, where agents interact with the ERP system via the Model Context Protocol (MCP), generate training data from successful executions, and iteratively retrain without human supervision.

Each solution represents a different point in the design space, navigating trade-offs between deployment complexity, computational requirements, privacy constraints, and the level of system autonomy. Rather than developing any single approach in depth, the thesis surveys this broad landscape of solutions for small and medium-sized enterprises seeking to integrate AI assistance into their ERP workflows while preserving data privacy and running on local infrastructure.

Résumé (French Abstract)

Les systèmes de planification des ressources de l'entreprise (ERP) sont essentiels aux organisations modernes, mais leur complexité crée des barrières significatives pour les utilisateurs, particulièrement au sein des petites et moyennes entreprises dépourvues de support informatique dédié. Cette thèse explore quatre approches progressivement autonomes pour construire des assistants intelligents pour les systèmes ERP : (1) une interface de chat augmentée par RAG utilisant des modèles commerciaux pour un déploiement immédiat, (2) un agent terminal capable de recherche autonome multi-étapes et de résolution de dépendances, (3) un modèle vision-langage affiné sur des données spécifiques au domaine, et (4) l'exploration de diverses stratégies d'auto-apprentissage incluant le vote multi-agents, le fine-tuning par rejet ancré dans l'environnement et l'apprentissage autonome sans supervision, où les agents interagissent avec le système ERP via le Model Context Protocol (MCP), génèrent des données d'entraînement à partir d'exécutions réussies et se ré-entraînent itérativement sans supervision humaine.

Chaque solution représente un point différent dans l'espace de conception, naviguant les compromis entre complexité de déploiement, exigences computationnelles, contraintes de confidentialité et niveau d'autonomie du système. Plutôt que de développer une seule approche en profondeur, cette thèse parcourt ce large paysage de solutions pour les petites et moyennes entreprises cherchant à intégrer une assistance IA dans leurs flux de travail ERP tout en préservant la confidentialité des données et en fonctionnant sur une infrastructure locale.

Contents

Acknowledgments	1
Abstract (English/Français)	2
1 Introduction	7
1.1 Context and Motivation	7
1.2 The Challenge of AI-Assisted ERP Navigation	7
1.3 Proposed Solutions	8
1.3.1 Phase 1: Foundational Approaches	8
1.3.2 Phase 2: Autonomous Self-Improving Agents	8
1.4 Thesis Statement	9
1.5 Contributions	10
1.6 Design Overview	10
1.7 Thesis Organization	11
2 Background and Related Work	12
2.1 Technical Foundations	12
2.1.1 Transformers and Vision-Language Models	12
2.1.2 Alignment: From SFT to Preference Optimization	12
2.1.3 Parameter-Efficient Fine-Tuning	13
2.1.4 Retrieval-Augmented Generation	13
2.2 Related Work	13
2.2.1 Document Understanding Benchmarks	13
2.2.2 Computer-Using and Terminal-Using Agents	14
2.2.3 Self-Improving AI Systems	14
2.2.4 Tool-Using Language Models	14
2.2.5 Structured Output and Function Calling	14
2.2.6 LLM-Integrated ERP Systems	15
2.2.7 Text-to-SQL for Enterprise Databases	15
2.3 The Target Domain: Enterprise Resource Planning	15
2.3.1 The Target System: Mediasoft MS.ERP	15
2.3.2 Open Source ERP for multiagent exploration: ERPNext	15
3 Commercial Model Approaches	16
3.1 Solution 1: ChatGPT Wrapper Interface	16
3.1.1 Documentation Preprocessing	16
3.1.2 RAG and Keyword Search	17
3.1.3 Images RAG and Image Search	17
3.1.4 Database or Mock Database Integration	17
3.1.5 Limitations	17
3.1.6 Internship at Wiven	18

3.2	Solution 2: Terminal-Using Agent	18
3.2.1	Agent Capabilities	18
3.2.2	Query Resolution Process	18
3.2.3	Security Considerations	18
3.2.4	Deployment	19
4	Solution 3: Fine-Tuned Vision-Language Model	20
4.1	Model Selection Rationale	20
4.2	Training Paradigms	20
4.3	Training Objectives	21
4.4	Data Sources	21
4.5	Balancing Specialization and Generalization	22
4.6	Dataset Construction for VLM Fine-Tuning	22
4.6.1	User Manual Processing	22
4.6.2	Automated Question Generation	23
4.6.3	Dataset Statistics	24
4.7	Training Procedure	24
4.8	Evaluation	26
4.8.1	Evaluation Methodology	26
4.8.2	Evaluation Metrics	26
4.8.3	VLM Fine-Tuning Results	29
4.8.4	Discussion: Dataset Requirements	46
5	Autonomous Learning	48
5.0.1	Design Philosophy	48
5.0.2	Failed SmolVLM2-256M Experiment	49
5.0.3	Platform and Model Changes	49
5.1	Base Model (Zero-Shot)	49
5.2	SFT (Warm up Supervised Fine-Tuning)	50
5.2.1	Dataset Design	50
5.2.2	Training Procedure	51
5.3	E-GRFT (Environment-Grounded Rejection Fine-Tuning)	51
5.3.1	Algorithm	52
5.3.2	Detailed Description	52
5.4	SSMCQ (Self-Supervised MCP Querying)	54
5.4.1	Algorithm	54
5.4.2	Detailed Description	54
5.5	Voting (Multi-Agent Voting Training)	55
5.5.1	Architecture	55
5.5.2	Voting Pipeline	56
5.5.3	Detailed Description	56
5.6	WFCSIT (Wave Function Collapse Self-Improvement Training)	57
5.6.1	Constraint Graph	57
5.6.2	WFC Algorithm	57
5.6.3	Detailed Description	58
5.6.4	Complexity Levels	60
5.6.5	Reward	60
5.7	SYS (Autonomous Zero-Supervision Learning)	61
5.7.1	Components	61
5.7.2	Safety	61
5.7.3	Detailed Description	62
5.8	MSYS (Modular SYS)	64

5.8.1	Architecture	64
5.8.2	Detailed Description	65
5.9	LSI (Layered Self-Improvement)	65
5.9.1	Layer 1: Dynamic Tool Discovery	66
5.9.2	Layer 2: ToolBench-Style Data Generation	66
5.9.3	Layer 3: Cold-Start SFT_auto	66
5.9.4	Layer 4: GRPO + DPO Reinforcement Learning	66
5.9.5	Layer 5: Tool Self-Evolution	67
5.10	In-Distribution Results (990 samples)	68
5.11	Out-of-Distribution Results (193 samples)	69
5.12	Generalization Gap Analysis	70
5.13	Live Execution Metrics	70
5.14	Per-Tool OOD Accuracy	71
5.15	Training Efficiency	72
5.16	Summary	72
5.17	Limitations	73
5.17.1	Dataset Size and Quality	73
5.17.2	Infrastructure Constraints and Experimental Scope	73
5.17.3	Out-of-Distribution Evaluation	74
5.17.4	Hidden Engineering Cost of Procedural Generation	74
5.17.5	Uneven Depth and Lack of Cross-Solution Comparison	74
5.18	Risk and Security	74
6	Conclusion	76
6.1	Future Work	78
6.1.1	Combining autonomous learning and lessons from Game-Playing AI	78
6.1.2	Discriminator-Based Evaluation for Complex Software	79
6.1.3	NPC-Driven Synthetic Data Generation	80
	Bibliography	81
A	Qualitative Examples	85
A.1	Standard Benchmark Outputs	86
A.2	Domain-Specific ERP Outputs	91
	Declaration on the Use of AI Tools	95

Chapter 1

Introduction

1.1 Context and Motivation

Enterprise Resource Planning (ERP) systems serve as the digital backbone of modern organizations, unifying operations across finance, human resources, inventory, logistics, and project management. These platforms centralize vital business processes, but as their scope expands, so does their complexity. Interfaces grow denser, workflows more intricate, and the cognitive load placed on users increases substantially. For many small and medium-sized enterprises, especially those without dedicated IT teams, working effectively with ERP software becomes a major operational challenge rather than a solved problem.

This complexity manifests in several concrete ways. New users frequently struggle to locate functionalities, interpret dashboards, or understand the relationships between interface components. Users consistently forget to tick important checkboxes that affect downstream processing. Support teams field recurring questions that depend on both visual context (such as screenshots of the ERP interface) and textual descriptions of user concerns. This leads to inefficiencies, slow onboarding, and a high volume of repetitive IT support tasks. As ERP systems become visually richer and more customizable, since each company typically requires its own tailored version, these challenges intensify, creating barriers to efficient system use and slowing organizational productivity.

1.2 The Challenge of AI-Assisted ERP Navigation

While modern vision-language models (VLMs) offer promising capabilities for understanding both images and text, solutions for the ERP domain remain nascent. Large VLMs exhibit strong reasoning capabilities but are costly to run and difficult to fine-tune for specialized enterprise contexts. These models often lack exposure to the interfaces of smaller, specialized ERP systems such as Mediasoft MS.ERP. Furthermore, large models frequently hallucinate, inventing non-existent modules or creating implausible amalgamations of features from major ERP platforms like Microsoft Dynamics and SAP.

Smaller models present a different set of trade-offs: they are more deployable on modest hardware but may lack the necessary reasoning capabilities to go beyond simple optical character recognition (OCR) and truly understand the relationships between displayed text, screenshot context, underlying software architecture, and real enterprise operations. Even general-purpose AI assistants, while increasingly powerful, sometimes exhibit limited reasoning capabilities when confronted with domain-specific

interface understanding tasks.

1.3 Proposed Solutions

To address these limitations, this thesis investigates a broad spectrum of approaches, organized into two main phases. The first phase explores three foundational architectures that differ in their reliance on commercial models, their level of system access, and their computational requirements. The second phase builds on these foundations with a family of increasingly autonomous self-improving methods that learn directly from interaction with a live ERP environment.

1.3.1 Phase 1: Foundational Approaches

1. **RAG-Enhanced Chat Interface (Solution 1):** A ChatGPT-based wrapper accessible via a web interface, combining Retrieval-Augmented Generation with keyword search to provide contextually relevant responses grounded in ERP documentation. This approach offers the fastest deployment path but depends on commercial model providers, raising privacy concerns when processing sensitive enterprise data.
2. **Terminal-Using Agent (Solution 2):** An agentic system capable of autonomous research, multi-step reasoning, and complex dependency resolution through programmatic access to documentation, codebases, and database schemas. Deployed as a Claude Code wrapper on an isolated server, it can follow chains of references across modules to resolve queries that exceed the capabilities of static retrieval.
3. **Fine-Tuned Vision-Language Model (Solution 3):** Comprehensive fine-tuning of SmolVLM2-256M-Video-Instruct using multiple training paradigms, full fine-tuning, parameter-efficient adaptation with QLoRA, supervised fine-tuning (SFT) on multiple-choice questions, and Direct Preference Optimization (DPO) for response quality alignment, enabling domain-specific visual understanding on a single 8 GB consumer GPU.

1.3.2 Phase 2: Autonomous Self-Improving Agents

The second phase shifts from human-curated pipelines to systems that generate their own training data by interacting with a live ERP instance via the Model Context Protocol (MCP). Starting from a multi-agent exploration framework, the thesis progressively removes human supervision:

4. **Multi-Agent Exploration (Solution 4):** A 5-agent voting system in which heterogeneous agents, both vision-language and text-only models, both trainable and frozen, independently propose actions for ERP tasks, execute them, and reach consensus through majority voting. Successful episodes become supervised training signal for the trainable agents.
5. **MCP Distillation:** A three-stage training pipeline combining coded task permutations for MCP tool use, semi-pretraining on ERPNext documentation, and knowledge distillation from commercial models (Codex, Gemini CLI), followed by iterative refinement via a Terminal-Using Agent. While effective, this approach may conflict with the terms of service of commercial providers.
6. **Environment-Grounded Rejection Fine-Tuning (E-GRFT):** The model interacts with a live ERPNext instance, and a four-component reward function (format, tool, parameter, and execution

correctness) filters successful experiences via rejection sampling. Only gold and silver episodes are mixed with the original SFT data for retraining, with automatic validation and rollback to prevent degradation.

7. **Self-Supervised MCP Querying (SSMCQ):** A variant of E-GRFT that combines execution-based filtering with self-play instruction expansion. The model generates tool calls and MCQ questions grounded in live ERP data, retains only verified successes, and retrains on the filtered samples, achieving the highest in-distribution parameter accuracy.
8. **Multi-Agent Voting Training:** An ensemble of five heterogeneous agents, including VLMs, LLMs, and a BM25-based RAG agent, that independently generate, execute, and vote on action candidates. Trainable agents are updated on the consensus winner, combining diverse model perspectives for fault-tolerant decision-making.
9. **Wave Function Collapse Self-Improvement Training (WFCSIT):** An adaptation of the Wave Function Collapse algorithm from procedural content generation to multi-step ERP document creation. ERPNext doctypes are modeled as tiles in a constraint graph, and the algorithm navigates the dependency DAG to produce valid document chains of increasing complexity, from single documents to cross-module workflows.
10. **Autonomous Zero-Supervision Learning (SYS):** The most autonomous approach, combining insights from six recent research papers into a nine-component system that autonomously discovers the ERP schema, builds a dependency graph, designs an adaptive curriculum, learns from execution feedback with a six-component reward, maintains an episodic BM25 memory bank, and retrains, all with zero human supervision. SYS achieves the smallest parameter generalization gap among all trained models.
11. **Modular SYS (MSYS):** A 10-module self-improvement system combining schema discovery, constraint inference, adaptive curriculum generation, BM25 episodic memory, RAG retrieval, and rejection sampling with a six-component reward.
12. **Layered Self-Improvement (LSI):** A 5-layer pipeline extending rejection sampling with true reinforcement learning. Layer 1 dynamically discovers tools via semantic routing; Layer 2 generates training data using ToolBench-style G1/G2/G3 tiers with DFSDT annotation; Layer 3 performs cold-start SFT; Layer 4 applies GRPO and DPO for policy optimization; Layer 5 auto-generates and tests 50 new MCP tools via Alita-style evolution.

Together, these approaches span a broad design space, from immediate-deployment wrappers relying on commercial infrastructure to fully autonomous agents that learn entirely from self-generated experience on local hardware. Each represents a different set of trade-offs between deployment complexity, computational requirements, privacy constraints, dataset availability, and the level of system autonomy.

1.4 Thesis Statement

This thesis explores how AI assistance for specialized ERP systems can be approached through multiple strategies at different levels of autonomy: retrieval-augmented chat interfaces and terminal-using agents for immediate deployment with commercial models; fine-tuned vision-language models for privacy-preserving local inference; and a family of self-improving methods, including environment-grounded rejection sampling, self-supervised MCP querying, multi-agent voting, constraint-based exploration, fully autonomous zero-supervision learning, modular iterative self-improvement, and layered reinforcement

learning with tool evolution, that progressively reduce the need for human annotation by generating training data through direct interaction with the ERP system via the Model Context Protocol. The choice among these approaches depends on organizational constraints including computational resources, security requirements, documentation quality, the complexity of user queries, and the desired level of system autonomy.

1.5 Contributions

This thesis makes the following contributions:

- A comparative exploration of multiple architectural approaches to AI-assisted ERP navigation, from RAG-enhanced wrappers and terminal-using agents to fine-tuned vision-language models and a family of autonomous self-improving systems, identifying the trade-offs and potential use cases for each.
- A dataset construction methodology that combines existing techniques (multiple-choice question generation, Direct Preference Optimization training pair extraction, and screenshot-based data augmentation) into an integrated pipeline for producing training data from ERP documentation and interface screenshots.
- An investigation of parameter-efficient fine-tuning (QLoRA) of a 256M-parameter VLM, suggesting it may act as an implicit regularizer that preserves pretrained capabilities, alongside observations that current small-scale models and datasets face challenges for transferable ERP domain knowledge.
- A family of self-improving training methods, E-GRFT, SSMCQ, WFCST, SYS, MSYS, and LSI, that interact with ERP systems via MCP and explore autonomous improvement through rejection sampling, reinforcement learning, and tool evolution on successfully completed task executions.
- A multi-agent voting framework exploring whether heterogeneous agent ensembles can provide fault tolerance under distribution shift, and examining the conditions under which consensus-based training produces a useful learning signal.
- Practical deployment recommendations for small and medium enterprises seeking to implement AI assistance for complex enterprise software, including an analysis of the economic trade-offs between commercial and local approaches.

1.6 Design Overview

The solutions proposed in this thesis span a wide range of architectural choices, reflecting distinct trade-offs between simplicity, privacy, and autonomy. Solution 1 wraps a commercial LLM with retrieval-augmented generation over ERP documentation, providing rapid deployment at the cost of data privacy. Solution 2 grants a terminal-using agent direct access to codebases and databases for complex dependency resolution. Solution 3 fine-tunes a compact vision-language model for domain-specific screenshot understanding, enabling fully local inference. Solution 4 introduces a multi-agent exploration framework that generates training data through autonomous ERP interaction via MCP. The Autonomous Learning chapter then builds on this foundation with progressively self-supervised training methods: MCP distillation from commercial models, environment-grounded rejection fine-tuning (E-GRFT),

self-supervised MCP querying (SSMCQ), multi-agent voting training, Wave Function Collapse self-improvement (WFCSIT), fully autonomous zero-supervision learning (SYS), modular SYS (MSYS), and layered self-improvement with reinforcement learning and tool evolution (LSI). Each method removes an additional layer of human supervision, culminating in systems that discover, learn, and retrain entirely without human intervention.

1.7 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2: Background and Related Work** provides necessary background on transformer architectures, vision-language models, retrieval-augmented generation, preference optimization techniques, and the target ERP systems.
- **Chapter 3: Commercial Model Approaches** details the RAG-enhanced ChatGPT wrapper interface for documentation-based assistance (Solution 1) and the autonomous terminal-based agent for complex dependency resolution (Solution 2).
- **Chapter 4: Solution 3 – Fine-Tuned Vision-Language Model** presents the parameter-efficient fine-tuning approach for SmoVLm, including dataset construction, training procedures, and evaluation results across multiple benchmarks.
- **Chapter 5: Autonomous Learning** introduces the multi-agent exploration and auto-learning system, then progresses through increasingly autonomous training methods: MCP distillation, E-GRFT, SSMCQ, multi-agent voting training, WFCSIT, SYS, MSYS, and LSI, with comparative evaluation across in-distribution and out-of-distribution tasks.
- **Chapter 6: Conclusion** summarizes findings, discusses limitations and security considerations, and outlines future research directions.

Chapter 2

Background and Related Work

This chapter first summarizes the technical foundations that the proposed solutions build upon (Section 2.1–2.1.4), then reviews directly relevant prior work (Section 2.2–2.2.7).

2.1 Technical Foundations

2.1.1 Transformers and Vision-Language Models

The Transformer architecture [2] replaced recurrence with self-attention, enabling parallel sequence processing and effective long-range dependency modeling. Its scaled dot-product attention computes $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$, where queries, keys, and values are linear projections of the input. This architecture, illustrated in Figure 2.1, underpins all models used in this thesis.

Vision-Language Models (VLMs) extend Transformers to jointly process images and text, typically comprising a vision encoder, a language model backbone, and a projection layer mapping visual features into the text embedding space. **SmolVLM2-256M-Video-Instruct** [3], used in this work, is a 256M-parameter VLM fine-tunable on a single 8 GB GPU.

2.1.2 Alignment: From SFT to Preference Optimization

Pre-trained language models predict the next token but do not follow instructions. *Supervised Fine-Tuning* (SFT) [4] bridges this gap by training on (instruction, desired response) pairs with cross-entropy loss, conditioning the model to behave as an assistant.

SFT alone does not reliably eliminate hallucinations or enforce safety constraints. *Reinforcement Learning from Human Feedback* (RLHF) [5] addresses this by training a reward model on human preference judgments, then optimizing the language model against that reward using *Proximal Policy Optimization* (PPO) [6]. *Group Relative Policy Optimization* (GRPO) [7] removes PPO’s separate critic network by computing advantages relative to a group of sampled outputs, reducing memory overhead. *Direct Preference Optimization* (DPO) [8] bypasses reward modeling entirely, optimizing a closed-form objective directly on preference pairs. DPO is used in several fine-tuning experiments in this thesis due to its simplicity and stability.

How Transformers Work: A Step-by-Step Breakdown

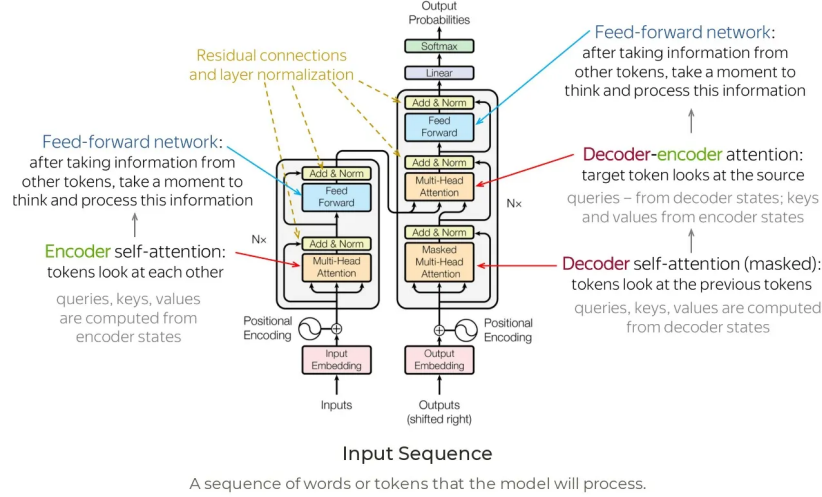


Figure 2.1: Transformer encoder-decoder architecture [2].

2.1.3 Parameter-Efficient Fine-Tuning

Full fine-tuning of large models is often impractical under hardware constraints. *Low-Rank Adaptation* (LoRA) [9] freezes pre-trained weights and injects trainable low-rank matrices B and A such that $W = W_0 + BA$, dramatically reducing the number of trainable parameters. Combined with *quantization*, representing weights at lower numerical precision (e.g., 4-bit), LoRA enables fine-tuning billion-parameter models on consumer GPUs. This QLoRA configuration is used throughout the experiments in this thesis.

2.1.4 Retrieval-Augmented Generation

RAG [1] augments model generation with externally retrieved documents, addressing the static nature of parametric knowledge. The approach used in this thesis combines vector similarity search (dense semantic matching) with keyword-based filtering for exact terminology such as error codes.

2.2 Related Work

2.2.1 Document Understanding Benchmarks

Benchmarks such as **DocVQA** [10] and **ChartQA** [11] evaluate VLM understanding of static documents and charts. This thesis extends evaluation to dynamic ERP software interfaces, a domain that remains under-explored.

2.2.2 Computer-Using and Terminal-Using Agents

Computer-Using Agents (CUA) interact with graphical user interfaces by observing screens and performing actions such as clicking and typing, enabling end-to-end automation across legacy software and dynamic websites. Terminal-Using Agents (TUA) operate within command-line interfaces, where the structured and predictable environment makes them faster and more reliable for technical tasks. **Endless Terminals** [12] demonstrated that fully autonomous pipelines can procedurally generate thousands of diverse terminal tasks without human annotation, enabling significant reinforcement learning (RL) [13]-based training gains. **ZeroGUI** [14] similarly trains GUI automation agents without human annotation by leveraging synthetic data from online environment interactions, achieving competitive performance at near-zero annotation cost. Both approaches are directly relevant to the multi-agent auto-learning framework in this thesis.

2.2.3 Self-Improving AI Systems

Self-play and autonomous learning have achieved superhuman performance in game-playing domains. **AlphaStar** [15] reached grandmaster level in StarCraft II via multi-agent RL with a league of diverse opponents, discovering unexpected strategies without human data. **Stockfish** with NNUE [16] combines traditional search with learned evaluation trained on self-play. **STaR** [17] introduced a bootstrapping approach where a model iteratively generates chain-of-thought rationales, keeps only those leading to verified-correct answers, and retrains on its own successful reasoning traces, eliminating the need for human-annotated rationales. These paradigms, iterative self-improvement through environment interaction and self-generated training signal, inform the multi-agent framework presented in this thesis.

2.2.4 Tool-Using Language Models

Toolformer [18] showed that LLMs can learn tool usage through self-supervised training data generation, using execution-based filtering to keep only those tool calls the model verified as useful. **TALM** [19] introduced iterative self-play bootstrapping for tool-augmented language models, starting from minimal demonstrations and growing training data through the model’s own generations. **ToolLLM** [20] scaled this to 16,000+ real-world APIs with a depth-first search decision tree for complex tool-use planning. All three are relevant to the structured ERP interactions explored here.

2.2.5 Structured Output and Function Calling

The **Model Context Protocol (MCP)** [21] provides an open standard for connecting AI assistants to external tools via JavaScript Object Notation (JSON)-RPC, and is central to the multi-agent framework in this thesis. Several open-source implementations enable MCP-based ERPNext integration, including dedicated ERPNext MCP servers [22] and Frappe assistant frameworks [23], built on top of libraries such as FastMCP [24]. Reliable structured output remains challenging: **StructuredRAG** [25] found 82.55% average JSON generation success, and **StructEval** [26] confirmed that format adherence varies significantly across tasks and output formats.

2.2.6 LLM-Integrated ERP Systems

Niederwieser et al. [27] present an LLM-driven ERP system with graph-based stateful workflows for customer request handling. **FinRobot** [28] proposes AI-native agents for ERP, reporting up to 40% reduction in processing time and 94% fewer errors. **ProcessGPT** [29] fine-tunes GPT models on business process data to generate domain-specific process flows.

2.2.7 Text-to-SQL for Enterprise Databases

Enterprise databases often contain hundreds of tables with complex joins. **Spider 2.0** [30] and **BEAVER** [31] benchmark text-to-SQL on real enterprise schemas, confirming that correct SQL generation remains challenging due to schema complexity and domain-specific naming conventions.

2.3 The Target Domain: Enterprise Resource Planning

This thesis focuses on the complex domain of Enterprise Resource Planning (ERP) systems. It primarily targets the proprietary Mediasoft MS.ERP solution, but also considers open-source standards for comparative context.

2.3.1 The Target System: Mediasoft MS.ERP

Mediasoft MS.ERP is a specialized ERP solution tailored for the Swiss construction and project industries.

- **Scope:** It manages accounting, payroll, HR, and project management.
- **Complexity:** The interface is highly intricate, with a user manual exceeding 1,600 pages. Mastering this domain requires a model to understand not just visual layouts, but the functional dependencies between nested user interface (UI) elements.

2.3.2 Open Source ERP for multiagent exploration: ERPNext

ERPNext [32] is a 100% fully open-source enterprise resource planning system, making it suitable for ERP research and experimentation involving autonomous agents.

- **Architecture:** Built on the Frappe Framework (a full-stack web framework using Python and JavaScript), ERPNext runs entirely in the browser, making it highly customizable and developer-friendly.
- **Licensing:** Unlike "open-core" competitors like Odoo that paywall enterprise features, ERPNext is 100% open source under the GNU General Public License (GPL). This ensures all modules, from Accounting to Manufacturing, are accessible without per-user licensing fees.
- **Unified Suite:** It consolidates core business functions into a single monolithic dashboard, replacing disjointed toolsets (e.g., spreadsheets or standalone accounting apps) with a unified system for Accounting, HR, CRM, Inventory, and Project Management.

Chapter 3

Commercial Model Approaches

3.1 Solution 1: ChatGPT Wrapper Interface

The first solution consists of a web-accessible interface built around ChatGPT, designed to assist users by enriching their queries with contextually relevant information extracted from ERP documentation. Rather than interacting directly with the ERP system, this approach focuses on leveraging existing knowledge artifacts to guide users through system functionality, configuration, and usage.

3.1.1 Documentation Preprocessing

A critical prerequisite for this solution is the preprocessing of documentation to ensure it can be efficiently and reliably consumed by a large language model. All Mediasoft documentation was therefore transformed into a unified, text-only format optimized for analysis by ChatGPT. This process covered a wide range of source materials, including PDF, Word, Excel, and Visio documents, as well as instructional videos and GLPI support tickets.

Non-textual content was systematically converted into textual representations. Images were replaced by their extracted text using Optical Character Recognition (OCR), while videos were substituted with full transcripts of their audio content. Visio diagrams were transformed into a structured, Mermaid-like JSON format that preserves graphical dependencies and relationships in a textual form.

GLPI tickets were then reviewed by a large language model agent, which filtered out information irrelevant to support technicians and condensed the remaining content into a structured knowledge base. To comply with data protection requirements, all personal data, such as names, phone numbers, and addresses, was removed using dedicated Python scripts.

Special attention was given to abbreviation handling in order to reduce the risk of hallucinations. All abbreviations were explicitly expanded upon every occurrence. For example, “MS” was consistently replaced by “MS (Mediasoft)”, while terms such as “sd100” were expanded to “sd100 (salary declaration, per SwissDec: the Swiss standard for electronic payroll)”. Once cleaned and normalized, the text corpus was segmented into chunks of approximately 5,000 words to support efficient indexing and retrieval by the language model.

3.1.2 RAG and Keyword Search

The processed documentation corpus serves as the foundation for a Retrieval-Augmented Generation (RAG) pipeline. Each text chunk is embedded using a language model and stored in a vector database, enabling semantic similarity searches over the documentation. In parallel, a keyword-based search mechanism is implemented to ensure that exact matches for technical terms, identifiers, and domain-specific vocabulary are not overlooked.

When a user submits a query, both the semantic retrieval process and the keyword search are executed simultaneously. The resulting candidate passages are then merged and ranked to identify the most relevant contextual information. These selected excerpts are passed as context to the ChatGPT Response API, allowing the model to generate answers that are both precise and grounded in Mediasoft's internal documentation. By combining semantic and lexical retrieval, this hybrid approach improves robustness, reduces hallucinations, and ensures that both conceptual relevance and exact terminology are respected.

3.1.3 Images RAG and Image Search

We missed the opportunity to explore image search, image-based RAG, and multimodal RAG, as we focused on a text-only knowledge base derived from OCR outputs in order to enable faster deployment.

3.1.4 Database or Mock Database Integration

For organizations willing to provide access to system data, typically through a database, the solution can be extended through Model Context Protocol (MCP) connections. This integration enables the language model to query actual database schemas and inspect table structures and field relationships. As a result, the model gains a deeper understanding of the underlying data structures and software architecture, allowing it to provide more accurate and context-aware guidance, particularly for questions related to data formats and inter-table dependencies. For companies that want to benefit from large language models learning the structure of their datasets and understanding key fields, while still maintaining strict data privacy, it is possible to use a mock database.

Using a mock database mirrors the schema, relationships, and data types of the real system without containing sensitive information. This allows the LLM to learn how the data is organized, how queries should be formed, and how workflows operate, all while ensuring that no confidential or regulated data is exposed. In this way, organizations can safely leverage AI capabilities for automation, analytics, and decision support without compromising privacy or compliance.

3.1.5 Limitations

Despite its practical advantages, the RAG-based approach presents several inherent limitations. Its effectiveness is strongly dependent on the quality of the underlying documentation; incomplete, outdated, or inaccurate documentation can significantly degrade performance, a common issue for smaller ERP systems. Furthermore, the retrieval mechanism struggles with queries that require a deep understanding of complex, multi-module workflows or implicit dependencies between system components. Finally, when documentation is anonymized to protect sensitive data, version-specific information is lost, meaning that a separate retrieval database must be maintained for each software version deployed across clients.

This per-version requirement increases storage and maintenance overhead, as each database must be independently updated whenever its corresponding version receives patches or configuration changes.

3.1.6 Internship at Wiven

Both the Wiven web application and the terminal-using agent have since been deployed in production for the after-sales team, business analysts, and support technicians, where they have demonstrated reliable day-to-day performance in assisting with documentation retrieval, ticket resolution, and ERP-related inquiries. LangGraph-based automation workflows were developed for Mediasoft clients.

3.2 Solution 2: Terminal-Using Agent

The second solution adopts an agentic architecture capable of autonomous, multi-step reasoning and exploration. This approach is implemented using a terminal-using agent (TUA) conceptually similar to Claude Code and is designed to address queries that exceed the capabilities of static documentation-based retrieval.

3.2.1 Agent Capabilities

Unlike the ChatGPT wrapper interface, the terminal-using agent is able to conduct iterative searches and refine its strategy based on intermediate findings. It can follow chains of references across documentation to understand how different ERP modules interact and depend on one another. When integrated with development environments, the agent can also inspect codebases, database schemas, and configuration files, enabling a deeper and more precise understanding of system behavior.

3.2.2 Query Resolution Process

When presented with a complex user query, the agent first analyzes the information requirements implied by the request. It then formulates an initial search strategy and executes queries against available documentation or code repositories. Based on the results, the agent may iteratively refine its approach, follow cross-references, or explore related modules. Once sufficient evidence has been gathered, it synthesizes a response that explicitly accounts for the discovered dependencies and interactions.

3.2.3 Security Considerations

The deployment of a terminal-using agent raises significant security challenges. To mitigate risks, the agent must operate within a strictly isolated, containerized environment, such as Docker or a dedicated external server, in order to prevent unauthorized access to host systems. Its permissions must be carefully constrained, typically to read-only operations on documentation and code, or fully isolated execution, since unrestricted terminal access could otherwise result in destructive actions, including accidental deletion of critical data. In addition, the system must account for prompt injection risks, whereby malicious users attempt to manipulate the agent into performing unauthorized operations through carefully crafted inputs.

3.2.4 Deployment

In practice, the terminal-using agent is deployed as a Claude Code wrapper using `claude-code-webui` [33], hosted on an external server and accessed through a web interface. This deployment strategy offers two main advantages. First, it provides an intuitive interaction model for employees who may be unfamiliar with command-line interfaces or uncomfortable using local terminals. Second, hosting the agent on an isolated external server significantly reduces the risk of catastrophic failures caused by destructive commands, such as inadvertent deletion of documentation or system files.

Chapter 4

Solution 3: Fine-Tuned Vision-Language Model

The third solution adapts a compact vision-language model for domain-specific understanding through comprehensive fine-tuning experiments spanning multiple training paradigms.

4.1 Model Selection Rationale

SmolVLM2-256M-Video-Instruct was selected for several reasons:

Criterion	Description
Size Efficiency	At only 256M parameters, the model can be fine-tuned on a single 8GB GPU, making it accessible in resource-constrained environments.
Instruction Tuning	The model's instruction-following capabilities provide a strong foundation for task-specific adaptation and user-facing assistance.
Multimodal Capability	Native vision-language integration enables understanding of interface screenshots without preprocessing.

Table 4.1: Model selection criteria for SmolVLM2-256M-Video-Instruct.

4.2 Training Paradigms

I investigate three distinct training paradigms, each with different trade-offs:

Paradigm	Description
Full Fine-Tuning	Updates all model parameters, providing maximum adaptation capacity but requiring more memory and risking forgetting of general capabilities. Uses a lower learning rate (5×10^{-6}).
QLoRA-Based	Parameter-efficient fine-tuning using Low-Rank Adaptation with 4-bit NF4 quantization. See Table 4.3 for hyperparameters.
Quantized Training	4-bit quantization using the NF4 data type with double quantization to reduce memory footprint.

Table 4.2: Training paradigms overview.

Parameter	Value
Rank (r)	32
Alpha (α)	64
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj, lm_head
Dropout	0.05
Learning Rate	1×10^{-5} (SFT), 1×10^{-6} (DPO)

Table 4.3: QLoRA hyperparameters.

4.3 Training Objectives

I employ multiple complementary training objectives:

Objective	Description
SFT on MCQ	Multiple-choice questions test the model’s understanding of interface functionality and ERP workflows. The model learns to select the correct answer from four options.
SFT on Chosen	Training on high-quality “chosen” responses from the DPO dataset to improve response generation quality without the contrastive DPO objective.
DPO	Preference pairs distinguish between detailed, accurate responses (chosen) and vague or incomplete alternatives (rejected). Encourages comprehensive, user-helpful responses.

Table 4.4: Training objectives.

4.4 Data Sources

Training data was generated using multiple large language models to ensure diversity. (Versions from September 2025 were used; most models are already outdated and underperform compared to their most recent versions.)

Model	Role
Gemini	MCQ and DPO pair generation
Nova Pro	Alternative data generation
Claude	Procedure-specific question generation

Table 4.5: Data source models.

4.5 Balancing Specialization and Generalization

A key design consideration is avoiding catastrophic forgetting of general vision-language capabilities. This is addressed through:

Strategy	Description
Multiple Paradigms	Comparing full fine-tuning vs. LoRA to understand the trade-off between adaptation capacity and knowledge preservation.
Benchmark Monitoring	Evaluation on general benchmarks (OCRBench, DocVQA, ChartQA) alongside domain-specific tasks to detect capability degradation.
Response Alignment	ROUGE scores and logprobs are tracked to ensure models are aligned to the chosen responses for user-facing assistance.
Conservative Hyperparameters	Low learning rates to reduce the risk of overwriting general knowledge.

Table 4.6: Strategies for balancing specialization and generalization.

4.6 Dataset Construction for VLM Fine-Tuning

4.6.1 User Manual Processing

The Mediasoft MS.ERP user manual (“Aide-General-MS-ERP.docs”) is used to generate the finetuning dataset and evaluation MCQ. The document contains 1,645-pages and 967 images. All 967 embedded images were extracted and saved with positional meta- data linking them to surrounding text.

Image Selection and Filtering

Interface screenshots were extracted from the user manual with the following preprocessing:

- **Size Filtering:** Images smaller than 400×400 pixels were excluded to avoid minor interface icons and decorative elements.
- **Context Extraction:** For each image, approximately 150 lines of surrounding text (both preceding and following) were extracted to provide functional context.

- **Position Recording:** Image positions within the document were recorded to support sequential reasoning about workflows.

4.6.2 Automated Question Generation

Question-answer pairs were generated using Amazon Bedrock (Nova Pro) and Google Gemini, orchestrated through the CrewAI agent library. For each image, DPO or MCP question-answer pairs were generated. The questions are either descriptive: The model must describe the interface. Or about Mediasoft software utilisation: The agent use the 150 lines of text before and after the image to create a question about Mediasoft software usage. **Procedures:** Procedures datasets Generate MCQ generated from OneNote images and text from 2 different Business analysts. **Claudette:** A text-only multiple-choice question dataset generated by Claude from the Mediasoft MS.ERP user manual. It evaluates whether models can answer domain-specific questions without image input, testing software comprehension independently of visual understanding.

Questions (DPO Format)

Descriptive questions target interface understanding:

```

1 {
2   "image_name": "image_001.png",
3   "prompt": "What interface elements are visible
4             and how are they organized?",
5   "chosen": "[Detailed 150+ word description of
6             interface elements, their organization,
7             and functional relationships]",
8   "rejected": "[Brief, vague description lacking
9               specific details or relationships]",
10  "type": "descriptive"
11 }
```

Listing 4.1: DPO entry structure

The chosen response is, on average, 6× longer than the rejected response, ensuring clear quality differentiation.

Functional Questions (MCQ Format)

Functional questions test operational understanding:

```

1 {
2   "image_name": "image_001.png",
3   "type": "mcq",
4   "mcq": {
5     "question": "What happens when a user selects
6                 a category in the left panel?",
7     "options": {
8       "A": "[Plausible but incorrect answer]",
9       "B": "[Plausible but incorrect answer]",
10      "C": "[Correct answer with functional insight]",
11      "D": "[Plausible but incorrect answer]"
12    },
13     "correct_answer": "C",
14     "explanation": "[Visual evidence for correct answer]"
15   }
```

```

15 }
16 }

```

Listing 4.2: MCQ entry structure

Distractor options (incorrect answers) are designed to be plausible, representing common misconceptions or superficially similar functionality.

4.6.3 Dataset Statistics

The constructed datasets comprise (all datasets use a 90%/10% train/test split):

Table 4.7: Dataset composition

Dataset	Generation Model	Pairs
DPO	Gemini	1,840
DPO	Nova Pro	1,868
MCQ	Gemini	3,680
MCQ	Nova Pro	3,736
MCQ (text-only)	Claudette(Claude)	1,099
MCQ	Procedure1	100
MCQ	Procedure2	100

MCQ position bias mitigation. Position bias in multiple-choice questions is a well-documented phenomenon: Attali and Bar-Hillel [34] showed that human test designers disproportionately place the correct answer at positions B and C, and that test-takers exploit this bias. LLMs may reproduce this behavior, exhibiting systematic preferences for certain answer positions due to patterns absorbed during pre-training on human-generated content. Additionally, position bias in LLMs can arise from tokenization artifacts, where the token representations of option labels (A, B, C, D) carry unequal prior probabilities that distort the model’s selections independently of the answer content [35]. To mitigate these effects, the SmolVLM MCQ datasets are reshuffled so that the correct answer is uniformly distributed across positions A, B, C, and D (25% each). The other MCQ datasets (Chapter 5) were already balanced during generation, with no position exceeding 30% of correct answers, ensuring that no positional shortcut can inflate accuracy.

4.7 Training Procedure

Training experiments were conducted on two hardware configurations: an NVIDIA GeForce RTX 4060 Laptop (8GB VRAM) for debugging, and the Izar SLURM cluster for running the comprehensive pipeline in parallel. Over 40 training configurations were evaluated; Table 4.8 summarizes the key hyperparameters.

Table 4.8: Training hyperparameters for full fine-tuning and QLoRA configurations.

Parameter	Full Fine-Tuning	QLoRA
Trainable params	256M (all)	LoRA adapters only
LoRA rank / α / dropout	—	32 / 64 / 0.05
Target modules	All	q/k/v/o/gate/up/down_proj, lm_head
Learning rate (SFT)	5×10^{-6}	1×10^{-5}
Learning rate (DPO)	1×10^{-7}	1×10^{-6}
Batch size	16	16
Epochs	101	101
Optimizer	AdamW, cosine sched., 10% warmup	AdamW 8-bit, cosine sched.
Precision	bfloat16 + grad. checkpointing	4-bit NF4, double quant., bf16 compute
DPO β	0.1	0.1
DPO max length	1024 (prompt: 512)	1024 (prompt: 512)

Training Strategies Evaluated

Table 4.9: Summary of training strategies

Category	Strategy	Description
Full FT Benchmark	full_sft_ocrbench	SFT on OCRBench, all params
	full_sft_docvqa	SFT on DocVQA, all params
	full_sft_chartqa	SFT on ChartQA, all params
Full FT ERP	full_sft_chosen_nova	SFT on Nova chosen responses
	full_sft_chosen_gemini	SFT on Gemini chosen responses
	full_sft_qcm_nova	SFT on Nova MCQ dataset
	full_sft_qcm_gemini	SFT on Gemini MCQ dataset
	full_sft_qcm_procedure1	SFT on Procedure 1 MCQ
	full_sft_qcm_procedure2	SFT on Procedure 2 MCQ
Full FT DPO	full_dpo_nova	DPO on Nova dataset, all params
	full_dpo_gemini	DPO on Gemini dataset, all params
QLoRA Benchmark	qlora_sft_ocrbench	SFT on OCRBench with QLoRA
	qlora_sft_docvqa	SFT on DocVQA with QLoRA
	qlora_sft_chartqa	SFT on ChartQA with QLoRA
QLoRA ERP	qlora_sft_qcm_nova	SFT on Nova MCQ with QLoRA
	qlora_sft_qcm_gemini	SFT on Gemini MCQ with QLoRA
	qlora_qcm_procedure2	SFT on Procedure 2 MCQ with QLoRA
QLoRA DPO	qlora_dpo_qcm_nova	DPO on Nova MCQ with QLoRA
	qlora_dpo_qcm_gemini	DPO on Gemini MCQ with QLoRA

4.8 Evaluation

This section presents the experimental evaluation of the VLM fine-tuning, comparing configurations across multiple dimensions including accuracy, deployment complexity, and resource requirements.

4.8.1 Evaluation Methodology

Benchmark Datasets

The benchmark datasets serve two purposes. First, they act as a sanity check to ensure that training is functioning correctly, with models eventually overfitting and training accuracy steadily increasing. Secondly, they are used to evaluate forgetting and the loss of capabilities in the trained VLMs. The evaluation covers both general vision-language benchmarks and domain-specific tasks. A detailed description of each dataset is provided in the appendix.

- **OCRBench** [36]: Text recognition and document-level OCR tasks (1,000 question-answer pairs).
- **DocVQA** [10]: Document visual question answering (1,200 examples used).
- **ChartQA** [11]: Chart and data visualization understanding.
- **ERP QCM**: The domain-specific multiple-choice questions on ERP interfaces.
- **ERP DPO**: Preference alignment evaluation on ERP tasks.

4.8.2 Evaluation Metrics

The evaluation framework employs multiple complementary metrics to assess different aspects of model performance. Each metric targets specific capabilities relevant to vision-language understanding and ERP domain adaptation. The corresponding evaluation results are presented in the figures of the following pages.

Multiple Choice Question (MCQ) Accuracy

For MCQ evaluation, a two-tier accuracy system is implemented to handle the variability in model response formats:

Strict Accuracy: The model’s response must begin with the correct answer letter (A, B, C, or D) as its very first character. This metric ensures the model follows the expected output format and provides unambiguous answers. For a response to be counted as strictly correct:

$$\text{Strict Match} = \mathbb{1}[\text{first_char}(r) = c]$$

where r is the model response and c is the correct answer letter.

Lenient Accuracy: When strict matching fails, an answer extraction algorithm searches the response for the correct letter using pattern matching. This handles cases where the model provides explanations before stating the answer (e.g., “The correct answer is B because...”). The extractor uses both English and

French patterns to accommodate multilingual responses, recognizing phrases like “La réponse est” or “The answer is”.

Both metrics are reported to provide insight into both format compliance and reasoning capability.

ROUGE Scores

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) [37] measures the overlap between generated responses and reference answers. Three variants are computed:

ROUGE-1 measures unigram overlap between the prediction and reference:

$$\text{ROUGE-1} = \frac{|\text{unigrams}(r) \cap \text{unigrams}(c)|}{|\text{unigrams}(c)|}$$

ROUGE-2 measures bigram overlap, capturing phrase-level similarity:

$$\text{ROUGE-2} = \frac{|\text{bigrams}(r) \cap \text{bigrams}(c)|}{|\text{bigrams}(c)|}$$

ROUGE-L measures the longest common subsequence (LCS), capturing sentence-level structure:

$$\text{ROUGE-L} = \frac{\text{LCS}(r, c)}{|c|}$$

I report the F1 score (harmonic mean of precision and recall) for each variant. ROUGE-L serves as the primary quality metric for DPO-trained models, as it best captures whether responses maintain the detailed, comprehensive style of the chosen training examples.

Average Normalized Levenshtein Similarity (ANLS)

ANLS is the standard metric for document VQA tasks [10], accounting for minor OCR errors and formatting variations. For each prediction-reference pair, the Normalized Levenshtein Similarity (NLS) is computed:

$$\text{NLS}(p, r) = \max\left(1 - \frac{\text{edit_dist}(p, r)}{\max(|p|, |r|)}, 0\right)$$

where `edit_dist` is the Levenshtein (edit) distance. A threshold of 0.5 is applied: if NLS falls below this threshold, the score is set to zero. For samples with multiple valid answers, the maximum NLS across all references is used:

$$\text{ANLS} = \frac{1}{N} \sum_{i=1}^N \max_{r \in R_i} \text{NLS}(p_i, r)$$

This metric is used for OCRBench, DocVQA, and ChartQA evaluations.

Containment-Based Accuracy

For benchmarks where exact matching is too strict, unidirectional containment matching is used. A prediction is considered correct if it contains the ground truth answer (after normalization). It was chosen not to use official benchmarks because the aim is to make a user-facing assistant and to be judged with more leniency:

$$\text{Match} = \mathbb{1}[\text{normalize}(c) \subseteq \text{normalize}(p)]$$

Normalization includes lowercasing, punctuation removal, and whitespace normalization. Importantly, only *unidirectional* matching is used, checking if the ground truth appears in the prediction, not vice versa. This prevents false positives from very short predictions that might coincidentally appear within longer ground truths.

The system also handles number-word equivalence: “1” matches “one”, “2” matches “two”, etc., for numbers 0–20.

Numeric Comparison with Tolerance

For ChartQA, where answers often involve numerical values extracted from charts, numeric comparison with a 5% relative tolerance is applied:

$$\text{Numeric Match} = \mathbb{1}[|p_{\text{num}} - c_{\text{num}}| \leq 0.05 \cdot |c_{\text{num}}| + 0.01]$$

The additive constant 0.01 handles the edge case of zero-valued ground truths. Numbers are extracted from text using regular expressions that handle commas, percentage signs, and decimal points

logprobsMetrics

Metric	Meaning
logprob_chosen_pp1	Perplexity of chosen responses. Lower = model is more confident about the good response. Calculated as $\exp(-\text{avg_logprob})$
logprob_rejected_pp1	Perplexity of rejected responses. Higher = model is less confident about the bad response (good sign)
logprob_acc	Preference accuracy = % of samples where $\text{logprob}(\text{chosen}) > \text{logprob}(\text{rejected})$. Higher = better alignment
logprob_margin	Average margin = $\text{avg_logprob}(\text{chosen}) - \text{avg_logprob}(\text{rejected})$. Positive = model prefers chosen. Larger positive = stronger preference

Table 4.10: Model Evaluation Metrics

4.8.3 VLM Fine-Tuning Results

In the following figures, the vertical axis represents accuracy as a percentage, while the horizontal axis corresponds to gradient steps. A gradient step denotes a single parameter update performed by the optimizer after computing the loss gradient over one batch of training data; the total number of gradient steps thus reflects how far training has progressed. Unless otherwise specified, results are reported on the full dataset (train + test).

Important caveat. The DPO evaluation results presented in this section should not be considered reliable. The DPO training pipeline suffered from implementation issues, including image preprocessing incompatibilities with SmolVLM2 and potential timeout, that could not be fully diagnosed.

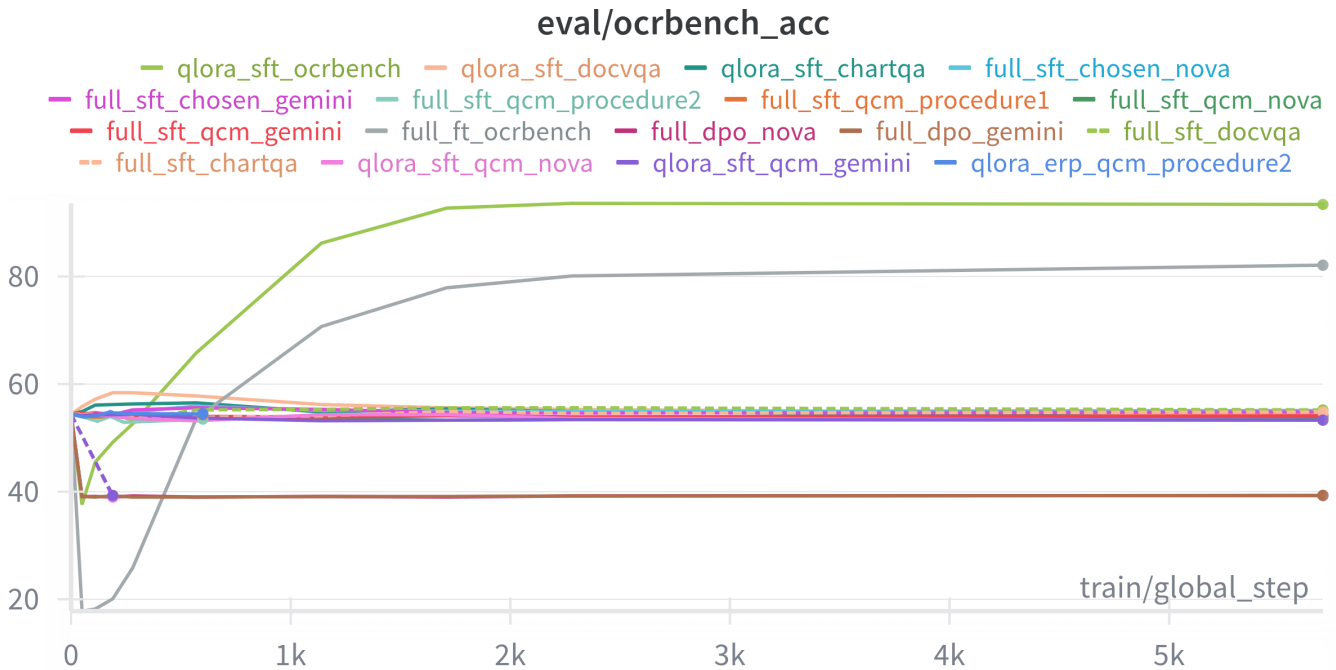


Figure 4.1: OCRBench accuracy (y-axis: accuracy in %, x-axis: gradient steps).

OCRBench Accuracy. Figure 4.1 presents the evaluation accuracy on the OCRBench benchmark. The `qlora_sft_ocrbench` configuration, which is trained directly on OCRBench, achieves the highest accuracy, reaching approximately 93% by the end of training on the full dataset (train + test).

The majority of other SFT configurations cluster within a narrow range of 54–57% accuracy, remaining close to baseline performance. This indicates that SFT generally preserves OCR capabilities and does not induce rapid catastrophic forgetting.

Notably, the SFT model trained on the same dataset exhibits an initial performance drop before gradually recovering and ultimately outperforming the other SFT configurations.

The ERP-specific training methods largely maintain OCR performance near the baseline, suggesting that domain-specific fine-tuning does not severely degrade general OCR abilities. An exception is DPO training, which results in a noticeable degradation in OCR performance. This behavior may indicate

that DPO negatively affects the model’s OCR representations; however, it cannot be ruled out that this degradation is partially due to implementation or configuration errors in the DPO training pipeline.

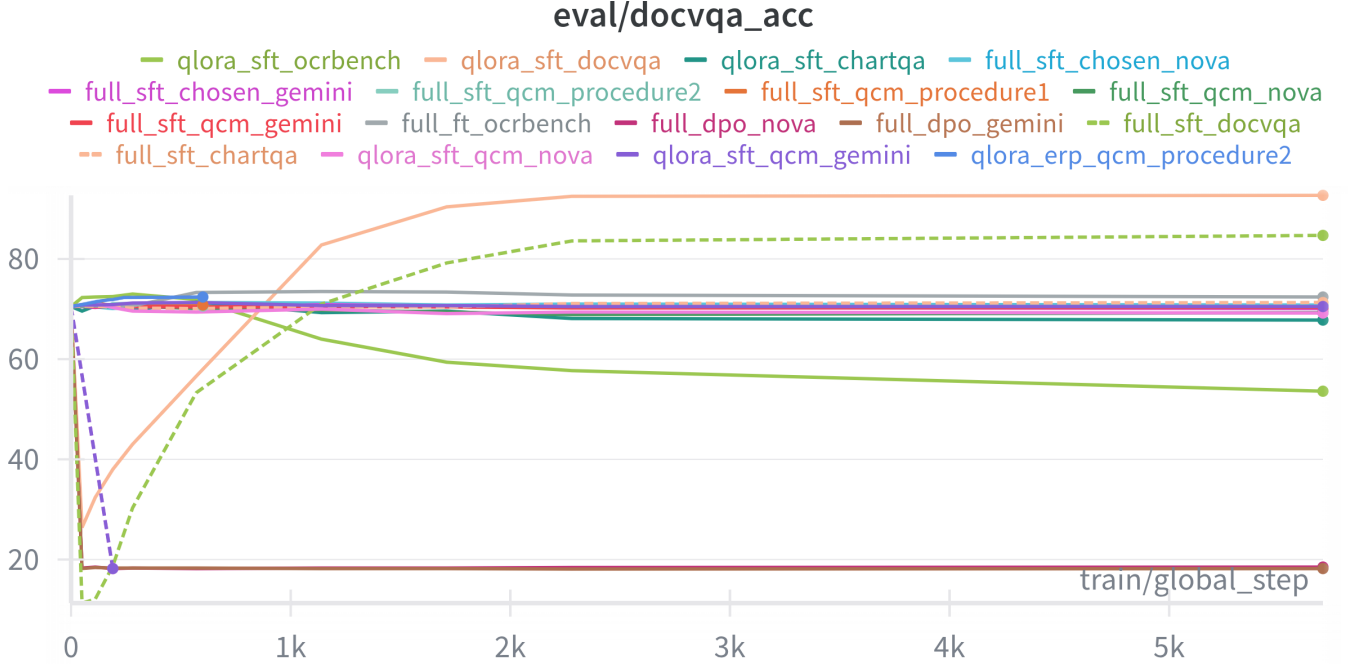


Figure 4.2: DocVQA accuracy (y-axis: accuracy in %, x-axis: gradient steps).

DocVQA Accuracy. Figure 4.2 presents the evaluation accuracy on the DocVQA benchmark, where behavior consistent with the ocrBench results is observed. The qLora_sft_docvqa configuration achieves the highest accuracy, reaching approximately 90% by the end of training, while full_sft_docvqa reaches 86%. Most other configurations maintain performance in the 70–75% range, near the baseline.

A critical observation is the severe degradation exhibited by the DPO methods: full_dpo_nova and full_dpo_gemini collapse to approximately 20% accuracy. This suggests that DPO training may catastrophically harm document understanding capabilities or alter answer formatting in a way that is penalized by the benchmark (e.g., shifting towards verbose assistant-style responses).

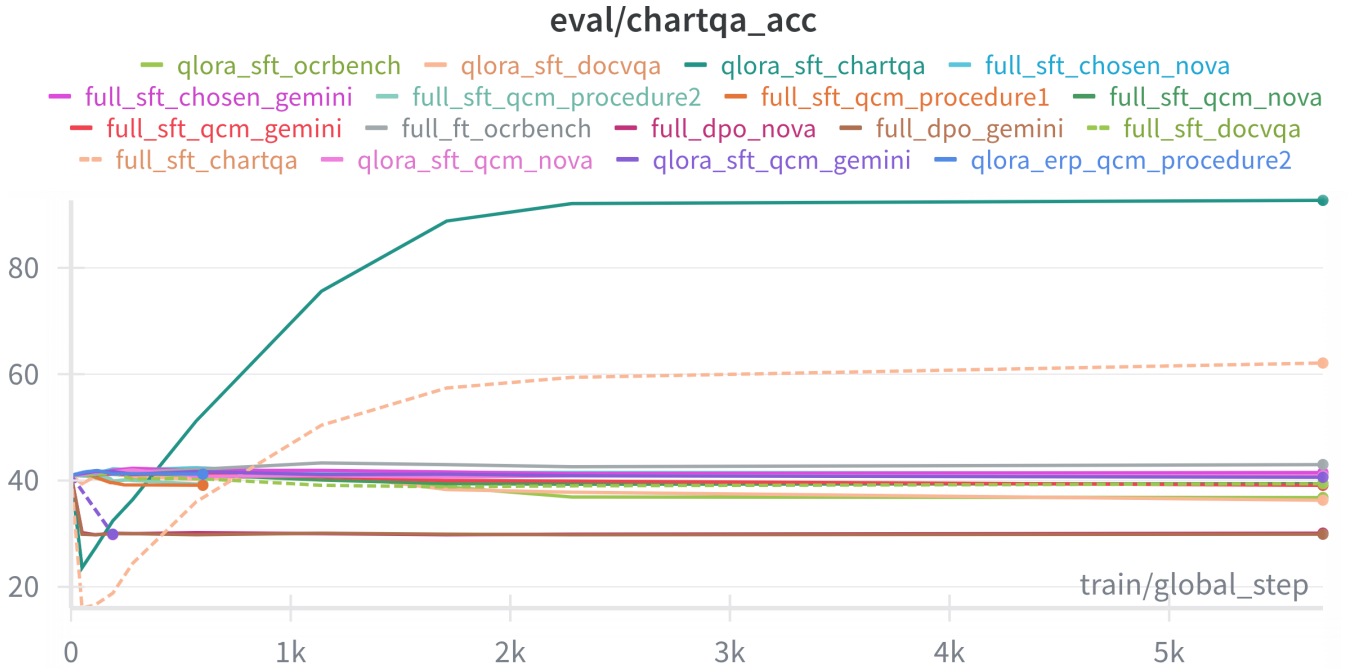


Figure 4.3: ChartQA accuracy (y-axis: accuracy in %, x-axis: gradient steps).

ChartQA Accuracy. Figure 4.3 presents the evaluation accuracy on the ChartQA benchmark, where behavior consistent with the OCRBench and DocVQA results is observed. The `qlora_sft_chartqa` configuration achieves the highest accuracy, while most other SFT configurations maintain performance near the baseline range.

As with the other benchmarks, the DPO methods exhibit significant degradation, with `full_dpo_nova` and `full_dpo_gemini` collapsing to low accuracy levels. This consistent pattern across OCRBench, DocVQA, and ChartQA reinforces the hypothesis that DPO training may catastrophically harm visual understanding capabilities or alter answer formatting in ways that are penalized by these benchmarks.

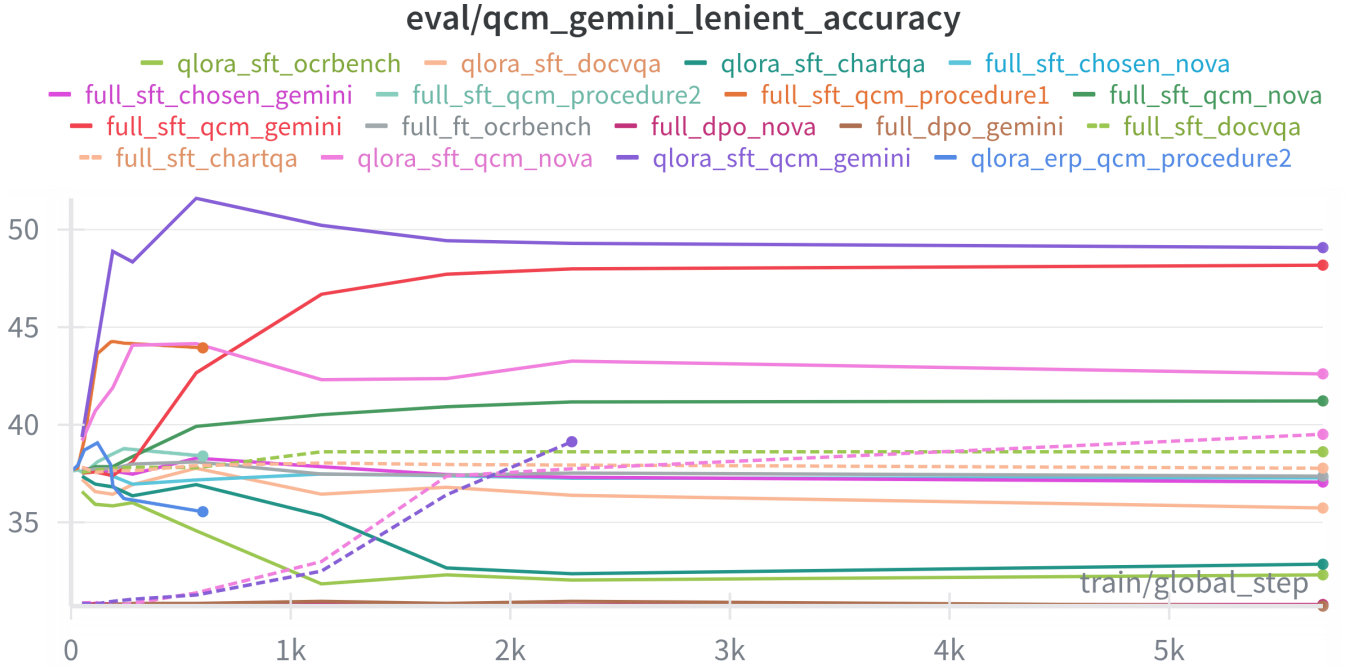


Figure 4.5: QCM results, Gemini (y-axis: accuracy in %, x-axis: gradient steps).

Gemini QCM Accuracy. Figure 4.5 reports the evolution of lenient QCM accuracy on the Gemini evaluation set. The QLoRA SFT model trained on QCM data with the Nova backbone achieves the highest performance, peaking above 50% accuracy early in training and remaining consistently above all other configurations. In contrast, the corresponding full SFT QCM Nova model converges to a lower plateau around 48%. Models trained on preference-style user–assistant data (chosen Nova and chosen Gemini) outperform benchmark-trained models across training steps, while OCRBench-, DocVQA-, and ChartQA-trained models remain close to chance-level performance. These results indicate that most gains stem from improved adaptation to the QCM format, with preference-based supervision providing a smaller but consistent additional benefit, suggesting limited transfer of ERP-related knowledge.

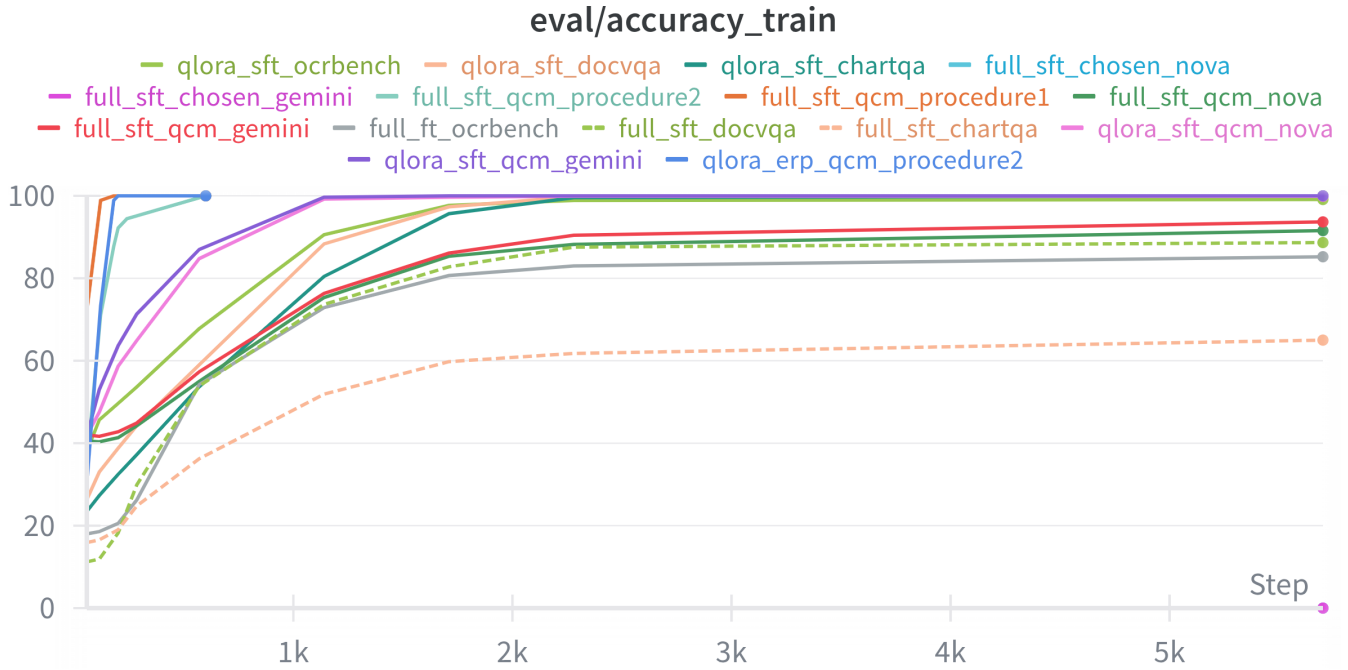


Figure 4.6: Training accuracy (y-axis: accuracy in %, x-axis: gradient steps).

Training Accuracy. Figure 4.6 presents the training accuracy across all configurations. Most methods converge rapidly to 90–100% training accuracy within the first 1–2k steps, demonstrating successful fitting to the training data. The `full_sft_chartqa` configuration stabilizes around 65% training accuracy; however, QLoRA SFT converges to the top, suggesting that full SFT can damage underlying capabilities and reasoning of the model, while QLoRA changes fewer parameters, keeping more capabilities intact. The high training accuracy across most configurations contrasts with the more modest test accuracies, indicating overfitting to the training data.

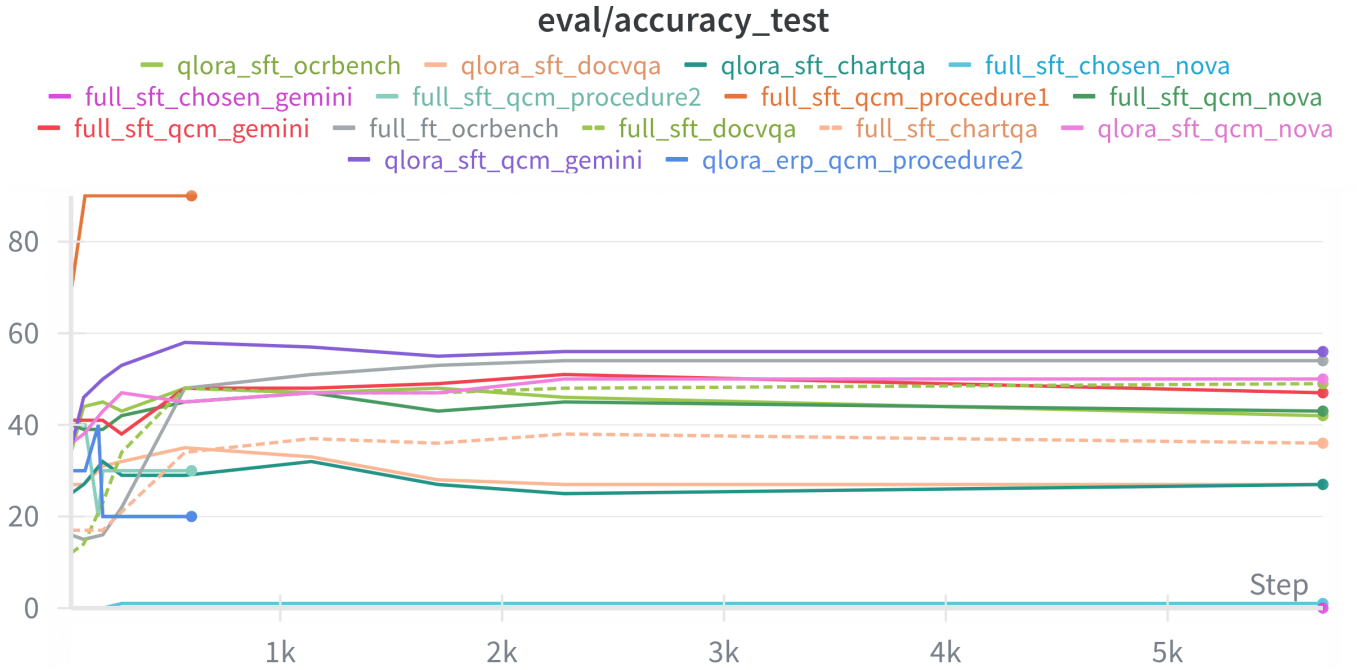


Figure 4.7: Test accuracy (y-axis: accuracy in %, x-axis: gradient steps).

Test Accuracy. Figure 4.7 presents the test accuracy across all configurations, revealing the generalization gap between training and evaluation performance. The `full_sft_chosen_gemini` configuration shows an initial spike to approximately 85% accuracy but then drops to around 50%, indicating overfitting. Most configurations stabilize between 40–60% test accuracy. Several configurations show initial volatility before stabilizing. Note that configurations trained on chosen preference datasets for user-facing assistants report 0% test accuracy, as these models are not trained on MCQ data and therefore do not have train/test accuracy values stored. The significant gap between training accuracy (near 100%) and test accuracy (40–60%) underscores the importance of regularization and parameter-efficient fine-tuning methods for domain adaptation.

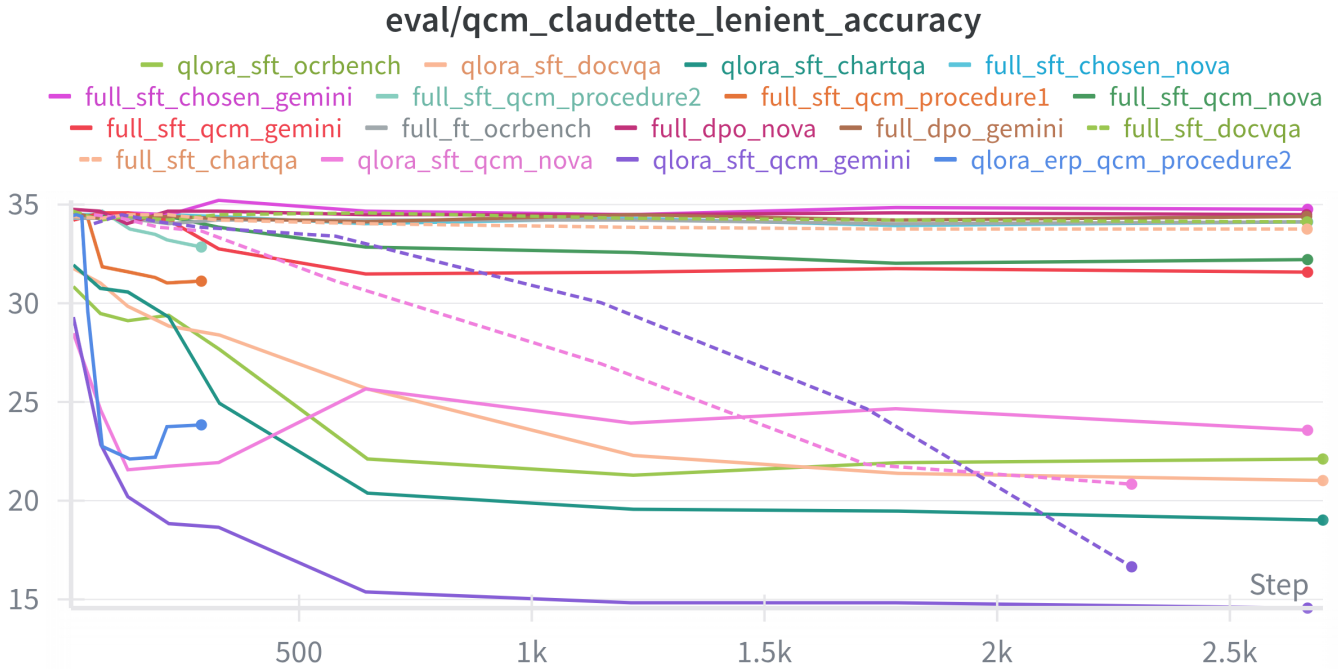


Figure 4.8: Claudette text-only MCQ accuracy (y-axis: accuracy in %, x-axis: gradient steps).

Claudette Text-Only MCQ Accuracy. Figure 4.8 presents the evaluation accuracy on the Claudette benchmark, a text-only multiple-choice question dataset generated by Claude for evaluating ERP software understanding without visual input. All configurations start near the baseline of approximately 35% accuracy. Critically, no configuration shows meaningful improvement over training; most either remain flat or exhibit degradation. Several configurations, including `full_sft_chosen_nova`, `full_dpo_nova`, and `full_dpo_gemini`, show significant accuracy drops to 20–22%, while the best-performing methods merely maintain baseline accuracy around 32–35%. This lack of improvement on the Claudette benchmark indicates that the models do not learn transferable knowledge about Mediasoft ERP software. Despite achieving improvements on vision-based benchmarks and training-set MCQs, the models fail to acquire understanding of ERP functionality that generalizes to text-only questions. This suggests that the training primarily teaches visual pattern recognition rather than deep understanding of ERP workflows and operations.

ROUGE Score Analysis

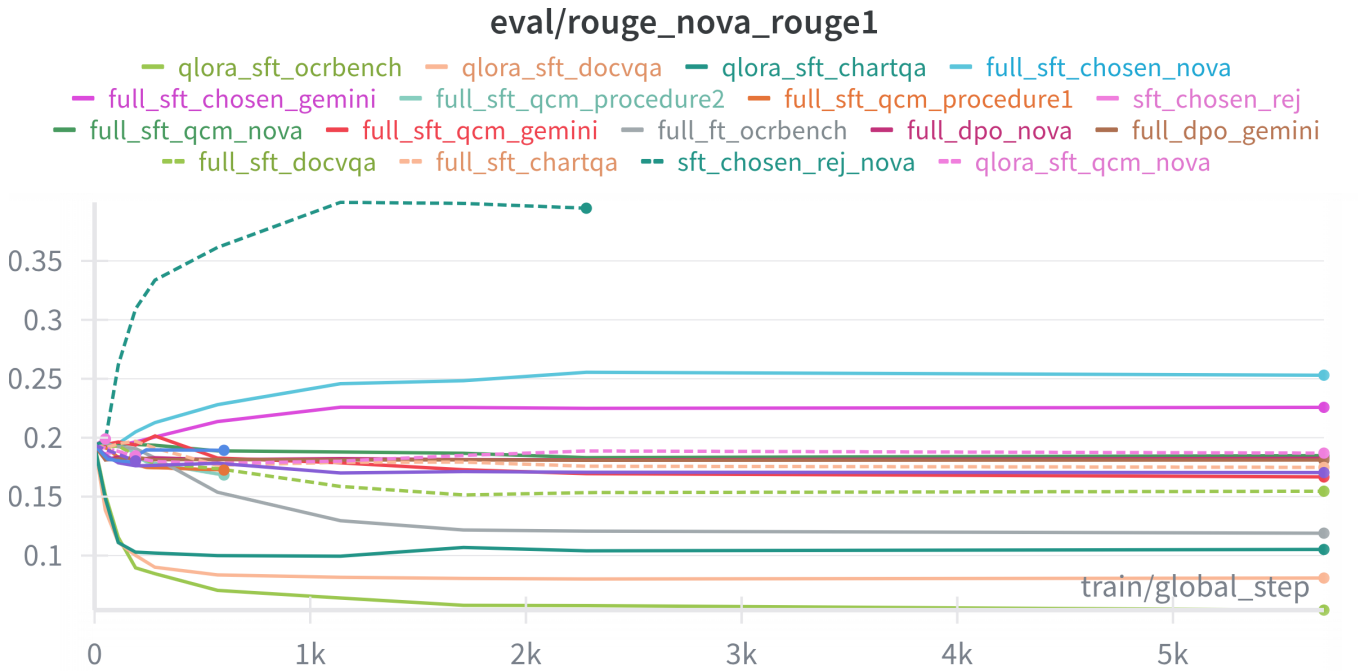


Figure 4.9: ROUGE-1 (Nova)

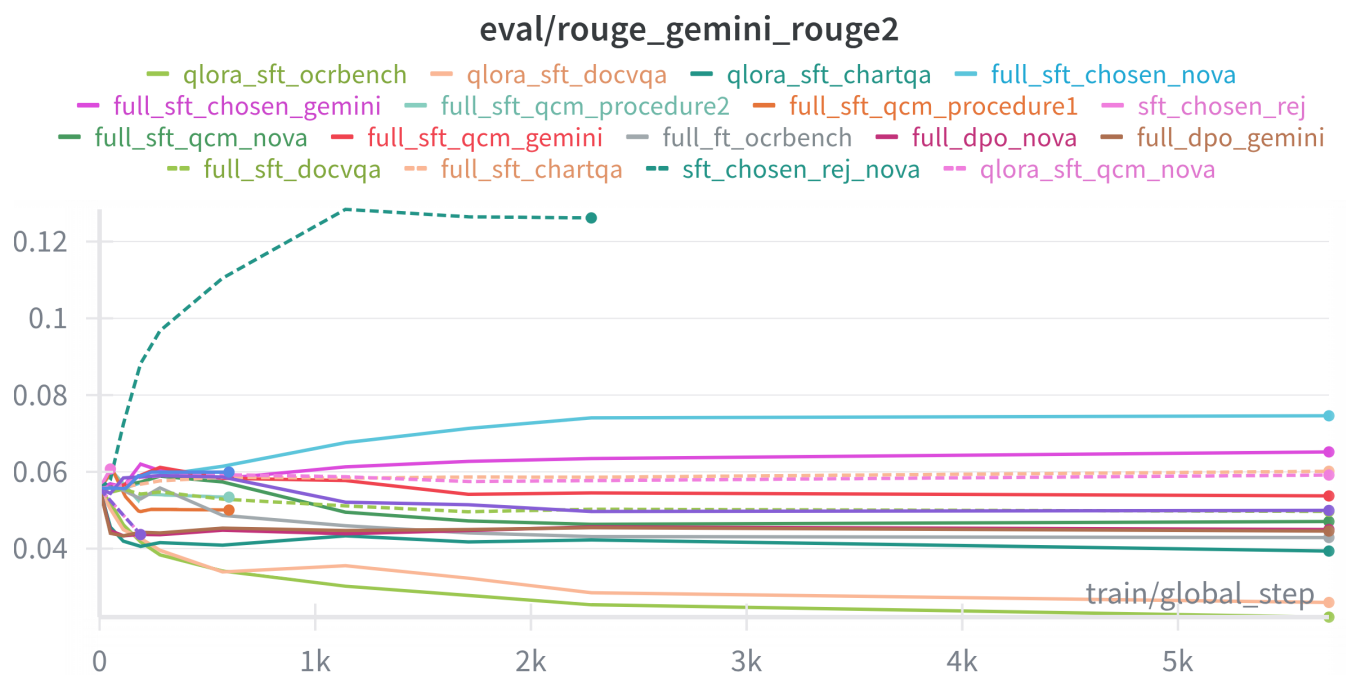


Figure 4.10: ROUGE-2 (Nova)

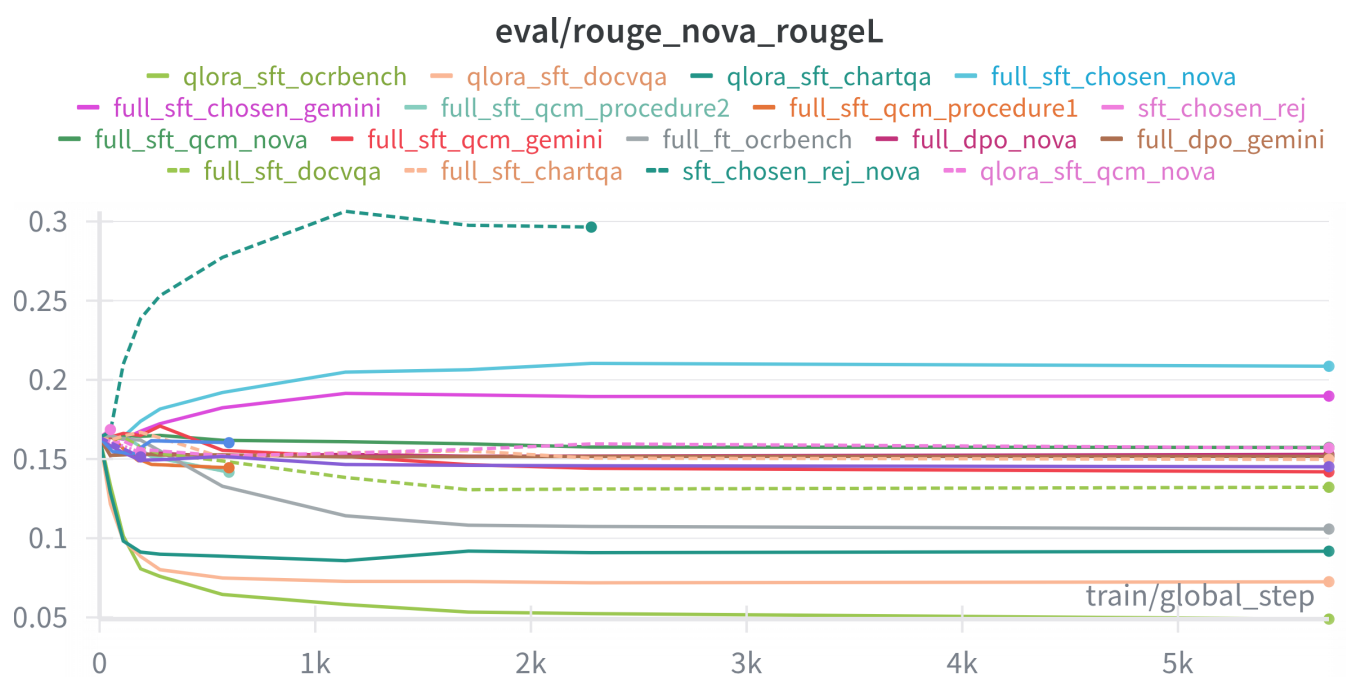


Figure 4.11: ROUGE-L (Nova)

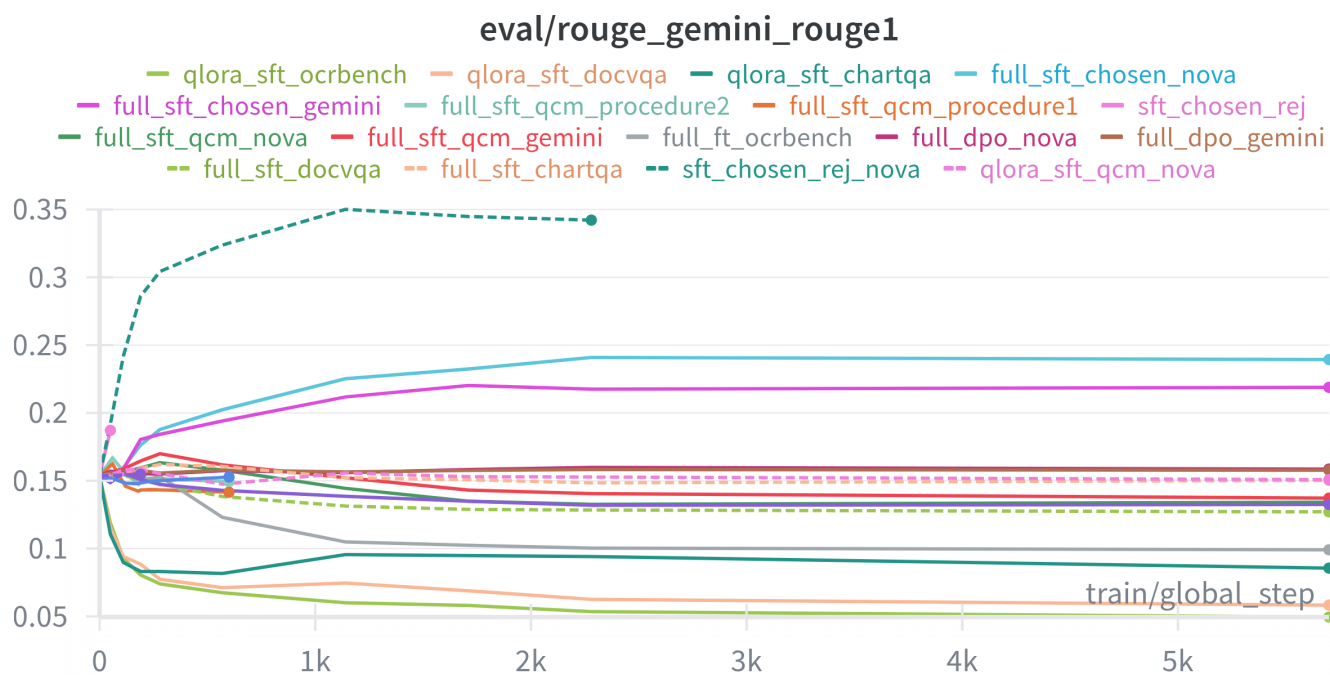


Figure 4.12: ROUGE-1 (Gemini)

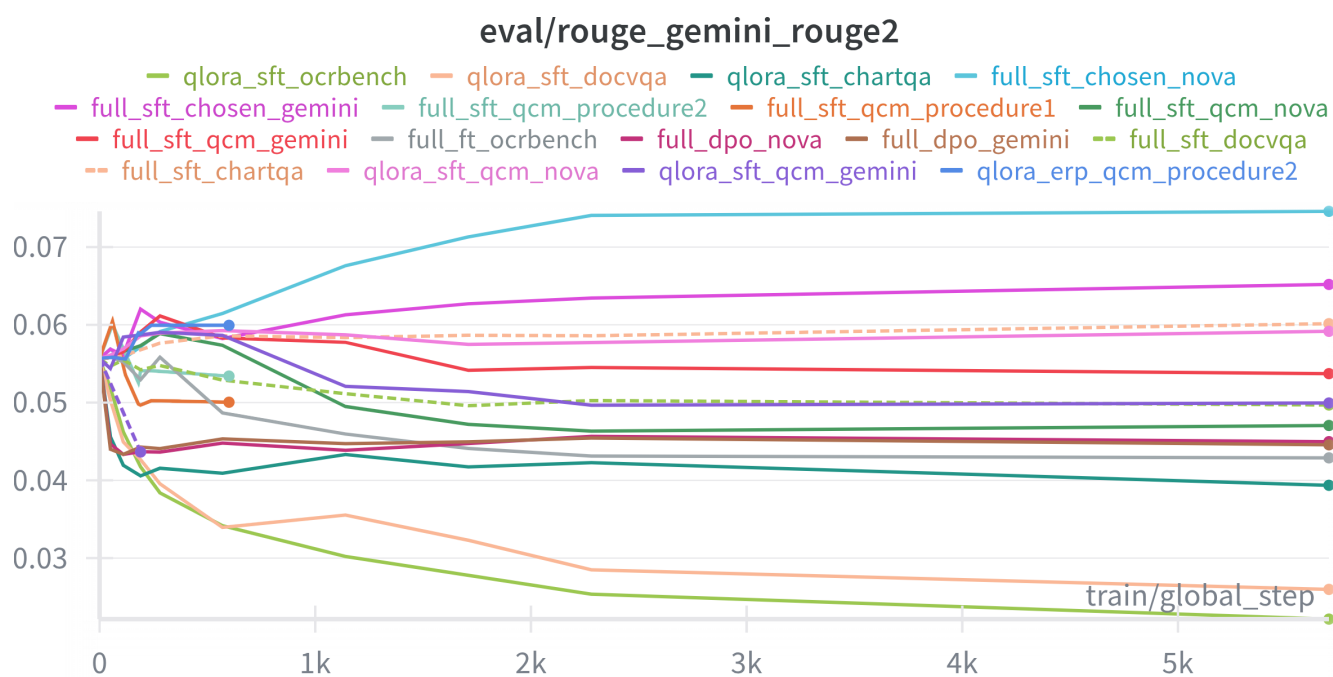


Figure 4.13: ROUGE-2 (Gemini)

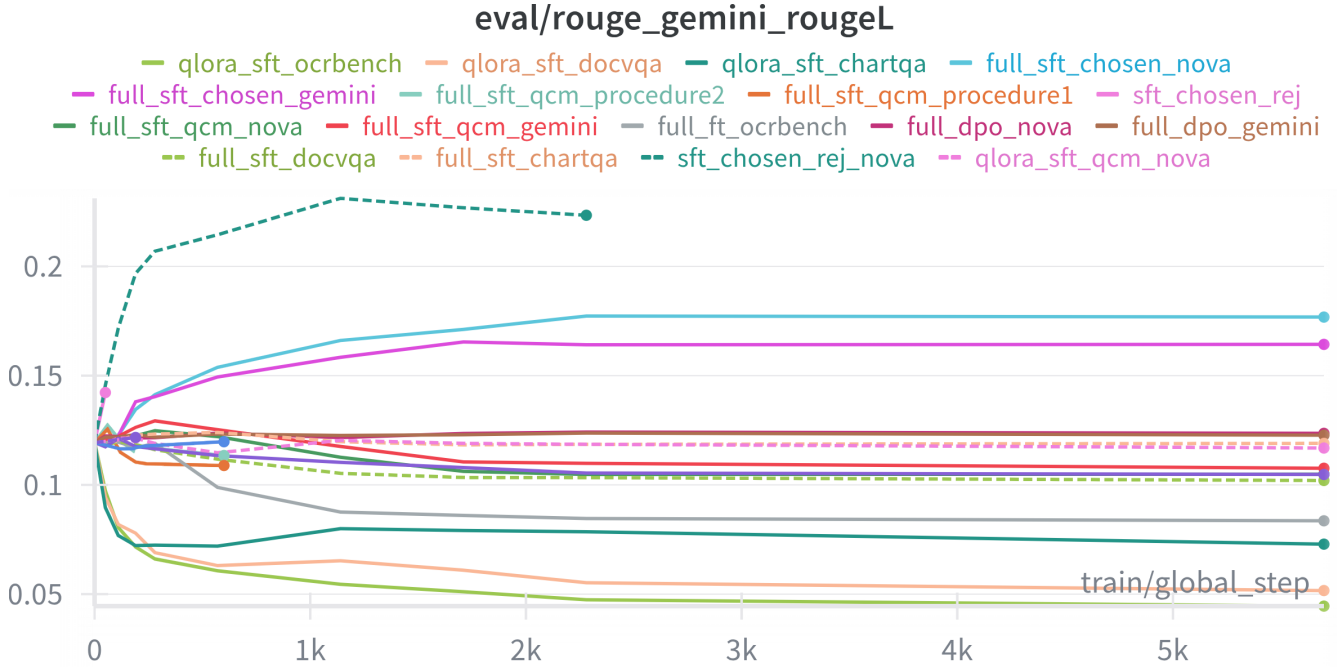


Figure 4.14: ROUGE-L (Gemini)

ROUGE Score Analysis. Figures 4.9–4.11 and Figures 4.12–4.14 present the ROUGE scores measuring response quality alignment with chosen responses on the Nova and Gemini evaluation datasets respectively. ROUGE-1 measures unigram overlap, ROUGE-2 measures bigram overlap, and ROUGE-L measures longest common subsequence, capturing different aspects of response similarity.

On the Nova evaluation dataset, the `full_sft_chosen_rej_nova` configuration achieves the highest scores, followed by `full_sft_chosen_nova` and `full_sft_chosen_gemini` at around 0.19. The `qlora_sft_chosen_nova` and `qlora_sft_chosen_gemini` runs were interrupted during training. The baseline starts around 0.19, and most configurations show degradation over training to 0.10–0.12, indicating that training on other objectives harms response generation quality aligned with Nova-style responses. MCQ-trained models tend to achieve higher scores than pure benchmark-trained ones, likely because they are trained not only to select the correct answer but also to explain why it is the correct answer. They also likely capture the style, syntax, and wording of the model from which they are distilled.

On the Gemini evaluation dataset, the same patterns can be observed, except that `full_sft_chosen_rej_nova` and `full_sft_chosen_nova` score higher than `full_sft_chosen_gemini`, which is surprising. Although models fine-tuned on Gemini data might be expected to perform best on Gemini evaluation, ROUGE scores favor surface-level lexical overlap and penalize stylistic diversity and verbosity. Gemini responses tend to be longer and more lexically varied, reducing n-gram overlap despite potentially higher semantic quality. As a result, models trained on Nova data, which produce more concise and templated responses, achieve higher ROUGE scores even on Gemini evaluations. This highlights a limitation of ROUGE for evaluating open-ended LLM responses. BERTScore, BLEURT, or LLM-based evaluation may provide more reliable measures of preference alignment and semantic correctness, as they better capture meaning and informational completeness than surface-level overlap metrics. However, the unexpectedly high scores could also result from poor model quality, as the model sometimes hallucinates and produces

repetitive answers.

Log-Probability Analysis

Log-probability metrics provide insight into how well models have learned to prefer chosen responses over rejected ones, which is critical for evaluating DPO-trained models and assessing alignment quality.

Log-Probability Metrics (Reminder)

Metric	Meaning
logprob_chosen_ppl	Perplexity of chosen responses. Lower = model is more confident about the good response. Calculated as $\exp(-\text{avg_logprob})$
logprob_rejected_ppl	Perplexity of rejected responses. Higher = model is less confident about the bad response (good sign)
logprob_acc	Preference accuracy = % of samples where $\text{logprob}(\text{chosen}) > \text{logprob}(\text{rejected})$. Higher = better alignment
logprob_margin	Average margin = $\text{avg_logprob}(\text{chosen}) - \text{avg_logprob}(\text{rejected})$. Positive = model prefers chosen. Larger positive = stronger preference

Table 4.11: Log-Probability Evaluation Metrics

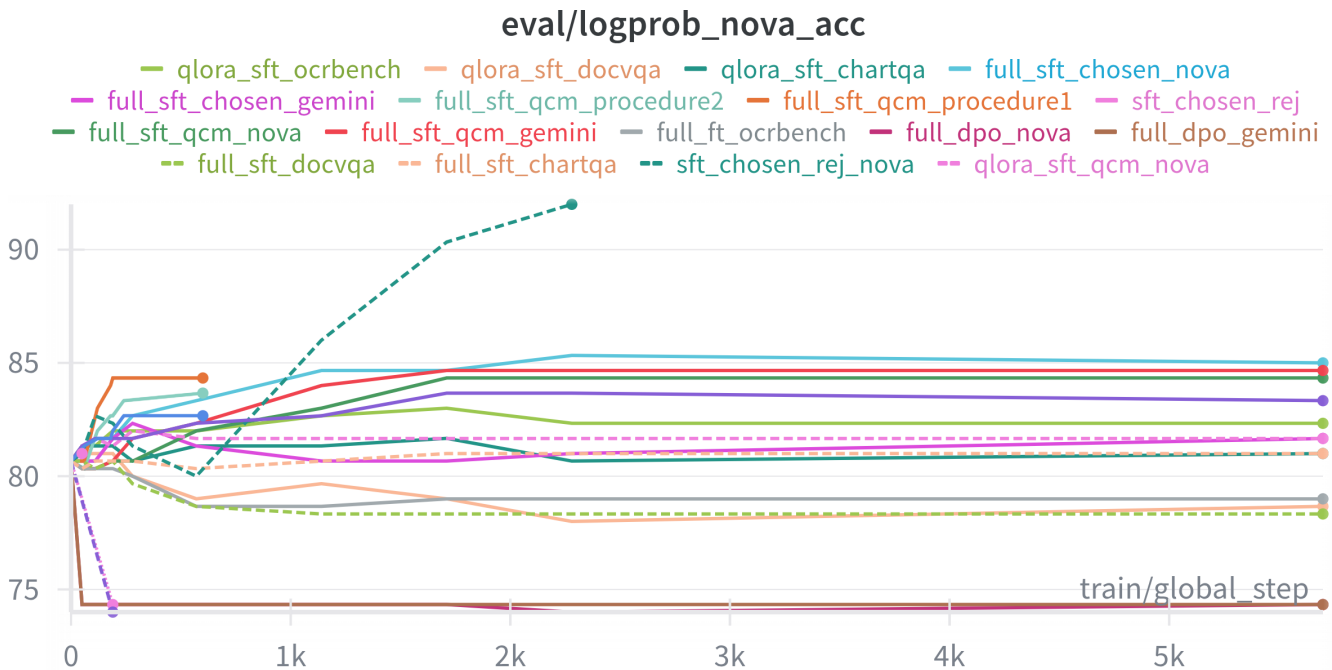


Figure 4.15: Log-prob preference accuracy (Nova)

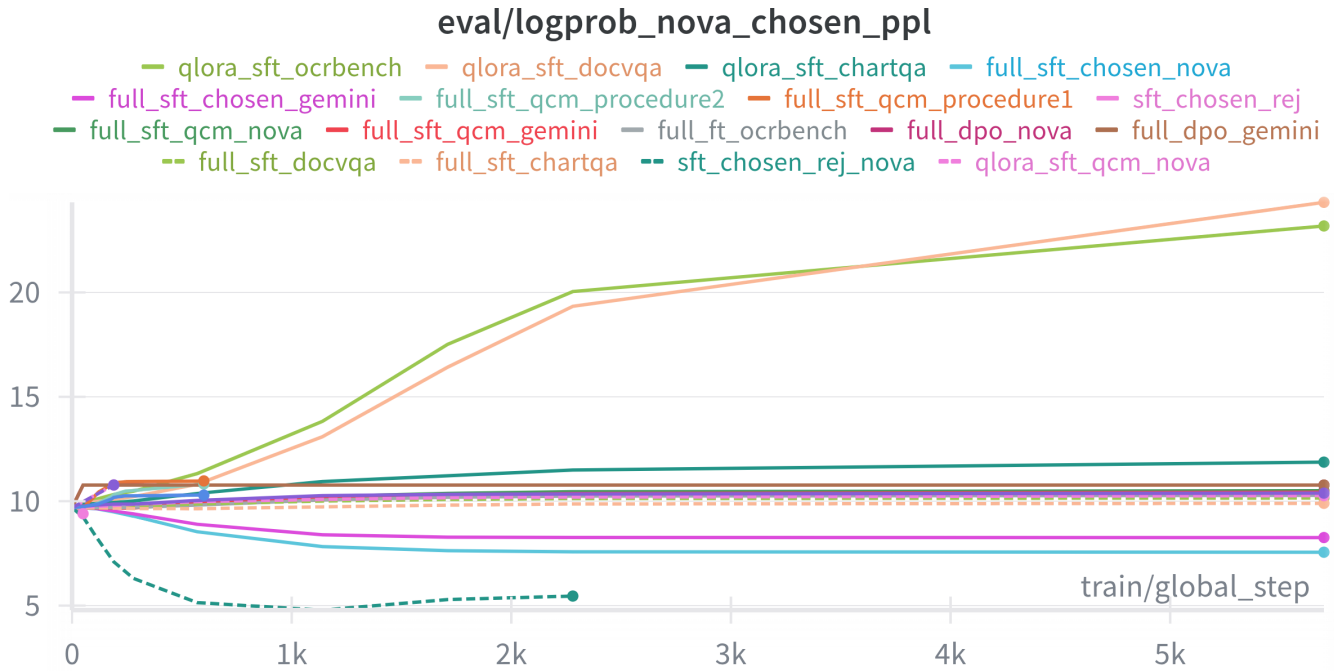


Figure 4.16: Perplexity of chosen responses (Nova)

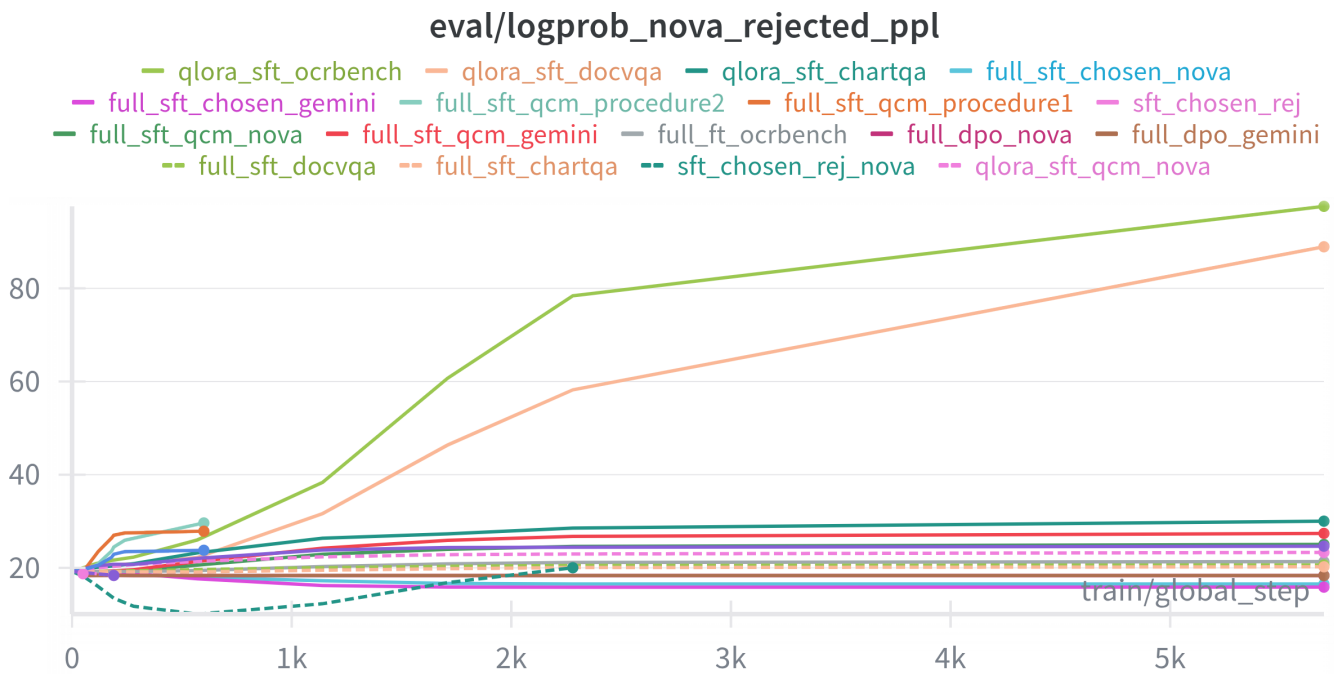


Figure 4.17: Perplexity of rejected responses (Nova)

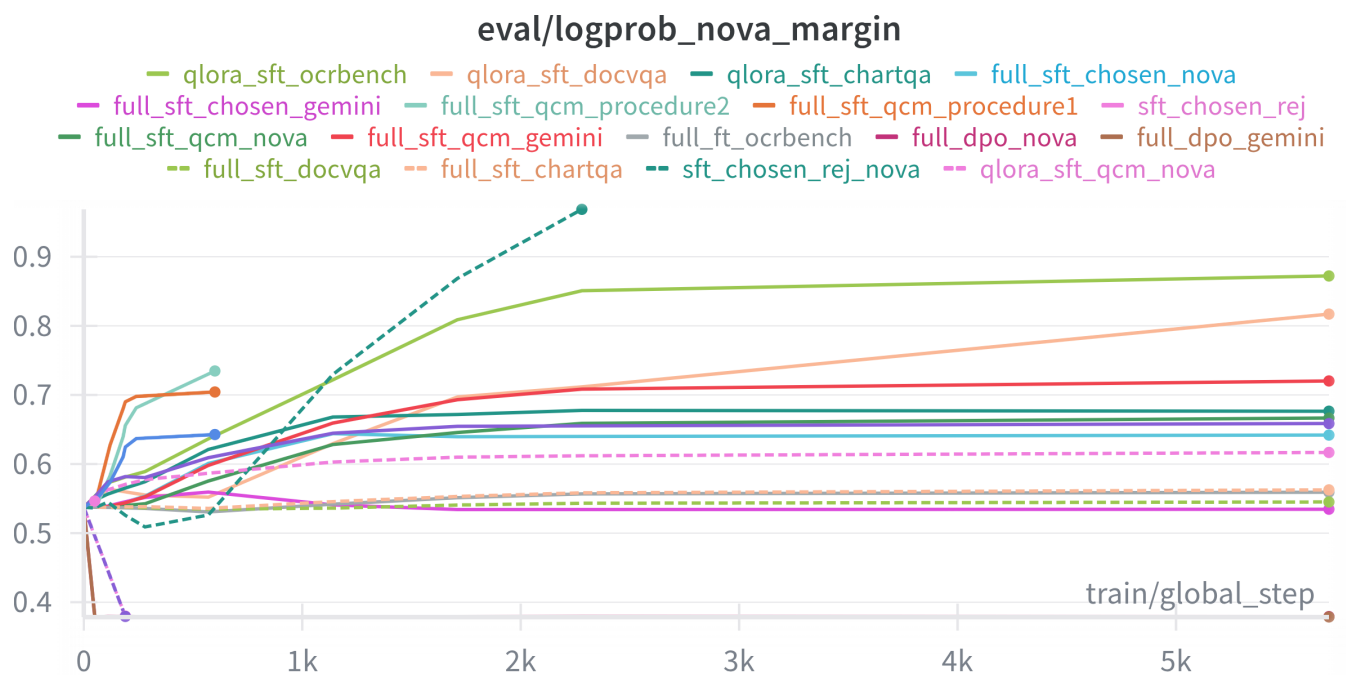


Figure 4.18: Log-prob margin chosen vs. rejected (Nova)

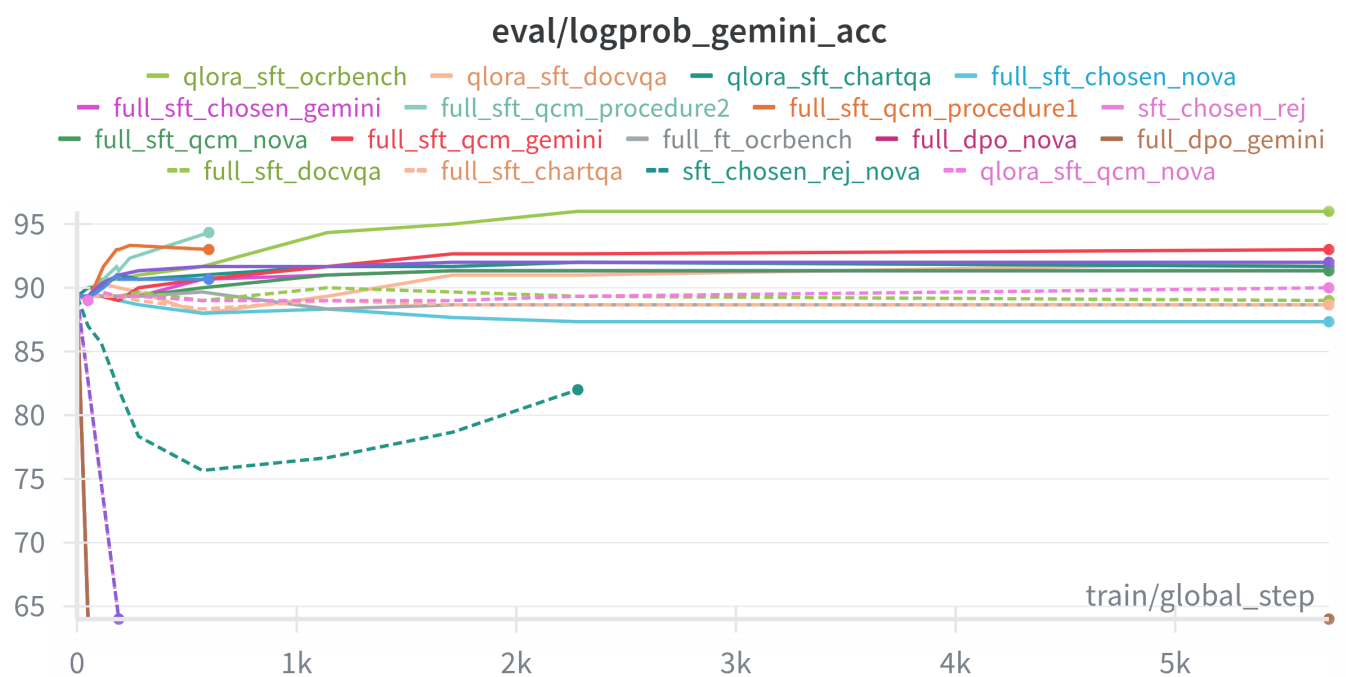


Figure 4.19: Log-prob preference accuracy (Gemini)

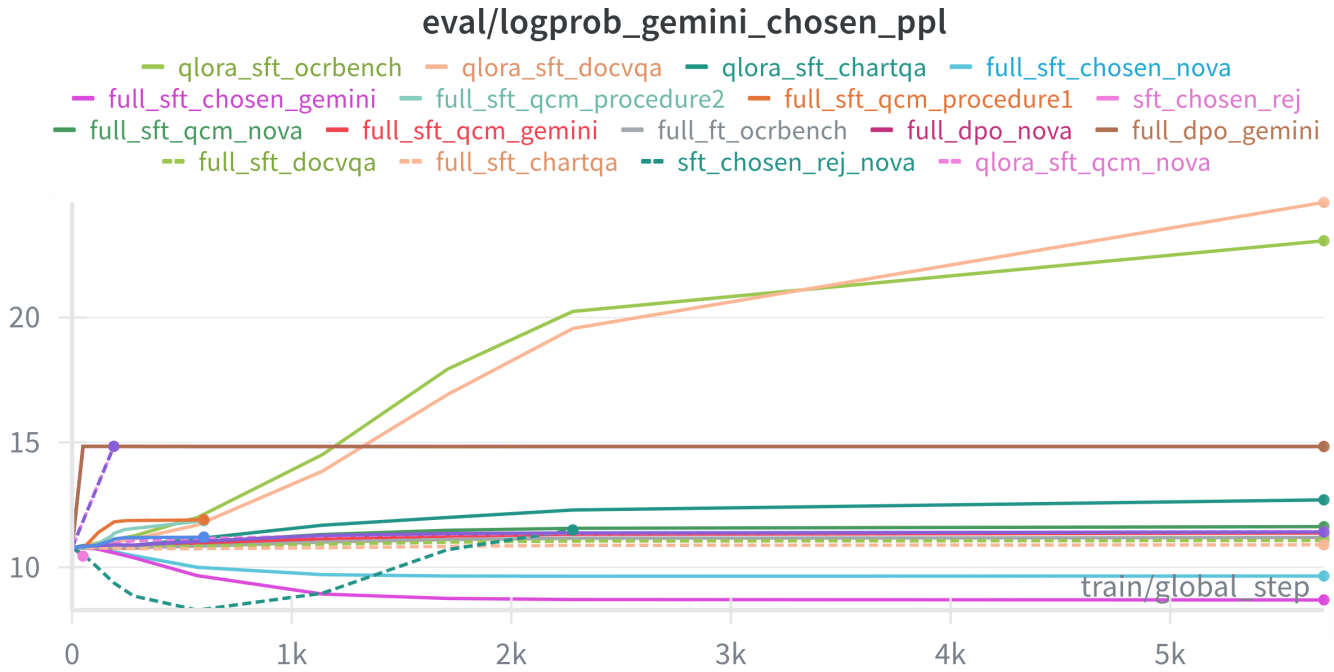


Figure 4.20: Perplexity of chosen responses (Gemini)

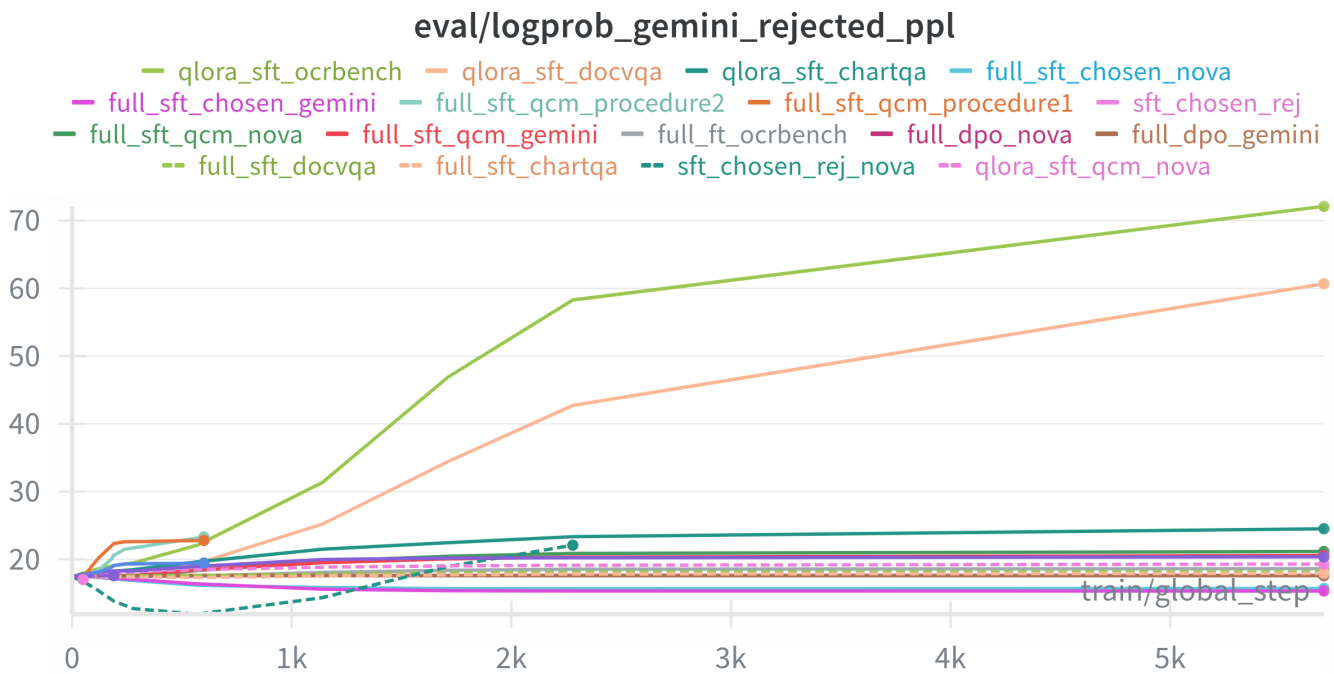


Figure 4.21: Perplexity of rejected responses (Gemini)

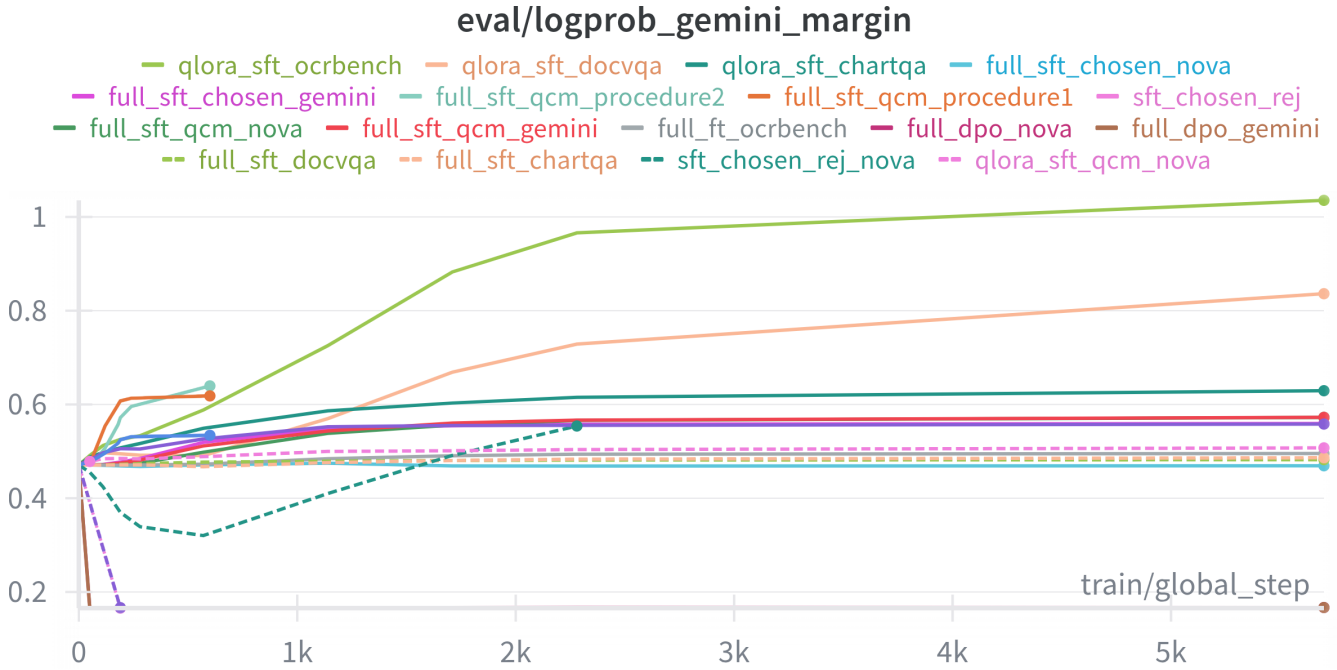


Figure 4.22: Log-prob margin chosen vs. rejected (Gemini)

Log-Probability Metrics Conclusion. Figures 4.16–4.17 and Figures 4.20–4.21 report perplexity-based metrics. The increase in `logprob_chosen_ppl` for models fine-tuned on OCRBench and DocVQA arises from a mismatch between training and evaluation objectives. OCRBench and DocVQA are task-oriented benchmarks focused on optical character recognition and visual question answering, rather than preference learning. Fine-tuning on these datasets induces extremely high confidence in predicting image-grounded tokens (e.g., visible text) while driving strong uncertainty for any alternative outputs.

This confidence imbalance substantially increases the log-probability margin between chosen and rejected responses on the preference benchmark, even though the models are neither trained on preference data nor on responses semantically close to the preferred answers. As a result, OCR-trained models can outperform models explicitly trained on the preference benchmark, not because they better capture preference structure, but because OCR-style supervision sharpens token-level confidence in a way that aligns favorably with the log-probability evaluation metric.

Figures 4.15–4.19 report log-probability accuracy, measuring how well models distinguish between chosen and rejected responses. As expected, the model trained on Nova assigns higher likelihood to Nova’s chosen responses than the other models, indicating that preference learning is functioning as intended.

However, Figure 4.19 shows that the model trained on OCRBench outperforms the model trained using Gemini preference data. This result is counterintuitive but consistent with the earlier analysis: OCR-based supervision amplifies token-level confidence differences that directly benefit log-probability-based metrics. Additional factors may contribute to this gap, including limitations of the CrewAI pipeline when using an older Gemini agent. In such cases, the agent often fails to fully understand the Mediasoft software, producing ill-formed or generic questions and defaulting to coarse image descriptions rather than task-specific queries, which weakens the quality of the generated preference data.

A similar pattern appears in the preference margin metrics shown in Figures 4.18 and 4.22. While Nova-trained models assign larger margins to chosen responses than to rejected ones, models trained on OCRBench and DocVQA exhibit even stronger preference margins on Gemini responses than the Gemini-preference-trained model itself. This further supports the conclusion that the observed gains stem from confidence amplification induced by OCR-style training rather than improved preference modeling. Notably, SFT models appear more stable under these metrics, suggesting greater robustness to such confidence imbalances.

Results Conclusion

In the experiments, QLoRA-based SFT training consistently outperformed full supervised fine-tuning. A likely explanation is that QLoRA updates only a small, low-rank subset of parameters, which helps preserve the model's pretrained knowledge and previously acquired skills. In contrast, full SFT could appear to overwrite these representations more aggressively, leading to faster overfitting and reduced generalization. Under limited data and model capacity, parameter-efficient fine-tuning therefore acts as an implicit regularizer, resulting in better robustness and transfer performance.

The model exhibits clear overfitting and a lack of transferability between the training and testing sets, as well as across different datasets. The minimal transfer observed initially is likely due to the instruct model shifting from an instructor-style output to a multiple-choice format. This issue stems from the insufficient size of the training dataset, which contains only 13,000 questions. To be effective, the model likely requires a dataset of 100,000 or even one million pairs. Furthermore, with only 256 million parameters, the model architecture is likely too small to acquire robust generalization capabilities. Additionally, there appear to be issues with the DPO training setup. DPO was more difficult to integrate with the SmolVLM2 libraries, as images must be preprocessed to ensure padding to a multiple of 16 for compatibility with the current implementation. This additional preprocessing may slow down the training pipeline and lead to timeouts, and it is possible that some of the observed behavior stems from implementation errors.

It is also important to note that DPO was not originally designed to improve performance on multiple-choice question (MCQ) tasks. In the setup used here, the rejected response is selected at random. This choice may inadvertently degrade the model's capabilities: by repeatedly labeling a specific option (e.g., "B") as incorrect, the model may learn to systematically avoid that option, even in cases where it is the correct answer.

A similar effect may be occurring in the benchmark-trained models. Using a randomly selected answer from a different question as the rejected response may implicitly teach the model that certain valid answers are undesirable, leading it to avoid them even when they are correct.

4.8.4 Discussion: Dataset Requirements

The primary limitation of the current approach lies in the insufficient size of the training dataset. Developing a useful local large language model that can operate efficiently on limited GPU resources requires substantially more training data to achieve robust generalization and stable performance. Several strategies may be considered to address this constraint.

One approach consists of constructing high-quality datasets generated by domain experts. While this method typically yields the most reliable and accurate supervision, it is often financially infeasible for small and medium-sized ERP companies due to the high cost of expert labor.

An alternative involves combining retrieval-augmented generation with keyword-based search and distillation from commercial models. It remains computationally expensive and may raise legal or contractual concerns related to the terms of services.

A notable aspect of this approach is that the ERP agent can be fine-tuned without traditional datasets by directly connecting a Teacher/Evaluator Agent (TUA). The TUA couples with the ERP agent during training by generating prompts, retrieving screenshots, and feeding them as inputs to the model. It simultaneously reviews ERPNext documentation and open-source code to evaluate both the agent's responses and real task outcomes within the software. Where responses fall short, the TUA corrects or improves them and uses these refined outputs to directly fine-tune the model, enabling iterative performance gains over time.

While this distillation-based approach is effective, it may conflict with the terms of service (ToS) of commercial model providers and should not be applied without authorization.

A lower-cost solution relies on retrieval-augmented generation combined with open-source vision-language or language models, where multiple generated responses are compared and the most consistent or accurate outputs are selected. This approach primarily incurs hardware and energy costs; however, freely available models often exhibit instability when operating outside their training distribution. In such situations, they may produce repetitive text, incoherent reasoning, or degenerate outputs, which can reduce dataset quality.

In addition, systematically storing user interactions, including software logs, email exchanges, chat messages, and recorded audio conversations, could provide a valuable source of real-world training data. These interaction traces could be leveraged by intelligent agents to generate structured supervision signals. Nevertheless, the current scale of available data remains far from sufficient. For example, even a corpus of approximately 13,000 interaction samples is modest by modern machine learning standards, particularly when a significant fraction corresponds to bug reports or low-information tickets that contribute limited ERP understanding. Much more traffic would be needed in order to finetune a model so this is not an options for small ERPs.

Finally, it is important to note that many ERP workflows are inherently deterministic and governed by predefined rules, particularly in form-based data entry and transactional processes. As a result, a substantial portion of training data could maybe be automatically generated through programmatic simulation or procedural generation rather than relying exclusively on human annotation or generative transformers. This synthetic data generation paradigm represents a promising avenue for scaling training datasets efficiently while maintaining structural correctness and domain consistency.

A longer-term research direction is the development of autonomous systems capable of interacting with ERP software environments through reinforcement learning or related self-improving methods. Such models could progressively acquire operational knowledge and continuously fine-tune themselves through experience, thereby reducing reliance on manually curated datasets.

In the next chapter, we will evaluate ways to improve models without using proprietary APIs as it could violate ToS and with minimal human interactions as it might be too expensive for a small ERP.

Chapter 5

Autonomous Learning

This chapter investigates approaches to training a model that learns to use ERPNext with minimal human involvement. The motivation is straightforward: small ERP companies cannot afford proprietary APIs or large-scale annotation campaigns. By developing methods that allow a model to teach itself through environment interaction, the goal is to make it feasible for small organizations to train and maintain a private, domain-specific assistant at low cost.

Distillation-based methods raise ToS concerns, and manual annotation is prohibitively expensive at this scale. The approaches explored here therefore minimize human cost and avoid dependence on commercial supervision. In this context, “human” does not refer to a physical annotator but to automated agents such as Claude Code, Gemini CLI, and Codex, which simulate task completion by working through a predefined JSON task list, marking each item complete before advancing to the next.

The pipeline uses exclusively free, open-source models, relying on self-generated data, environment interaction, and execution-based filtering to drive improvement. Experiments use a 2B-parameter model to validate feasibility under resource constraints; in practice, the same pipeline can be applied to larger open-source models for stronger results.

5.0.1 Design Philosophy

All approaches presented in this thesis interact with ERPNext through the Model Context Protocol (MCP) rather than through pixel-based graphical interfaces, where the model would need to interpret rendered screenshots, identify button locations in pixel coordinates, and simulate mouse clicks and keyboard input as a human user would. MCP exposes ERP operations as structured JSON tool calls, where each action is represented as a tool name paired with typed parameters, for example `{"tool": "create_document", "parameters": {"doctype": "Customer", "name": "Acme"}}`. This symbolic representation is substantially easier for small language models to learn than raw pixel coordinates or direct UI manipulation, because the model only needs to produce valid JSON conforming to a fixed schema rather than locating screen elements and generating precise click coordinates. By reducing the problem from visual perception and spatial reasoning to structured text generation, MCP lowers both the perceptual and control burden on the model, making it feasible to achieve useful ERP assistance even with models as small as 256M parameters.

5.0.2 Failed SmolVLM2-256M Experiment

Early experiments applying GRPO [7] to SmolVLM2-256M failed to produce meaningful improvement. The model lacked the representational capacity to generate diverse candidate outputs, instead producing near-identical responses across rollouts. Without meaningful variation between candidates, the relative preference signal that GRPO depends on disappeared entirely, and training either stalled or collapsed to degenerate outputs. This limitation would probably extend to DAPO [38] and VAPO [39], which rely on similar group-based preference mechanisms.

This failure is consistent with the high token-level perplexity observed in small models fine-tuned on OCR-style data (see Table 4.11, Figures 4.16–4.17). Such training induces extreme confidence on image-grounded tokens while driving high uncertainty on all other outputs, confining sampling to a narrow region of output space and making reward-bearing trajectory discovery really hard, even with high temperatures.

A separate voting experiment with the multi-agent system (Section 5.5) provided a complementary insight: the text-only LLM agents in the ensemble consistently outperformed the VLM agents on structured tool-call generation, likely because LLMs receive substantially more code, JSON, and tool-use data during pretraining. This suggests that distilling tool-use knowledge from LLMs into VLMs, for instance through voting consensus or direct knowledge distillation with OCR-converted screenshots as input, is a more promising path than training VLMs on RL objectives they lack the capacity to explore.

SFT avoids the RL failure modes by directly aligning model outputs with correct structured actions, providing stable convergence without requiring the model to discover good trajectories on its own. Reinforcement learning may become beneficial at sufficient model scale, but applying it to a 256M-parameter VLM may be ineffective in practice.

5.0.3 Platform and Model Changes

The experiments in this chapter are conducted on ERPNext rather than Mediasoft MS.ERP. This decision was driven by a practical constraint: no MCP-compatible interface was available for the Mediasoft MS.ERP system, whereas ERPNext exposes a fully documented REST API that can be wrapped as an MCP server, enabling programmatic environment interaction without manual intervention.

In addition, the model architecture was changed from SmolVLM2 to the Qwen2-VL family (Alibaba). Early experiments with SmolVLM2 revealed limited capacity for structured output generation and tool-call formatting, yielding insufficiently reliable results for the self-improving training pipelines explored here. The Qwen2-VL models demonstrated stronger baseline performance on both tool selection and parameter filling, making them a more suitable foundation for the autonomous learning methods that follow.

5.1 Base Model (Zero-Shot)

The base model uses Qwen2-VL-2B-Instruct with no fine-tuning. The full 27-tool MCP schema is injected into the system prompt, providing tool definitions, parameter descriptions, and examples. Greedy decoding (temperature=0) produces JSON tool calls.

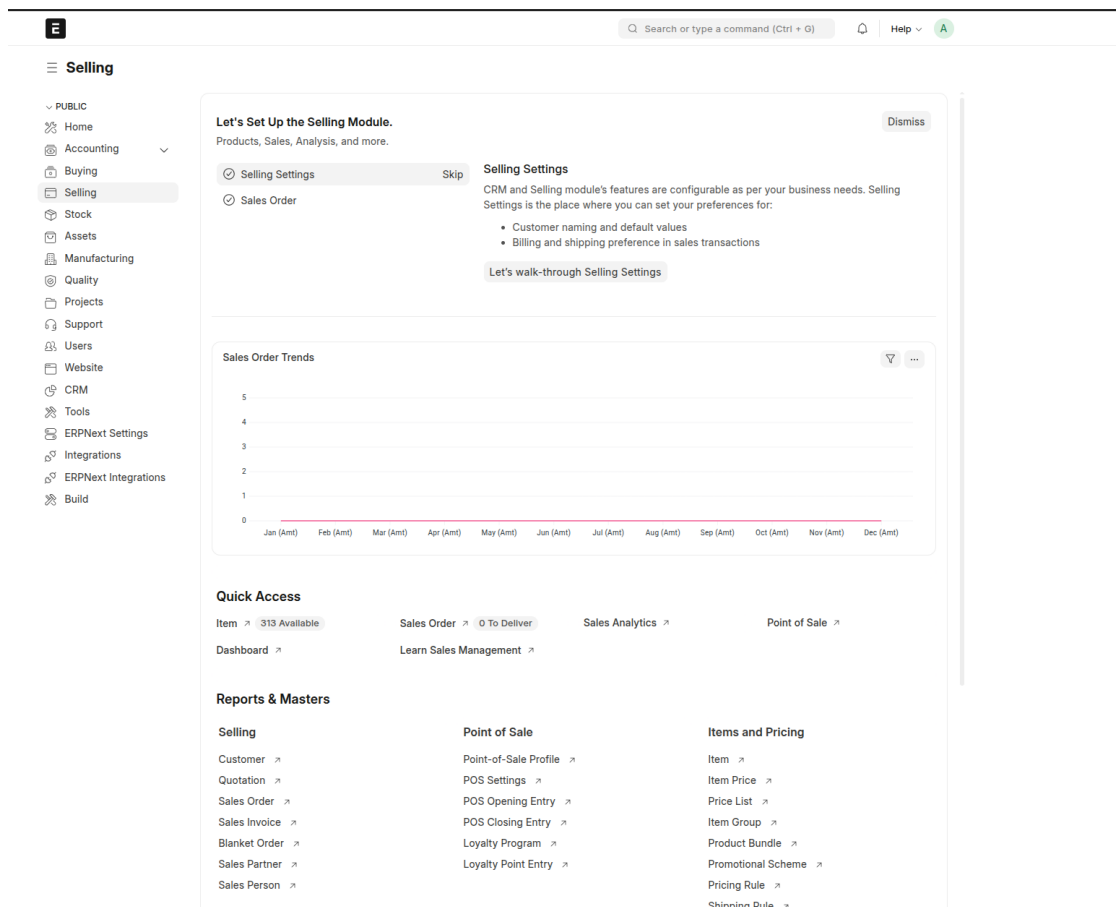


Figure 5.1: ERPNext Selling module interface showing the main dashboard.

Purpose. Establish a lower bound. The base model achieves 74.9% action accuracy (correct tool selection) but only 28.4% parameter accuracy on in-distribution data, demonstrating that pre-trained VLMs have inherent tool-use ability from web-scale training, but accurate parameter filling requires fine-tuning.

5.2 SFT (Warm up Supervised Fine-Tuning)

Idea. Warm up the models by fine-tuning on a curated dataset of (prompt, expected_tool_call) pairs using standard next-token prediction loss. The models could also have undergone a domain-adaptive pretraining step to further improve the pipeline, for example by scraping pages from frappe.io, splitting the text at a random point (30–90%), applying transforms, and training the model to predict the continuation.

5.2.1 Dataset Design

The dataset contains 3,480 balanced samples:

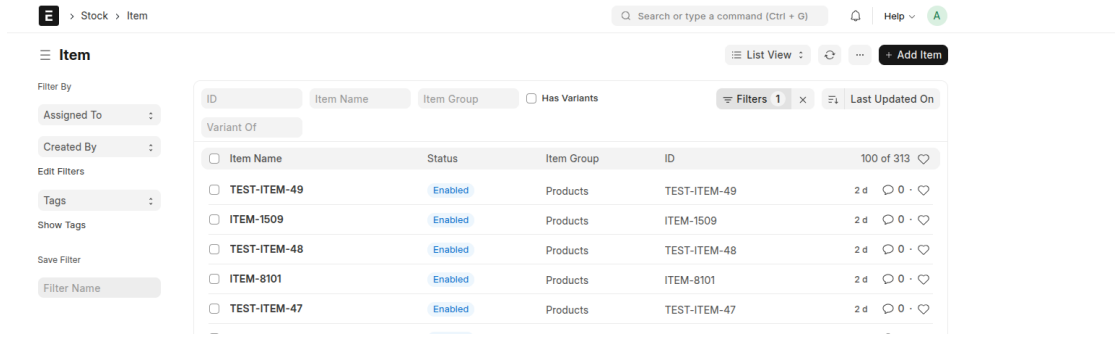


Figure 5.2: ERPNext Item List View displaying inventory items.

- 1/3 **action tasks**: Tool call generation from natural-language requests.
- 1/3 **screenshot descriptions**: Describe what is shown in an ERPNext screenshot.
- 1/3 **knowledge QA**: Multiple-choice questions about ERPNext concepts.

5.2.2 Training Procedure

Table 5.1: SFT hyperparameters.

Parameter	Value
Adapter	QLoRA ($r=16$, $\alpha=32$, dropout=0.05)
Target modules	q_proj, k_proj, v_proj, o_proj
Learning rate	2×10^{-4} (VLM), 1×10^{-4} (LLM)
Epochs	3
Effective batch	8 (batch=2, grad_accum=4)
Warmup steps	50
Training time	~60 min per model

Result. 100% action accuracy, 82.7% parameter accuracy on ID data. The primary limitation is the OOD generalization gap: −10.4pp action, −26.1pp parameter accuracy.

5.3 E-GRFT (Environment-Grounded Rejection Fine-Tuning)

Idea. E-GRFT draws on five established lines of work. First, the four-component reward function is directly inspired by **ToolRL** [40], which demonstrated that fine-grained reward decomposition for tool selection and application leads to more stable and effective learning than coarse-grained binary signals. Second, the rejection sampling strategy, generating many rollouts and training only on those exceeding a quality threshold, follows the rejection fine-tuning (RFT) paradigm introduced in **STaR** [17] and popularized at scale in **LLaMA 2** [41], as well as recent extensions to agent tuning [42]. Third, the environment-grounded pillar, where the agent learns from real state changes in a running system rather than static datasets, is rooted in **GLAM** [43], which studied grounding LLMs in interactive environments through online reinforcement learning. Fourth, the use of live execution results as reward signal mirrors **RLEF** [44], which grounds code LLMs in execution feedback by evaluating each attempt against test

cases and feeding results back into the training loop. Fifth, E-GRFT builds upon foundational SFT-based tool-use works such as **Toolformer** [18] and **API-Bank** [45], replacing their static datasets with live environment interaction. Concretely, the model interacts with a live ERPNext instance, collects execution feedback, filters high-quality experiences via rejection sampling, and retrains.

5.3.1 Algorithm

The system has 6 components operating in a loop:

1. **ERPNextExplorer**: Discovers real data: scans 26 key doctypes, retrieves existing customers, items, suppliers, warehouses, and field schemas.
2. **TaskGenerator**: Creates tasks grounded in discovered entities. Example: if customer “Acme Corp” exists, generates “*Get the details of customer Acme Corp*”. Difficulty progression:
 - Early cycles: 80% easy, 20% medium.
 - Middle cycles: 50% easy, 40% medium, 10% hard.
 - Late cycles: 30% easy, 40% medium, 30% hard.
3. **Agent prediction**: Current model generates a tool call.
4. **Executor**: Executes against live ERPNext with safety tiers:
 - **SAFE** (17 tools): all read-only operations.
 - **GUARDED** (2 tools): `create_document`, `update_document` with SELFLEARN- prefix.
 - **BLOCKED** (8 tools): `delete`, `submit`, `cancel`, etc.
5. **RewardScorer**: 4-component reward (inspired by ToolRL):

$$R = \underbrace{R_{\text{format}}}_{0.25} + \underbrace{R_{\text{tool}}}_{0.25} + \underbrace{R_{\text{params}}}_{0.25} + \underbrace{R_{\text{exec}}}_{0.25} \in [0, 1] \quad (5.1)$$

- R_{format} : Valid JSON with `tool` and `parameters` keys.
- R_{tool} : Correct tool name (partial credit 0.10 if same category).
- R_{params} : 50% Jaccard similarity of keys + 50% value match.
- R_{exec} : API call succeeded.

Grades: **gold** (≥ 0.75), **silver** (≥ 0.50), **bronze** (≥ 0.25), **failed** (< 0.25).

6. **Retrain**: Mix 70% original SFT data + 30% gold/silver experiences. Train 2 epochs at $\text{lr} = 5 \times 10^{-5}$. Validate on held-out test set; promote if improved, rollback if degraded.

Key insight. The reward decomposition provides a fine-grained learning signal: even when execution fails, the model earns partial credit for correct format, tool, or parameters.

5.3.2 Detailed Description

Environment-Grounded Rejection Fine-Tuning, or E-GRFT, is a method for improving a trained model after its initial supervised fine-tuning phase by letting it practice against a live system and learning only from its best attempts. It draws on three established ideas. The first is rejection fine-tuning, which means that instead of keeping every output the model produces during practice, only those meeting a quality threshold are retained. The second is environment-grounded training, where the model learns from real state changes in an actual running system rather than from a static, pre-built dataset. The third is grounded reinforcement learning, where feedback from the environment itself serves as the reward signal that indicates what went well and what did not.

In practice, the system contains six components that operate together in a loop. The first component is the **ERPNextExplorer**, which connects to a live ERPNext instance and performs a discovery pass. It scans 26 key doctypes such as Customer, Supplier, Item, Sales Order, Warehouse, and others, retrieves the names of existing documents within each doctype, and pulls field schema definitions. This discovery step ensures that every task the system generates references real entities that actually exist in the database, rather than invented placeholders that would fail on execution.

The second component is the **TaskGenerator**, which uses the discovered environment state to create practice tasks that are grounded in reality. For instance, if the explorer found that a customer called “Acme Corp” exists, the generator might produce the natural language prompt “Get the details of customer Acme Corp” with the expectation that the model should respond with a `get_document` tool call targeting that specific customer. The generator also controls difficulty progression across cycles. In the early cycles, 80% of tasks are easy read operations like fetching a document or listing records, and the remaining 20% are medium-difficulty tasks like running reports or checking stock balances. As training progresses into the middle cycles, the mix shifts to 50% easy, 40% medium, and 10% hard. In the late cycles, the distribution reaches 30% easy, 40% medium, and 30% hard, where hard tasks include guarded write operations like creating or updating documents. The generator also includes an exploration bonus that favors under-explored tools, weighting selection inversely to how many times each tool has already been practiced.

The third step is **agent prediction**, where the current model receives each task prompt and generates a tool call response. The fourth component is the **Executor**, which takes that predicted tool call and runs it against the live ERPNext instance. To prevent the model from causing damage during practice, the executor enforces a three-tier safety system. Seventeen read-only tools such as `get_document`, `list_documents`, `get_count`, `search_link`, and various reporting and inventory queries are classified as **safe** and can be executed freely. Two tools, `create_document` and `update_document`, are classified as **guarded**, meaning they are allowed to execute but only after the system enforces a `SELFLEARN-` prefix on all name fields, so that any documents created during practice are clearly identifiable and can be cleaned up afterward. Eight tools including `delete_document`, `submit_document`, `cancel_document`, and `amend_document` are **blocked** entirely and will never be executed regardless of what the model predicts. In a longer production deployment, these safety restrictions would need to be relaxed: the name prefixes prevent the model from learning to create real documents, and the blocked tools prevent it from practicing the full workflow lifecycle. Once the model has demonstrated sufficient reliability, the guarded prefixes and tool blocking should be progressively removed so that the training signal reflects actual production usage.

The fifth component is the **RewardScorer**, which assigns a fine-grained score to each prediction using a four-component reward function inspired by ToolRL [40]. The total reward ranges from zero to one and is the sum of four equally weighted terms, each worth 0.25. The format reward checks whether the model produced valid JSON containing the required `tool` and `parameters` keys. The tool reward checks whether the predicted tool name matches the expected one, with partial credit of 0.10 awarded if the predicted tool is wrong but belongs to the same functional category. The parameter reward is computed as a blend of two sub-scores: 50% comes from the Jaccard similarity between the predicted parameter key names and the expected ones, and 50% comes from the proportion of overlapping keys whose values match exactly. The execution reward is simply whether the API call succeeded when run against the live system. Based on the total score, each experience is graded as gold if it scores ≥ 0.75 , silver if between 0.50 and 0.75, bronze if between 0.25 and 0.50, or failed if below 0.25. The key insight of this decomposition is that the model can earn meaningful partial credit even when the overall execution fails. A prediction that produces well-formed JSON with the correct tool name but slightly wrong parameters still receives credit for format and tool accuracy, giving the training process a more informative signal than a binary success-or-failure label would.

The sixth component handles **retraining**. Every few cycles, the system collects all gold and silver experiences and mixes them with the original supervised fine-tuning dataset in a 30-to-70 ratio, meaning 70% of the mixed training set comes from the original SFT data and 30% from the self-generated high-quality experiences. The model is then retrained for 2 epochs at a reduced learning rate of 5×10^{-5} , which is lower than the initial SFT learning rate to avoid catastrophic forgetting. After retraining, the new model is validated against a held-out test set. If action accuracy improved or held steady without any key metric dropping by more than 5 percentage points, the new model is promoted and hot-swapped into the loop for subsequent cycles. If performance degraded, the system rolls back and continues with the previous model. A circuit breaker also monitors the execution success rate and halts the entire loop if it drops below 10%, preventing model collapse. In a production setting, the system should maintain its own growing dataset rather than relying on a fixed SFT base, so that the mix evolves as the model improves. Every dataset generated at each cycle should also be persisted separately, enabling retrospective statistical analysis of how data quality, reward distributions, and task difficulty change over time.

Result. The end result of this approach was the best out-of-distribution performance observed across all training methods: 93.1% action accuracy and 63.6% parameter accuracy, achieved by adding only 242 self-generated samples on top of the base supervised fine-tuning dataset. Execution success improved from 0% at the start of the loop to 73.5% across 3 improvement cycles, demonstrating that even a small number of high-quality, environment-grounded experiences can meaningfully improve a model beyond what static dataset training alone achieves.

5.4 SSMCQ (Self-Supervised MCP Querying)

Idea. A variant of E-GRFT combining three established principles: execution-based filtering from Toolformer [18], iterative self-play bootstrapping from TALM [19], and verified-correctness filtering from STaR [17]. The model generates tool calls and knowledge questions grounded in live ERP data, executes or verifies each, and keeps only confirmed successes, requiring no human labels.

5.4.1 Algorithm

Two data collection phases:

1. **ALTU (Auto-Label Tool Use):** Generate predictions for MCP tools (`get_document`, `list_documents`, `get_count`, `search_link`, etc.). Validate against the MCP schema, execute against live ERPNext, and keep only predictions that produce valid JSON *and* succeed at execution.
2. **SPIE (Self-Play Instruction Expansion):** Query ERPNext for existing entities (customers, items, suppliers), generate MCQ questions with one correct and three wrong answers, and keep only correctly answered samples.

Retrain: continue from SFT adapter with 231 self-collected samples (2 epochs, $lr = 5 \times 10^{-5}$).

5.4.2 Detailed Description

SSMCQ is a variant of E-GRFT that replaces the graded reward function with a binary accept/reject criterion: predictions either pass the full verification pipeline or are discarded entirely. It draws on

three established principles. From Toolformer [18], it takes execution-based filtering, using the environment itself as the quality oracle. From TALM [19], it borrows self-play bootstrapping, where the model generates both questions and answers. From STaR [17], it adopts verified-correctness filtering, where only confirmed-correct predictions enter the training set. A complementary perspective comes from Exploratory Iteration [46], which grows a task space by selectively sampling informative intermediate states, a principle echoed by SSMCQ’s SPIE phase, which expands its own task space by generating new questions from discovered environment entities.

The first phase, **ALTU** (Auto-Label Tool Use), creates prompts targeting MCP tools, grounded in real data discovered from the live ERPNext instance by scanning available doctypes and listing existing documents. The model generates a tool call prediction for each prompt. Each prediction is validated against the MCP tool schema (recognized tool name, required parameters present), then executed against live ERPNext. The acceptance criterion is strict: a prediction is accepted only if it produces valid JSON, passes schema validation, and the execution succeeds. There is no partial credit or fuzzy matching. Across 3 cycles with roughly 80 ALTU tasks each, the model achieved approximately 90% live execution success, yielding around 213 accepted action samples from 237 attempts.

The second phase, **SPIE** (Self-Play Instruction Expansion), creates multiple-choice knowledge questions from discovered environment data. The system picks a real entity (e.g., a customer name), generates three wrong alternatives by sampling other entities or fabricating names with a **FAKE-** prefix, shuffles the four options, and presents the model with the question. Only correctly answered MCQs are accepted. This phase was harder for the model (20–28% accuracy across cycles), which is close to the 25% random baseline for 4-choice MCQs, suggesting the model has near-chance knowledge of specific ERP entities. Nevertheless, correctly answered samples still provided valuable knowledge-grounding signal.

For retraining, the approach continues from the existing SFT adapter. The retrain script loads Qwen2-VL-2B in 4-bit quantization, applies the SFT QLoRA adapter as a starting point, and fine-tunes on the 231 accepted samples for 2 epochs at $\text{lr} = 5 \times 10^{-5}$ with gradient accumulation of 8. The training data includes both ALTU action samples (with the MCP schema in the system prompt) and SPIE MCQ samples (with a knowledge-oriented system prompt).

Result. SSMCQ achieves the highest ID parameter accuracy (84.4%) with 99.3% action accuracy, and 87.3% OOD action accuracy with 59.0% parameter accuracy.

5.5 Voting (Multi-Agent Voting Training)

Idea. Use an ensemble of 5 heterogeneous agents that independently generate candidates, execute them, and vote on the best answer. Train the trainable agents on the consensus winner.

5.5.1 Architecture

Five agents share a single 8 GB GPU via serial loading:

1. **VLM-Trainable:** Qwen2-VL-2B + QLoRA (trainable).
2. **VLM-Frozen:** Qwen2-VL-2B, 4-bit (frozen).
3. **LLM-Trainable:** Qwen2.5-1.5B + QLoRA (trainable).
4. **LLM-Frozen:** Qwen2.5-3B, 4-bit (frozen).
5. **RAG-Agent:** BM25 retrieval over crawled ERPNext docs (no GPU).

5.5.2 Voting Pipeline

1. **RAG retrieval:** BM25 agent returns top-5 relevant documentation chunks.
2. **Generate:** Each agent generates a tool call (serial: load $\rightarrow$ infer $\rightarrow$ unload).
3. **Execute:** Test each candidate against ERPNext.
4. **Short-circuit:** If exactly 1 prediction succeeds, it wins immediately.
5. **Vote:** All 5 agents vote on the best candidate (odd number guarantees majority).
6. **Train:** Trainable agents compute loss on the winning action ($lr = 1 \times 10^{-5}$).

5.5.3 Detailed Description

The Voting approach departs from the single-agent paradigm used by every other method in this project. Instead of relying on one model to produce the correct tool call, it assembles an ensemble of five heterogeneous agents that independently generate candidate answers, tests each candidate against the live ERPNext environment, and then has all five agents vote on which candidate is best. The trainable agents in the ensemble are updated on the consensus winner, so the training signal comes not from a pre-written dataset label but from the collective judgment of the group filtered through real-world execution feedback.

Ensemble composition. The five agents are deliberately diverse. Two are vision-language models that can process screenshots natively, and two are text-only language models that convert screenshots to text through OCR before reasoning about them. One agent in each pair is trainable while the other is frozen with no parameter updates ever applied. The fifth agent is a pure retrieval system with no neural model at all, generating its candidates through keyword matching against the tool schema. This architectural heterogeneity is the core premise: agents with different model families, different input representations, and different training states will make different mistakes, and their disagreements carry information about where the correct answer lies.

Pipeline execution. The pipeline begins with a retrieval step where the non-neural agent queries a search index over crawled ERPNext documentation and returns relevant context that is shared with all other agents. Each agent then independently generates a candidate tool call. Every valid candidate is executed against the live ERPNext API, which returns success or failure purely based on whether the request was well-formed and semantically valid. A short-circuit mechanism then checks the execution results: if exactly one agent produced a call that the API accepted while all others failed, that agent wins immediately and voting is skipped. The reasoning is that the environment itself has already arbitrated: if only one candidate actually worked, there is nothing to deliberate about.

Voting and training. When the short-circuit does not trigger, either because no candidates succeeded or because multiple did, all five agents vote. Each receives the original task alongside all five candidates with their execution outcomes, and selects the one it considers best. Since the ensemble always contains an odd number of voters, a strict majority is guaranteed with no possibility of a tie. The winning candidate becomes the training target: each trainable agent takes a single gradient step to move its output distribution closer to the winner, using a learning rate an order of magnitude lower than the initial supervised warmup to account for the noisier signal. In a production deployment, an additional verification step should follow the vote: the winning candidate would be executed against the live environment, and the agents would vote a second time with the execution outcome as additional evidence. This post-execution

vote would let the ensemble correct its initial consensus when environment feedback reveals that the majority choice was actually incorrect.

Frozen anchors. The frozen agents play a critical stabilizing role. Because they never receive gradient updates, their behavior remains constant throughout training. If the trainable agents start drifting toward poor predictions due to noisy training signal, the frozen agents continue voting for reasonable candidates and counterbalance the drift. This matters because the training signal is self-referential, as agents are being trained on outputs chosen by a vote that they themselves participate in. Without frozen anchors, this feedback loop could amplify errors. With two frozen voters out of five, at least two votes always come from stable, unbiased sources.

Two-phase schedule. Training proceeds in two phases. The first phase is a standard supervised warmup that gives the trainable agents baseline competence before they enter the voting loop. The second phase is the voting training itself, where tasks are drawn, the full pipeline runs, and trainable agents learn from the consensus. The result is a system where the training signal is grounded in both environmental feedback and peer agreement rather than in static labels, allowing the ensemble to potentially correct for errors that any individual agent would reinforce on its own.

5.6 WFCSIT (Wave Function Collapse Self-Improvement Training)

Idea. Adapt the Wave Function Collapse (WFC) algorithm [47], originally designed for procedural generation of 2D and 3D scenes, to generate multi-step ERPNext document creation scenarios. The key insight is that ERPNext documents form a directed acyclic graph (DAG) through Link fields, analogous to foreign-key dependency graphs in relational databases where topological sorting determines valid insertion order [48–50]. WFC navigates this graph to produce valid document chains. In retrospect, the WFC abstraction may be unnecessarily complex for this setting: since the business dependency graph is already a well-defined DAG, a simpler topological traversal with constraint propagation could achieve the same result without the overhead of entropy calculations and tile superposition borrowed from spatial generation.

5.6.1 Constraint Graph

ERPNext doctypes are modeled as *tiles* in a constraint graph:

- **State:** SUPERPOSITION (uncreated), COLLAPSED (created), or FAILED.
- **Field domains:** Set of possible values per field.
- **Entropy:** $\mathcal{H} = \log_2(\prod_i |D_i|)$ where $|D_i|$ is the domain size of field i . Lower entropy = more constrained = selected first.

5.6.2 WFC Algorithm

1. **Select:** Pick the lowest-entropy ready tile (most constrained document).
2. **Generate:** Create a natural-language task for that tile.
3. **Predict:** Agent generates an MCP tool call.
4. **Execute:** Run against ERPNext.

5. **Propagate:** If successful, update dependent tiles' field domains (e.g., a created Customer's name fills Sales Order's customer field).
6. **Backtrack:** If execution fails, mark tile as FAILED, undo constraints, try alternative.
7. **Repeat** until all tiles are collapsed or all alternatives exhausted.

5.6.3 Detailed Description

WFCSIT adapts the Wave Function Collapse (WFC) algorithm [47], originally created for procedural generation of 2D tile maps and 3D scenes, to the problem of generating multi-step document creation scenarios in ERPNext. The key insight is that ERPNext documents form a directed acyclic graph (DAG) through their Link fields, which function exactly like foreign keys in a relational database [48, 49]. A Sales Order cannot exist without a Customer and an Item. A Delivery Note cannot exist without a Sales Order. A Payment Entry cannot exist without a Sales Invoice. This dependency structure is structurally analogous to the adjacency constraints in spatial WFC, where certain tiles can only be placed next to certain other tiles. WFCSIT exploits this analogy to systematically generate valid document creation chains that respect all referential integrity constraints, and then uses these chains as training scenarios for the agent.

The algorithm traverses the DAG from roots to leaves, following topological order. It begins with Tier 0 documents that have no dependencies (Customer, Supplier, Item, Warehouse) and progresses toward higher tiers: transactions (Sales Order, Purchase Order), then fulfillment (Delivery Note, Invoice), and finally settlement (Payment Entry). A tile is only considered ready when all its dependencies have been collapsed, so the engine is forced to create a Customer and an Item before it can attempt a Sales Order, just as a relational database requires referenced records to exist before a foreign-key insert succeeds. This root-to-leaf traversal means the WFC engine builds business workflows in their natural creation order. Traversing the DAG in the reverse direction, from leaves to roots, should also be possible and would generate deletion or cancellation scenarios, where settlement documents must be cancelled before the transactions they reference can be removed. Furthermore, when the model encounters a missing dependency during forward traversal, it could create the required documents on the fly rather than failing, dynamically extending the workflow chain as needed.

At the foundation of the system lies the **constraint graph**, which reimagines ERPNext doctypes as tiles in a WFC grid. Each tile represents a single document slot, such as `customer_0` or `sales_order_1`, and can be in one of four states: `SUPERPOSITION` (not yet created), `COLLAPSED` (successfully created in ERPNext), `FAILED` (creation attempt did not succeed), or `BACKTRACKED` (undone during a recovery step). Every tile contains a set of field domains, where each domain represents the set of valid values that a particular field can take. For example, a Sales Order tile's `customer` field domain might start with all known customer names, but once a specific Customer tile is collapsed, constraint propagation narrows that domain to a single value, the name of the just-created customer. The entropy of a tile is calculated as $\mathcal{H} = \sum_i \log_2 |D_i|$ across all its non-fixed fields. A tile with many free fields has high entropy because many combinations are still possible, while a tile whose fields have been almost entirely constrained by its dependencies has low entropy because very few valid configurations remain.

The **doctype registry** encodes the full ERPNext schema as a 5-tier hierarchy. Tier 0 contains the root party documents: Customer, Supplier, Item, Warehouse, and Employee. Tier 1 holds transactions: Sales Order, Purchase Order, Quotation, and Stock Entry. Tier 2 covers fulfillment: Delivery Note, Purchase Receipt, Sales Invoice, and Purchase Invoice. Tier 3 represents settlement: Payment Entry and Journal Entry. Tier 4 handles manufacturing: BOM and Work Order. For each doctype, the registry stores the required fields and their types, the optional fields, the Link field relationships that point to other doctypes, the child table specifications with their own field requirements, and a set of default value generators that produce WFCLEARN-prefixed names to ensure all created documents are identifiable as training

artifacts. The registry also encodes all the constraint edges between doctypes. For instance, the edge from Customer to Sales Order specifies both an existence constraint, meaning a Customer must be collapsed before the Sales Order can be attempted, and a value constraint using the `copy_value` propagation function, meaning the collapsed Customer’s committed name must be copied into the Sales Order’s customer field.

The **scenario templates** define 6 levels of increasing complexity that serve as the curriculum for training. Level 1 scenarios involve creating a single document, such as just a Customer or just an Item. Level 2 scenarios introduce a parent–child relationship, such as creating a Customer, an Item, and then a Sales Order that depends on both. Level 3 scenarios form short chains, such as Supplier → Item → Purchase Order → Purchase Receipt → Purchase Invoice, requiring the agent to navigate 5 sequential dependencies. Level 4 scenarios represent full business cycles by extending the chain through to Payment Entry, producing 7-step sequences. Level 5 scenarios cross module boundaries, combining procurement, inventory, and sales in a single scenario with 10 steps, such as buying items, stocking them via a Stock Entry, and selling them through a Sales Order and Sales Invoice. Level 6 scenarios involve multi-branch parallelism, such as running two suppliers with two separate purchase order chains plus a full business cycle in a single 14-step scenario. The scenario template registry implements a curriculum schedule: early training cycles weight heavily toward Level 1 and 2 scenarios with 40% and 30% respectively, middle cycles shift toward Levels 3 and 4 at 30% and 25%, and late cycles favor Levels 5 and 6 at 30% and 20%.

The **WFC engine** itself runs the core algorithm in a loop until all tiles in the current scenario are either collapsed or failed. At each iteration, it first selects the lowest-entropy tile among those whose dependencies are all satisfied, i.e., the most constrained document that is ready to be created. It then passes the tile and the current graph state to the task generator, which produces a natural language prompt describing what the agent should create. The prompt includes context about what documents already exist in the scenario, so for example a Sales Order prompt might read: “Create a Sales Order for customer WFCLEARN-CUST-3847 with 50 units of WFCLEARN-ITEM-2913 at rate 245.50, delivery by 2026-03-20. Context: Already created Customer WFCLEARN-CUST-3847, Item WFCLEARN-ITEM-2913.” The agent then generates an MCP tool call prediction. If the prediction parses as valid JSON, the engine executes it against the live ERPNext instance. On success, the tile is marked as collapsed with its committed name and values, and the engine propagates constraints to all dependent tiles. The `copy_value` propagation function fixes a target tile’s field domain to the source tile’s committed value, for example copying the just-created Customer’s name into the Sales Order’s customer field. The `date_gte` and `range_le` propagation functions narrow domains based on temporal or numeric relationships. If execution fails, the engine either attempts backtracking by reverting the tile’s state and restoring its dependents’ field domains (up to a maximum of 3 backtracks per scenario) or marks the tile as failed and moves on. If no ready tiles remain and the scenario is incomplete, all remaining tiles in superposition are marked as failed.

The **reward system** operates at two levels. At the individual step level, it reuses the same 4-component formula from E-GRFT: 0.25 for valid JSON format, 0.25 for correct tool name with partial credit of 0.10 for same-category tools, 0.25 for parameter quality measured as the average of Jaccard key similarity and value match ratio, and 0.25 for successful execution. At the scenario level, it adds two bonuses on top of the average step reward. The chain bonus rewards consecutive successful collapses at a rate of 0.05 per consecutive gold or silver step up to a maximum of 0.15, so three consecutive successes earn the full chain bonus. The completion bonus awards +0.10 if $\geq 90\%$ of tiles were collapsed and +0.05 if at least 50% were collapsed. The scenario-level grading is stricter than step-level grading: gold requires a total ≥ 0.90 , silver requires ≥ 0.70 , bronze requires ≥ 0.50 , and anything below 0.50 is failed. This two-tier scoring means that a model which can correctly create individual documents but fails to chain them together in the right order will earn good step grades but poor scenario grades, providing a learning signal specifically for the multi-step dependency reasoning that WFCST is designed to teach.

The **training loop** runs over multiple cycles. In each cycle, the curriculum selects a set of scenario templates, instantiates each as a constraint graph, runs the WFC engine with the agent providing predictions, scores both the individual steps and the overall scenarios, and accumulates all experiences. After each scenario, a cleanup manager deletes all WFCLEARN-prefixed documents from ERPNext in reverse dependency order, starting from Tier 3 settlements and working back to Tier 0 parties, to respect referential integrity. Every few cycles, the system retrains the trainable agents by mixing 70% of the original SFT data with 30% of the gold and silver WFC step experiences. A notable innovation in the training sample format is that the scenario context is included in the user prompt, so the model learns that the correct action for a given step depends on what documents already exist rather than being a static mapping from instruction to tool call.

Result. WFCST achieves 99.5% action accuracy and 83.2% parameter accuracy on the in-distribution test set, and 86.1% action accuracy and 55.5% parameter accuracy on the out-of-distribution test set. These results are slightly below E-GRFT on OOD performance, which reached 93.1% action accuracy. The likely explanation is that WFCST’s training signal is heavily concentrated on `create_document` operations across a structured doctype hierarchy, whereas the OOD test set includes a broader range of tools and query patterns. The approach’s strength lies in teaching the model to reason about document dependencies and multi-step sequences, a capability that single-step self-improvement methods like E-GRFT and SSMCQ do not directly address.

5.6.4 Complexity Levels

Table 5.2: WFC scenario complexity levels.

Level	Structure	Example
L1	Single document	Create a Customer
L2	Parent $\rightarrow$ child	Customer $\rightarrow$ Sales Order
L3	Chain	Customer $\rightarrow$ Item $\rightarrow$ Sales Order
L4	Full cycle	Customer $\rightarrow \dots \rightarrow$ Sales Invoice
L5	Cross-module	Purchase Order + Sales Order
L6	Multi-branch	Parallel chains

5.6.5 Reward

Per-step: same 4 components as E-GRFT. Per-scenario bonuses:

- Chain bonus: $0.05 \times$ consecutive successes (max 0.15).
- Completion bonus: +0.10 if all tiles collapsed.

Grades: gold (≥ 0.9), silver (≥ 0.7), bronze (≥ 0.5), failed (< 0.5).

Result. 33 gold samples from 3 cycles. L1 scenarios achieve 100% success; L3+ scenarios limited to 39% completion due to ERPNext’s strict field validation. Despite few samples, WFCST matches Voting’s OOD performance (86.1% action, 55.5% param).

5.7 SYS (Autonomous Zero-Supervision Learning)

Idea. Combine insights from six recent research papers into a system where the agent autonomously discovers ERPNext’s schema, designs its own curriculum, learns from execution feedback, and retrains, with **zero human supervision**.

5.7.1 Components

Nine components:

1. **SchemaDiscoverer:** Auto-discovers all doctypes and fields from the ERPNext API. No hard-coded list (vs. E-GRFT’s 26 doctypes).
2. **ConstraintInferer:** Builds a dependency DAG from Link fields and applies topological sorting [48] to determine a valid processing order, a well-established technique for resolving foreign-key constraints in relational databases [49, 50]. E.g., Sales Order’s customer field (type = Link, options = Customer) $\Rightarrow$ edge Customer $\rightarrow$ Sales Order.
3. **CurriculumEngine:** Adaptive task generation:
 - Per-tool difficulty adapts to rolling success rate (Agent0-inspired).
 - Failed tasks spawn 2–3 simpler follow-ups (WebRL-inspired).
 - Curiosity bonus prioritizes unexplored (tool, doctype) pairs.
4. **AuditObserver:** Queries ERPNext Activity Log after each action, injects observations as context (World of Workflows-inspired).
5. **RewardScorer:** 6-component reward:

$$R = \underbrace{R_{\text{format}}}_{0.20} + \underbrace{R_{\text{tool}}}_{0.20} + \underbrace{R_{\text{params}}}_{0.20} + \underbrace{R_{\text{exec}}}_{0.20} + \underbrace{R_{\text{curiosity}}}_{0.10} + \underbrace{R_{\text{chain}}}_{0.10} \quad (5.2)$$

where $R_{\text{curiosity}} = \min\left(0.10, \frac{0.10}{\text{visit_count}+1}\right)$ and $R_{\text{chain}} = \min(0.10, 0.025 \times \text{streak})$.

6. **MemoryBank:** BM25-indexed experience store (Memento-inspired). Retrieves top-5 similar past tasks as few-shot examples at inference time.
7. **AgentWrapper:** Qwen2-VL-2B + QLoRA. Handles load/unload for 8 GB VRAM constraint.
8. **TrainingManager:** 70/30 mix (original SFT + gold/silver autonomous). 2 epochs, $\text{lr} = 5 \times 10^{-5}$. Validate and promote/rollback.
9. **AutonomousLoop:** Orchestrator:
 - Phase 0: Schema discovery + constraint DAG construction.
 - Phase 1 (cycles 0–2): Broad exploration, build memory bank.
 - Phase 2 (cycles 3+): Adaptive curriculum with failure follow-ups.
 - Phase 3 (every N cycles): Periodic retrain.

5.7.2 Safety

- All created documents use AUTOLEARN- prefix.
- Cleanup in reverse dependency order after each cycle.
- Circuit breaker: stop if $<10\%$ execution success for 10+ consecutive tasks.
- Blocked tools: delete, submit, cancel, amend.

5.7.3 Detailed Description

The Autonomous Zero-Supervision Learner represents the most ambitious training approach in this project. Where previous methods like E-GRFT operated over a hard-coded set of 26 doctypes and SSMCQ used binary accept/reject filtering, SYS attempts to build an ERPNext agent that discovers the environment entirely on its own, learns which document types exist, infers how they depend on one another, and progressively masters the full tool set with no human-designed curriculum. The system draws on six recent research papers: ToolRL [40] for reward decomposition, Agent0 [51] for co-evolving curricula, WebRL [52] for failure-driven task spawning, DigiRL [53] for offline-to-online VLM training, World of Workflows [54] for audit log observations, and Memento [55] for episodic memory retrieval.

The system is composed of nine components that work together inside a single orchestration loop. The first component, **SchemaDiscoverer**, queries the live ERPNext instance through its REST API to enumerate all available doctypes, retrieve their field schemas, and sample existing documents. Rather than relying on a static list, the discoverer calls the Frappe API endpoint to get the complete list of document types, then for each one retrieves field metadata including field names, types, whether they are mandatory, and crucially which fields are Link fields pointing to other doctypes. To handle the sheer number of doctypes in a real ERPNext installation, it employs a tiered strategy: it first attempts to discover all doctypes from the API, falls back to a curated priority list if the API call fails, and caches the results to disk so subsequent cycles can start faster. The discovery process also samples up to five existing documents per doctype, which later become reference data for generating realistic task parameters.

The second component, **ConstraintInferer**, takes the discovered schema map and constructs a dependency DAG by analyzing Link fields [48, 49]. If a Sales Order has a Link field pointing to Customer, this creates a directed edge from Customer to Sales Order, meaning Customer must exist before a Sales Order can reference it. The inferer performs a topological sort on this DAG to determine a valid creation order, handling cycles gracefully by breaking them at the weakest link. This constraint graph serves a purpose similar to the hand-crafted WFC constraint graph from WFCST, but is derived automatically from the live system rather than being manually encoded.

The third component, **CurriculumEngine**, generates tasks adaptively based on the agent’s current performance. Unlike E-GRFT’s fixed difficulty progression across cycles, SYS dynamically allocates its task budget across four categories: approximately 40% single-step tasks drawn from the full tool-doctype space, 30% multi-step tasks that chain dependent operations following the constraint DAG, 20% failure follow-up tasks inspired by WebRL [52] where previously failed tool-doctype combinations are retried at a simpler level, and 10% curiosity-driven tasks that deliberately target previously unseen tool-doctype pairs the agent has never attempted. The curriculum engine maintains a per-pair coverage table tracking both the number of attempts and success rate for every combination of tool and doctype it has encountered. Tools with a consistently low success rate have their difficulty automatically reduced, while tools the agent has mastered are pushed toward harder variants. This per-tool difficulty adaptation is a practical implementation of Agent0’s [51] co-evolving curriculum idea: the difficulty distribution is not scheduled globally but adapts independently for each tool based on observed performance.

The fourth component, **AuditObserver**, introduces a mechanism inspired by World of Workflows’ [54] insight that language model agents suffer from “dynamics blindness” because they cannot see the side effects of their actions in the underlying system. After each action is executed, the observer queries ERPNext’s Activity Log for entries created in the last five seconds, formats them as a human-readable observation string, and passes this context to the agent on its next prediction. This allows the agent to see, for example, that creating a Sales Invoice also triggered a GL Entry or that submitting a Purchase Order changed its workflow state. In practice, this turns the system from a blind predict–execute loop into a perceive–predict–execute loop.

The fifth component, **RewardScorer**, extends the four-component reward from E-GRFT with two additional terms, resulting in a six-component weighted sum. The format reward at weight 0.20 checks whether the output is valid JSON with the required `tool` and `parameters` keys. The tool reward at 0.20 verifies that the predicted tool name matches the expected one, with partial credit of 0.10 awarded if the predicted tool is in the same functional category as the expected one. The parameter reward at 0.20 computes a Jaccard-like overlap between predicted and expected parameter key-value pairs, with string similarity for value matching. The execution reward at 0.20 is binary: 1.0 if the ERPNext API returned success, 0.0 otherwise. The curiosity reward at 0.10 is inversely proportional to how many times this tool-doctype pair has been visited before, computed as $\frac{1.0}{\log(\text{visit_count}+1)}$. This incentivizes the agent to explore unfamiliar territory rather than repeatedly executing easy queries it already knows. Finally, the chain reward at 0.10 awards a bonus for consecutive successful steps within a multi-step task, encouraging the agent to maintain accuracy across dependent operations rather than treating each step in isolation. The total reward maps to the same grading system used by other approaches: gold for scores ≥ 0.85 , silver ≥ 0.65 , bronze ≥ 0.45 , and failed below 0.45.

The sixth component, **MemoryBank**, implements a BM25-indexed episodic memory store inspired by Memento’s [55] approach of augmenting agents with retrieval over past experiences without requiring fine-tuning. Every gold or silver experience is added to the memory bank with its task prompt and the successful tool call. When the agent faces a new task, the memory bank retrieves the top three most similar past experiences using BM25 scoring with parameters $k_1 = 1.5$ and $b = 0.75$. These retrieved examples are formatted as few-shot context and prepended to the agent’s prompt, giving it concrete demonstrations of how similar tasks were solved before. The memory bank persists across cycles and is saved to disk, so the agent’s accumulated knowledge survives between training runs. By the end of the run, the memory bank grew to 144 entries spanning 83 distinct tool-doctype pairs.

The seventh component, **TrainingManager**, handles the rejection sampling and retraining pipeline. It loads the original SFT dataset and mixes it with new gold and silver experiences at a 70/30 ratio, the same split used across all approaches in this project. Retraining runs for 2 epochs at a learning rate of 5×10^{-5} with gradient accumulation of 8, again consistent with the other approaches. After retraining, the manager runs validation on a held-out test set and makes a promote-or-rollback decision: if action accuracy dropped by more than 5 percentage points, the new adapter is rejected and the previous version is restored. If accuracy improved or remained within a 2-point tolerance while parameter accuracy improved, the adapter is promoted. This safety gate prevents catastrophic forgetting from a bad batch of training data.

The eighth and final component, **AutonomousLoop**, is the orchestrator that ties everything together. It begins with a Phase 0 schema discovery step, querying the live ERPNext instance to build the complete doctype schema map and constraint DAG. It then loads the agent and computes baseline metrics on the held-out test set. The main loop runs for a configurable number of cycles, defaulting to 30, with 100 tasks per cycle. On each task within a cycle, the orchestrator retrieves few-shot context from the memory bank, obtains audit observations from the previous action, has the agent generate a prediction, routes that prediction through the safety tier system, executes it if permitted, scores it with the six-component reward, records the experience, updates the curriculum’s coverage statistics, and stores successful experiences in the memory bank. The safety tier system partitions the 27 tools into three classes: 17 safe tools that perform read-only queries and can be executed freely, 2 guarded tools for creating and updating documents that require the `AUTOLEARN-` sandbox prefix to be injected into name fields, and 8 blocked tools for destructive or irreversible operations like deleting, submitting, cancelling, or amending documents that are never executed. After each cycle, all `AUTOLEARN-` prefixed documents are cleaned up in reverse dependency order to prevent accumulation of test data.

The orchestrator includes several safety mechanisms beyond the tool tiers. A circuit breaker halts training if the execution success rate within a single cycle drops below 10%, which would indicate a sys-

temic problem like the ERPNext server being down or the model having degraded catastrophically. Every five cycles, the schema is re-discovered to pick up any changes to the ERPNext instance. A convergence detector logs when more than 70% of explored tool-doctype pairs have achieved a success rate above 70%, though it does not stop training at that point to allow continued refinement.

Result. The training run processed 150 experiences across 3 cycles of 50 tasks each. In the first cycle, the agent achieved 33 gold and 15 silver experiences with a 76% execution success rate and an average reward of 0.799. By the third cycle, the execution success rate stabilized at 62% while the number of explored tool-doctype pairs grew from 39 to 83, and high-success pairs increased from 27 to 57. Notably, zero tasks received a failed grade across all three cycles, with only 2 bronze per cycle, indicating that the combination of memory bank retrieval and adaptive curriculum effectively shielded the agent from catastrophic prediction errors. One retrain was performed, producing adapter version v001 which was promoted after validation. The per-tool difficulty adaptation revealed that `create_document` was the hardest tool with a difficulty of 0.0, meaning it was always presented at the easiest level, while `get_item_price` reached a difficulty of 0.7, indicating the agent had sufficiently mastered it to receive harder variants.

Table 5.3: Research papers incorporated into SYS.

Paper	Venue	What SYS borrows
ToolRL [40]	NeurIPS 2025	Reward decomposition
Agent0 [51]	2025	Co-evolving adaptive curriculum
WebRL [52]	ICLR 2025	Failure-driven task spawning
DigiRL [53]	NeurIPS 2024	Offline-to-online for small VLMs
World of Workflows [54]	2026	Audit log observation
Memento [55]	2025	Episodic BM25 memory retrieval

5.8 MSYS (Modular SYS)

Idea. Build a modular, 10-component self-improvement system that runs extended iterative cycles (10 cycles of 50 tasks each) with rejection sampling, combining schema discovery, constraint inference, adaptive curriculum generation, BM25 episodic memory, and RAG retrieval. MSYS extends the core ideas of E-GRFT and SYS into a mature, production-oriented pipeline operating over 27 MCP tools.

5.8.1 Architecture

Ten modules:

1. **SchemaDiscoverer:** Auto-discovers 167 doctypes from the REST API (vs. E-GRFT’s 26 hard-coded).
2. **ConstraintInferer:** Builds a dependency DAG from Link fields, identical to SYS.
3. **CurriculumEngine:** Adaptive task generation with four categories:
 - Early cycles (0–2): 70% single-step, 5% multi-step, ~20% failure follow-ups.
 - Later cycles (3+): 40% single-step, 30% multi-step, 20% failure follow-ups, 10% curiosity.
4. **AgentWrapper:** Qwen2-VL-2B + QLoRA ($r = 16$, $\alpha = 32$, 4-bit NF4).
5. **MemoryBank:** BM25-indexed episodic memory (top-3 similar past experiences as few-shot context).
6. **RAG Index:** Okapi BM25 over crawled frappe.io documentation and GitHub sources (top-2 matches).

7. **SandboxEnforcer + CleanupManager:** 3-tier safety with V5LEARN- prefix; cleanup in reverse dependency order.
8. **RESTExecutor:** Direct Frappe REST API execution for all 27 MCP tools.
9. **RewardScorer + ExperienceBuffer:** 6-component reward (identical to SYS):

$$R = \underbrace{R_{\text{format}}}_{0.20} + \underbrace{R_{\text{tool}}}_{0.20} + \underbrace{R_{\text{params}}}_{0.20} + \underbrace{R_{\text{exec}}}_{0.20} + \underbrace{R_{\text{curiosity}}}_{0.10} + \underbrace{R_{\text{chain}}}_{0.10} \quad (5.3)$$

10. **TrainingManager:** QLoRA retraining every 3 cycles. 70% original SFT + 30% gold/silver. $\text{lr} = 5 \times 10^{-5}$, 2 epochs. Promotion if action accuracy holds; rollback otherwise.

5.8.2 Detailed Description

MSYS represents a matured implementation of the self-improvement paradigm explored in E-GRFT and SYS. Where E-GRFT ran 3 cycles with 26 hard-coded doctypes and SYS ran 3 cycles with autonomous schema discovery, MSYS combines schema discovery with an extended 10-cycle training loop of 540 total tasks. The modular architecture separates concerns into 10 independent components, making each one testable and replaceable.

The curriculum engine operates in two phases. During the first three cycles, it generates predominantly single-step tasks (70%) with minimal multi-step scenarios (5%), allowing the model to build basic competence. From cycle 3 onward, multi-step tasks increase to 30%, failure follow-ups remain at 20%, and curiosity-driven exploration accounts for 10% of the task budget. Write operations are gated by learned difficulty: they are enabled with probability proportional to the model’s demonstrated success rate, capped at 30%.

Each task receives three types of context before the agent generates a prediction: RAG context from the top-2 BM25 matches in crawled documentation, memory context from the top-3 similar past experiences in the episodic memory bank, and audit context from recent ERPNext Activity Log entries. This triple-context approach gives the model both domain knowledge and concrete demonstrations of how similar tasks were previously solved.

Retraining occurs every 3 cycles using rejection sampling: only gold (≥ 0.75) and silver (≥ 0.50) experiences are retained. The mixed training set maintains the 70/30 ratio used across all approaches. After each retrain, validation on 100 held-out samples determines whether to promote or rollback the new adapter, with a 5-point tolerance on action accuracy degradation.

Result. Over 10 cycles: 540 total tasks, 325 gold (60.2%), 190 silver (35.2%), 18 bronze (3.3%), 7 failed (1.3%). Three retrains performed, all promoted (zero rollbacks). 132 unique (tool, doctype) pairs explored, 126 of which (95.5%) achieved $>70\%$ success rate. Self-improvement action accuracy reached 98%, execution success improved from 68.0% to 86.2%. Total runtime: 4h 52min on RTX 4060.

5.9 LSI (Layered Self-Improvement)

Idea. Replace rejection sampling with true reinforcement learning by organizing the self-improvement pipeline into 5 successive layers: dynamic tool discovery, automated data generation, cold-start SFT, GRPO+DPO reinforcement learning, and Alita-style tool evolution. The pipeline draws on ToolBench [20]

for data generation, DeepSeekMath [7] for GRPO, and dynamic tool discovery via semantic routing for reduced hallucination.

5.9.1 Layer 1: Dynamic Tool Discovery

Three components replace the static 27-tool schema:

- **ToolRegistry:** A dynamic dataclass registry seeded with 26 baseline tools. Supports runtime registration of evolved tools from Layer 5.
- **SemanticRouter:** Uses all-MiniLM-L6-v2 (CPU-only, ~1ms/query) for two-level routing: first matching the query to one of 12 ERPNext modules (Selling, Buying, Stock, HR, Accounts, etc.), then ranking tools within the matched module by cosine similarity. This constrains tool selection to a relevant subset, reducing hallucination.
- **EndpointDiscoverer:** Probes the REST API to discover 81 doctype schemas, collecting field definitions and existing documents.

5.9.2 Layer 2: ToolBench-Style Data Generation

Inspired by ToolBench/ToolLLM, with live ERPNext annotation:

- **G1** (single-step): 2,000 template-based instructions. E.g., “List all customers.”
- **G2** (intra-module, 2–3 steps): 806 instructions requiring multiple tools within one module.
- **G3** (cross-module, 3–5 steps): 664 instructions spanning multiple modules.
- **DFSDT Annotator:** Executes G2/G3 instructions against live ERPNext using depth-first search with backtracking, recording (thought, action, observation) traces.
- **Quality Filter:** Hash-based deduplication, consistency checks, and difficulty grading. Aggressively removes 80% of raw samples, yielding 693 final samples (623 train / 70 test).

5.9.3 Layer 3: Cold-Start SFT_auto

QLoRA fine-tuning on the auto-generated data from Layer 2 (not the curated 3,480-sample SFT dataset used by other approaches): Qwen2-VL-2B in 4-bit NF4, $r = 16$, $\alpha = 32$, targeting q/k/v/o_proj. Learning rate 7×10^{-6} with cosine scheduler and 50 warmup steps, 3 epochs, effective batch size 8. Training loss decreased from 0.75 to 0.32 (57% reduction).

5.9.4 Layer 4: GRPO + DPO Reinforcement Learning

Custom GRPO. Standard GRPO implementations batch all rollouts simultaneously, exceeding 8 GB VRAM. LSI generates $N = 4$ rollouts **sequentially** per task at temperature $\tau = 0.7$, executes each against live ERPNext, and computes group-relative advantages:

$$A_i = \frac{r_i - \bar{r}}{\sigma_r}, \quad L = -\min(\rho_i A_i, \text{clip}(\rho_i, 1-\varepsilon, 1+\varepsilon) A_i) + \lambda_{\text{KL}} \cdot \text{KL}(\pi_{\text{old}} \parallel \pi_{\theta}) \quad (5.4)$$

with $\varepsilon = 0.2$, $\lambda_{\text{KL}} = 0.01$, $\text{lr} = 5 \times 10^{-6}$, peak VRAM ~3.5 GB. Gradients are accumulated over 4 prompts before each optimizer step.

Entropy-aware task selection. Inspired by ARPO, each candidate task is probed with a 32-token generation (~50ms) and its per-token entropy measured. Only the top 30% highest-entropy tasks are selected for GRPO training, avoiding wasted rollouts on already-mastered operations.

DPO. Every 3 GRPO cycles, when ≥ 20 preference pairs are collected (best/worst reward gap $> \delta = 0.3$), DPO is applied with LoRA weight swapping: the reference model is a CPU copy of the LoRA weights (~12 MB), incurring zero extra VRAM. Parameters: $\beta = 0.1$, $\text{lr} = 1 \times 10^{-6}$, 1 epoch.

7-component reward. Extends MSYS’s 6-component reward with a multi-step term:

$$R = \underbrace{R_{\text{fmt}}}_{0.15} + \underbrace{R_{\text{tool}}}_{0.20} + \underbrace{R_{\text{par}}}_{0.20} + \underbrace{R_{\text{exe}}}_{0.20} + \underbrace{R_{\text{cur}}}_{0.05} + \underbrace{R_{\text{chain}}}_{0.05} + \underbrace{R_{\text{multi}}}_{0.15} \quad (5.5)$$

where R_{multi} measures the fraction of correctly completed steps in multi-step workflows.

5.9.5 Layer 5: Tool Self-Evolution

Frappe’s REST API is uniform: every doctype follows identical CRUD URL patterns. Layer 5 exploits this by auto-generating three tools per uncovered doctype (`list_<doctype>`, `get_<doctype>`, `count_<doctype>`) using closure-based handlers that delegate to existing executor methods. Each generated tool is tested against the live ERPNext instance; passed tools are registered in the ToolRegistry and the semantic index is rebuilt.

Result. The complete 8-phase pipeline ran in 36 minutes on RTX 4060. Layer 1 discovered 81 doctype schemas. Layer 2 generated 3,470 raw samples filtered to 693. Layer 3 reduced loss from 0.75 to 0.32. Layer 4 ran 3 GRPO cycles with mean reward 0.671 (DPO did not trigger due to insufficient preference pairs in the short run). Layer 5 generated and tested 50 new tools with 100% pass rate, bringing the total to 76 tools.

Why LSI underperforms on the standardized benchmark. The near-base-model performance of LSI-SFT_{auto} and LSI-RL is **expected** and stems from two deliberate design choices. First, LSI’s semantic routing (Layer 1) presents the model with only ~10 tools relevant to each query during training, while the standardized evaluation uses the full 27-tool prompt shared by all approaches. The model was never trained to select among 27 tools simultaneously. Second, Layer 5’s tool evolution generates 50 *new* doctype-specific tools (e.g., `list_customer`, `get_employee`) that do not exist in the evaluation prompt, so the model’s learned vocabulary diverges further from the evaluation schema. Together, these mismatches make the standardized benchmark unsuitable for evaluating LSI’s actual capabilities; the approach should be assessed with its own semantic routing prompt and expanded tool set.

5.10 In-Distribution Results (990 samples)

Approach	JSON (%)	Action (%)	Param (%)	MCQ (%)
Base Model	99.5	74.9	28.4	80.4
SFT	100.0	100.0	82.7	100.0
E-GRFT	99.8	99.8	82.9	100.0
SSMCQ	99.8	99.3	84.4	100.0
Voting (V3 VLM)	100.0	100.0	83.6	100.0
WFCST	100.0	99.5	83.2	100.0
SYS	99.8	99.3	71.8	80.7
SYS (no SFT)	99.8	72.5	25.4	80.7
MSYS	100.0	99.5	80.8	80.7
LSI-SFT _{auto}	95.5	70.4	26.3	79.6
LSI-RL	97.6	72.0	25.6	79.3

Table 5.4: In-Distribution evaluation results. *Action*: percentage of predictions selecting the correct MCP tool (e.g., `create_customer`, `get_list`). *Param*: percentage of predictions passing the correct parameter values (e.g., `name`, `id`). **Bold** indicates best result per column.

Key observations.

- All supervised approaches (SFT, E-GRFT, SSMCQ, Voting, WFC-SIT, MSYS) achieve 99.3–100% action accuracy on ID data. The differences appear mainly in parameter accuracy.
- SSMCQ reaches 84.4% ID parameter accuracy, slightly above Voting (83.6%), EG-RFT (82.9%), and SFT (82.7%).
- MSYS reaches 80.8% parameter accuracy, below SFT (82.7%) but with comparable action accuracy (99.5%).
- WFCST reaches 83.2% parameter accuracy from 33 scenario-based samples.
- SYS has lower ID parameter accuracy (71.8%), likely due to its training data composition: 95% original SFT data + 5% autonomous experiences.
- LSI-SFT_{auto} (70.4% action, 26.3% param) and LSI-RL (72.0% action, 25.6% param) perform near base model level due to a compounding prompt mismatch. LSI generates its own MCP tools via Layer 5 (Alita-style tool evolution), and Layer 1’s semantic router selects only ~10 relevant tools per query during training. Furthermore, the SFT training data was generated with an unfilled tool schema placeholder, meaning the model never learned to process tool lists at all. At evaluation time, the full static 27-tool prompt is used, creating a distribution shift the model was never trained to handle.
- The safety prefixes used during self-improvement (SELFLEARN-, WFCLEARN-, etc.) may also depress parameter accuracy, since the model learns to generate prefixed names that do not match the evaluation targets. In a production deployment, removing these prefixes should improve parameter scores.

5.11 Out-of-Distribution Results (193 samples)

Approach	JSON (%)	Action (%)	Param (%)	MCQ (%)
Base Model	100.0	71.1	26.6	83.3
SFT	99.4	89.6	56.6	100.0
E-GRFT	100.0	93.1	63.6	91.7
SSMCQ	99.4	87.3	59.0	91.7
Voting (V3 VLM)	95.4	86.1	55.5	83.3
WFCSIT	95.4	86.1	55.5	83.3
SYS	98.3	90.2	57.2	75.0
SYS (no SFT)	99.4	65.9	25.4	83.3
MSYS	100.0	91.3	58.4	83.3
LSI-SFT _{auto}	97.1	69.4	28.3	83.3
LSI-RL	97.7	67.6	27.2	83.3

Table 5.5: Out-of-Distribution evaluation results. *Action*: percentage of predictions selecting the correct MCP tool. *Param*: percentage of predictions passing the correct parameter values. **Bold** indicates best result per column.

Key observations.

- E-GRFT has the highest OOD scores: 93.1% action (+3.5pp over SFT) and 63.6% parameter (+7.0pp over SFT).
- MSYS reaches 91.3% OOD action and 58.4% parameter accuracy, close to SSMCQ (59.0%) on parameters.
- SYS reaches 90.2% OOD action accuracy, above SFT (89.6%), SSMCQ (87.3%), Voting and WFCSIT (86.1%), without human supervision during self-improvement.
- SSMCQ reaches 59.0% OOD parameter accuracy, above SFT (56.6%), likely benefiting from the higher quality of its self-generated data.
- WFCSIT and Voting obtain the same OOD results (86.1% action, 55.5% param) despite WFCSIT using 33 samples vs. Voting’s 50 iterations.
- LSI-SFT_{auto} (69.4% action) and LSI-RL (67.6% action) remain near base model level on OOD, consistent with the prompt mismatch noted above.
- Several approaches that were warm-started from the SFT adapter show lower OOD scores than SFT itself (e.g., SSMCQ 87.3% vs. SFT 89.6% action, Voting and WFCSIT 86.1% vs. 89.6%). This suggests that further training can introduce perturbations or partial forgetting of capabilities acquired during supervised fine-tuning, even when the additional data is high quality.

5.12 Generalization Gap Analysis

Table 5.6: Generalization gap: difference between ID and OOD performance. Smaller gaps indicate better generalization. **Bold** indicates smallest gap per column.

Approach	Action Drop (pp)	Param Drop (pp)
Base Model	−3.8	− 1.8
SFT	−10.4	−26.1
E-GRFT	− 6.7	−19.3
SSMCQ	−12.0	−25.4
Voting (V3 VLM)	−13.9	−28.1
WFCSIT	−13.4	−27.7
SYS	−9.1	−14.6
SYS (no SFT)	−6.6	+ 0.1
MSYS	−8.2	−22.4
LSI-SFT _{auto}	− 1.0	+ 2.0
LSI-RL	−4.4	+1.6

Analysis. The base model’s small gap reflects its low absolute performance (less room to drop). Among trained models:

- E-GRFT has the smallest action drop among high-performing approaches (−6.7pp).
- MSYS shows a −8.2pp action gap and −22.4pp parameter gap, smaller than SFT (−26.1pp), SSMCQ (−25.4pp), and Voting (−28.1pp).
- SYS has a −14.6pp parameter drop, the smallest among high-performing trained models, possibly due to its broader exploration across 83 tool-doctype pairs.
- LSI-SFT_{auto} and LSI-RL have small gaps numerically (−1.0pp and −4.4pp action), but both perform near base model level on ID and OOD, so there is little room for a gap. The positive parameter gap (+2.0pp) reflects both scores hovering around 26–28%.
- Voting has the largest parameter gap (−28.1pp), followed by WFCSIT (−27.7pp).

5.13 Live Execution Metrics

Unlike the offline evaluation above, live execution metrics measure actual success when calling the ERPNext API.

Table 5.7: Live execution metrics from self-improvement cycles. These measure actual API execution success, not offline accuracy.

Approach	Best Exec (%)	Best Reward	Failed (%)	Notes
E-GRFT	73.5	0.792	11–19%	26 hard-coded doctypes
SSMCQ	90.5	—	—	Binary accept/reject filtering
WFCSIT	39.0	1.0	61%	Multi-step scenarios
SYS	76.0	0.799	0%	83 tool-doctype pairs
MSYS	86.2	0.795	1.3%	132 tool-doctype pairs, 10 cycles
LSI-RL	67.1	0.671	—	3 GRPO cycles, 4 rollouts each

Note. SSMCQ’s 90.5% execution rate reflects its strict binary filtering. WFCSIT’s 39% rate reflects the difficulty of multi-step document creation with ERPNext’s validation rules. SYS’s zero failure rate may be partly due to its BM25 memory bank and task selection strategy. MSYS reaches 86.2% execution success across 540 tasks using the full tool set. LSI-RL’s mean reward of 0.671 from 3 GRPO cycles is hard to interpret given the short run.

5.14 Per-Tool OOD Accuracy

Table 5.8: Per-tool OOD action accuracy (selected tools).

Tool	Base	SFT	E-GRFT	Voting	WFCSIT	SYS
create_document	60.0	100.0	100.0	100.0	100.0	100.0
get_document	50.0	66.7	75.0	91.7	91.7	66.7
list_documents	92.3	100.0	100.0	92.3	100.0	84.6
search_link	66.7	88.9	88.9	77.8	77.8	77.8
make_mapped_doc	60.0	80.0	100.0	80.0	80.0	100.0
amend_document	0.0	60.0	60.0	60.0	60.0	100.0
get_stock_balance	100.0	100.0	100.0	88.9	88.9	88.9
run_report	66.7	100.0	83.3	83.3	83.3	100.0
get_supplier_details	60.0	100.0	100.0	80.0	80.0	100.0
get_party_balance	83.3	66.7	66.7	60.0	60.0	66.7
list_doctypes	40.0	80.0	80.0	100.0	100.0	60.0

Per-tool notes.

- SYS reaches 100% on amend_document (OOD), where other approaches reach 60%. This tool appeared during autonomous exploration but is rare in the SFT dataset.
- SYS scores lower on list_documents (84.6% vs. 100%) and list_doctypes (60% vs. 100%), likely because the autonomous curriculum undersampled these tools.

Note: Per-tool breakdowns for MSYS, LSI-SFT_{auto}, and LSI-RL are omitted from this table as they were evaluated on the same standardized benchmark without per-tool logging.

5.15 Training Efficiency

Table 5.9: Training efficiency comparison.

Approach	Training Samples	GPU Time	OOD Param (%)
SFT	3,480	~60 min	56.6
E-GRFT	3,480 + 242	~2.5h	63.6
SSMCQ	3,480 + 231	~1.5h	59.0
Voting	3,480	~6h	55.5
WFCSIT	3,480 + 33	~2h	55.5
SYS	2,732 + 144	~1.5h	57.2
MSYS	3,480 + 540	~5h	58.4
LSI-SFT _{auto}	693	~36 min	28.3
LSI-RL	693 + RL	~36 min	27.2

Training Samples denotes the total number of samples used for fine-tuning, expressed as base SFT samples plus autonomously generated samples where applicable. *GPU Time* reports the approximate wall-clock training duration on a single consumer GPU. *OOD Param* measures out-of-distribution parameter accuracy, i.e., generalization to unseen tool–doctype combinations. For LSI, the 693 training samples are autonomously generated after filtering out 80% of the 3,470 raw candidates. The “+ RL” suffix indicates three additional GRPO reinforcement learning cycles applied after SFT.

5.16 Summary

Table 5.10 provides a concise summary of the key findings.

Table 5.10: Summary of key results across all approaches.

Approach	ID		OOD		Gap	
	Act.	Par.	Act.	Par.	Δ Act.	Δ Par.
Base Model	74.9	28.4	71.1	26.6	−3.8	−1.8
SFT	100.0	82.7	89.6	56.6	−10.4	−26.1
E-GRFT	99.8	82.9	93.1	63.6	−6.7	−19.3
SSMCQ	99.3	84.4	87.3	59.0	−12.0	−25.4
Voting	100.0	83.6	86.1	55.5	−13.9	−28.1
WFCSIT	99.5	83.2	86.1	55.5	−13.4	−27.7
SYS	99.3	71.8	90.2	57.2	−9.1	−14.6
MSYS	99.5	80.8	91.3	58.4	−8.2	−22.4
LSI-SFT _{auto}	70.4	26.3	69.4	28.3	−1.0	+2.0
LSI-RL	72.0	25.6	67.6	27.2	−4.4	+1.6

Summary.

1. E-GRFT has the highest OOD scores (93.1% action, 63.6% param), using a hard-coded doctype list and task templates.
2. MSYS reaches 91.3% OOD action and 58.4% param from 10 self-improvement cycles (540 tasks), and 86.2% live execution success.
3. SYS has the smallest parameter gap (−14.6pp) among high-performing models, with no human supervision during self-improvement. Its 90.2% OOD action accuracy is above SFT (89.6%).

4. SSMCQ has the highest ID parameter accuracy (84.4%) from 231 self-collected samples, and 59.0% OOD param accuracy.
5. WFCST and Voting reach the same OOD results (86.1% action, 55.5% param), though WFCST uses 33 samples vs. Voting’s 50 iterations.
6. Voting does not improve over SFT: metrics are unchanged after voting training, as agents already agree on most predictions.
7. LSI-SFT_{auto} and LSI-RL remain near base model level (70–72% action, 26–28% param) due to the prompt mismatch between training (~10 tools) and evaluation (27 tools). This suggests that training and evaluation prompts need to be aligned.
8. SYS trades 11–13pp of ID parameter accuracy (71.8% vs. 82.7–84.4%) for a smaller OOD gap. MSYS sits between the two: 80.8% ID param with 58.4% OOD param.

Toward a complete autonomous learning pipeline. The results presented in this thesis suggest that it should be feasible for an open-source model to autonomously learn how to operate ERPNext, provided it can reliably distinguish successful executions from failures using live environment feedback. The core loop would proceed as follows: the model first crawls the ERPNext website, official documentation, and GitHub repository to build an initial understanding of the system, generating supervised fine-tuning datasets and learning or defining MCP tool. It then infers the business logic dependency graph, the directed acyclic graph of doctypes linked through foreign-key relationships, and uses topological sorting to determine valid creation orders. This DAG serves as a natural curriculum: the model generates grounded tasks of increasing complexity at each tier, from simple single-document creation at the leaves to cross-module multi-step workflows deeper in the graph, producing additional SFT and MCP training data along the way. As the model traverses deeper into the dependency structure, it executes each predicted action against the live ERPNext instance and verifying correctness through real-time environment feedback and iteratively refining its behavior at every stage. Additionally, the model could capture screenshots of the ERPNext interface at each step, pairing them with the corresponding actions to generate vision-grounded training data, thereby enabling it to evolve into a user-facing assistant capable of interpreting and acting upon the graphical interface directly.

5.17 Limitations

5.17.1 Dataset Size and Quality

The ERP-specific datasets, while substantial, were automatically generated from documentation. Human validation and refinement could improve dataset quality and potentially lead to better model performance.

5.17.2 Infrastructure Constraints and Experimental Scope

A fundamental limitation of this work is the inability to run an ERP environment with screenshot capture while having access to GPU resources. The Mediasoft software was not available, and ERPNext requires running servers and initialized databases, but EPFL’s cluster nodes do not provide sudo access and GPU nodes lack internet connectivity. It remains unclear whether it is feasible to run or simulate servers and databases within an Apptainer .sif container under these constraints. This infrastructure gap prevented end-to-end training with live ERP interactions on large models and limited all autonomous learning experiments to Qwen2-VL-2B, a model whose behavior may not be representative of larger architectures.

Critically, the autonomous learning experiments reported here were conducted on very short runs with small datasets and limited action sets. The framework was designed for extended unsupervised sessions involving thousands of iterations, but the results presented reflect only brief evaluations. This discrepancy means the reported metrics likely overestimate real-world performance: prolonged unsupervised training would almost certainly surface additional failure modes such as catastrophic forgetting, response collapse (e.g., the model converging to single-sentence outputs), reward hacking, and compounding errors from incorrect self-generated training signal. These results should therefore be interpreted as preliminary proof-of-concept rather than reliable performance estimates, and substantial further research is required to validate the approach at the scale for which it was intended.

5.17.3 Out-of-Distribution Evaluation

The “out-of-distribution” (OOD) test set shares the same ERPNext environment, tool schema, and task structure as the in-distribution data; it differs only in the specific doctype–action combinations. This makes it closer to a held-out extension of the test set than a genuine distribution shift. True OOD evaluation would require unseen ERP systems, different UI layouts, or entirely new tool APIs, none of which were tested. The reported OOD metrics should therefore be interpreted as measuring interpolation ability within a familiar environment rather than robust generalization to unseen settings.

5.17.4 Hidden Engineering Cost of Procedural Generation

Although the autonomous learning methods report zero or minimal human annotation hours, this metric omits the substantial engineering effort required to design and implement the procedural generation pipelines, task templates, reward functions, and the nine-component SYS system itself. It remains unclear whether deploying these procedural generation techniques at a real scale, in a real enterprise setting with production data, would require comparable engineering investment, potentially negating the claimed reduction in human cost.

5.17.5 Uneven Depth and Lack of Cross-Solution Comparison

The four solutions receive markedly uneven treatment: Solutions 1 and 2 (ChatGPT wrapper and terminal-using agent) are described at a high level with limited technical detail or quantitative evaluation, whereas Solutions 3 and 4 (VLM fine-tuning and autonomous learning) are developed in considerably greater depth. This imbalance reflects the practical trajectory of the internship rather than a deliberate methodological choice. More importantly, the thesis does not provide a controlled comparison across solutions, each operates on a different ERP system, model scale, and evaluation protocol. A unified evaluation framework covering all four approaches on shared tasks and metrics would substantially strengthen the contribution.

5.18 Risk and Security

As AI systems become increasingly capable of understanding interfaces, executing actions, and autonomously learning from interaction, the range of possible applications in enterprise software will expand rapidly. Alongside these opportunities, however, equally significant risks will emerge.

Advanced vision-language models already enable realistic generation of documents, voices, and videos. In an ERP context, this raises the possibility of highly convincing fake invoices, fabricated transaction records, or synthetic customer communications that are difficult to distinguish from legitimate data. When combined with automated workflows, such manipulations could propagate rapidly through accounting systems, leading to financial losses, regulatory violations, or large-scale fraud before human oversight intervenes.

Security threats also extend to the models themselves. Large language models and agentic systems remain vulnerable to prompt injection, jailbreak techniques, and adversarial manipulation. Malicious users may coerce an assistant into executing unauthorized commands, installing malware, modifying system configurations, or creating hidden backdoors within enterprise infrastructure. Model weights, fine-tuning pipelines, and external tool integrations represent additional attack surfaces that could be exploited if not carefully secured.

Furthermore, as multi-agent systems gain autonomy and self-improvement capabilities, failures may become harder to trace and attribute. A corrupted training signal, poisoned dataset, or compromised agent could gradually distort system behavior, producing subtle but persistent operational errors. In highly automated ERP environments, such drift could remain undetected for extended periods.

These risks underscore the necessity of robust security engineering, strict access control, continuous auditing, and human-in-the-loop governance for any AI-driven enterprise system. Technical progress must be accompanied by advances in monitoring, verification, and regulatory compliance to ensure that automation enhances reliability rather than amplifying vulnerabilities.

Note on Swiss data protection. In Switzerland, customer data processing is governed by the revised Federal Act on Data Protection (FADP/DSG), which imposes obligations regarding transparency, lawful processing, and data security. Transmitting ERP data to third-party LLM providers without explicit user consent, valid legal basis, or appropriate data processing agreements may constitute a breach of this law. Cross-border data transfers require additional safeguards. Non-compliance may lead to sanctions by the Federal Data Protection and Information Commissioner and civil liability. This regulatory context reinforces the motivation for privacy-preserving local models explored in this thesis.

Ultimately, the same technologies that enable powerful ERP assistants can also be weaponized or misused. The future of AI in enterprise software will therefore depend not only on model performance, but on the ability to design systems that are resilient, transparent, and secure by construction. Balancing innovation with safety will be essential to realizing the productivity gains of intelligent automation without exposing organizations to unacceptable operational and security risks.

Chapter 6

Conclusion

Multiple viable approaches exist for integrating AI assistants into ERP systems, each involving distinct trade-offs between capability, cost, and privacy. Chatbot wrappers and terminal-based user agents built on commercial models, augmented with documentation access and connected to the ERP through protocols such as MCP and direct database queries, can provide highly effective assistance. Open-source ERP vendors can further facilitate integration by exposing source code and APIs to the AI system.

It is worth noting that simply making ERP documentation publicly available online already constitutes an effective baseline strategy, as modern large language models can perform web searches and leverage this information to assist users directly. Moreover, the rapid pace of development in this field means that general-purpose AI coding and collaboration tools, such as Anthropic's Claude Cowork, may already address many of the use cases targeted in this work. As the ecosystem matures, increasingly capable off-the-shelf solutions will continue to emerge.

However, reliance on commercial AI providers introduces significant privacy and security risks. A malicious actor with access to ERP data at a third-party provider could, for instance, identify financially struggling customers and sell this information to competitors, enabling predatory business practices. Although containerization, strict access controls, and isolated server hosting can mitigate many cybersecurity risks, achieving robust security in practice remains a complex and ongoing challenge.

Privacy-preserving alternatives exist but generally require greater engineering effort, more expensive hardware, and ongoing maintenance for model training. Nevertheless, with well-designed agent frameworks and sufficient computational resources, it is feasible to develop high-quality ERP assistants that operate entirely on local infrastructure.

As open-source foundation models continue to improve, fine-tuning them for domain-specific tasks will become increasingly accessible. In particular, a capable open-source terminal-using agent could be iteratively fine-tuned in a self-improving loop, analogous to the autonomous learning methods explored in this thesis. Major AI companies likely already employ similar strategies at scale, training fleets of agents that compete and improve through structured evaluation and do research. Knowledge distillation from commercial models offers a shortcut to strong performance, but may conflict with the terms of service of the source model provider; explicit authorization should be obtained before pursuing such approaches. As high-quality open-source fine-tuned models become more widely available, combining them with retrieval-augmented generation and structured keyword search should be sufficient to achieve strong enterprise performance without reliance on proprietary providers.

The better the open source gets the easier it will be to finetuning them, one day TUAs might be able

to independently learn how to use the ERP software and finetune itself.

Another promising direction involves introducing an anonymization layer with systematic name and identifier mapping between the terminal-based agent, the user, and the ERP system. In this setup, the agent would interact with the database and software through MCP while all inputs and outputs pass through a consistent anonymization mechanism, ensuring that sensitive fields, such as customer names, company identifiers, or internal references, are replaced with pseudonyms. However, the overall effectiveness of such privacy protection remains uncertain: structural patterns in the data, contextual clues, or re-identification attacks could still allow sensitive information to be inferred, particularly when large volumes of operational data are involved. Anonymization alone should therefore not be considered a complete security solution but rather a complementary safeguard within a broader privacy-preserving architecture.

Finally, AI in ERP systems should not be viewed solely as a user-facing assistant. With deeper integration and controlled autonomy, AI agents could progressively assume portions of routine human interaction by executing workflows, generating insights, and managing operations directly. While this could greatly improve efficiency and consistency, it also introduces challenges in accountability and reliability, requiring strong governance frameworks and human oversight mechanisms.

The economic reality further underscores the difficulty of the local approach. Commercial AI assistants are available for approximately 22.50 CHF per user per month, providing immediate access to state-of-the-art models with no infrastructure investment. By contrast, training and hosting local models requires hardware investments that can range from thousands of CHF or more, depending on the desired scale, with no guarantee that the resulting system will match commercial quality. The experiments in this thesis illustrate this risk concretely: despite considerable engineering effort, the locally trained models remain substantially less capable than commercial alternatives. For small ERP companies with limited budgets, this cost disparity represents a significant barrier. Investing heavily in local AI infrastructure only to produce a system too mediocre for users to adopt is a real risk. This suggests that, in the near term, the most pragmatic strategy for small ERP vendors may be to rely on commercial models with appropriate data protection safeguards, while monitoring the rapid improvement of open-source alternatives for a future transition to local deployment.

What was demonstrated. The RAG-enhanced wrapper and terminal-using agent confirm that commercial models augmented with documentation access and MCP connectivity can provide effective ERP assistance today, with minimal engineering effort. However, they require sending enterprise data to third-party providers, which creates privacy and regulatory risks, particularly under Swiss data protection law.

The VLM fine-tuning experiments on SmolVLM2-256M revealed an important negative result: despite achieving improvements on visual benchmarks, the models failed to acquire transferable ERP knowledge, as shown by the Claudette text-only evaluation where no configuration improved over baseline. QLoRA consistently outperformed full fine-tuning, acting as an implicit regularizer that preserved pretrained capabilities. These findings suggest that 256M parameters and 13,000 synthetic training samples are insufficient for meaningful domain adaptation, the approach requires either substantially larger models, larger datasets, or both.

The autonomous learning framework needs further research. The autonomous learning framework yielded encouraging results across all proposed methods, with each approach demonstrating measurable improvements over the supervised baseline on both in-distribution and out-of-distribution evaluation sets (see Chapter 5 for detailed results). These findings suggest that self-improving training through

live environment interaction is a viable path toward privacy-preserving ERP assistance, though at this stage the framework should be understood as a proof of concept rather than a validated production methodology.

These results must be interpreted with caution. All autonomous learning experiments were conducted on short runs with small action sets and limited iterations. The framework was designed for extended unsupervised sessions involving thousands of cycles, but the results presented reflect only brief evaluations. Prolonged training would almost certainly surface additional failure modes, catastrophic forgetting, reward hacking, response collapse, and compounding errors from incorrect self-generated training signal.

Despite these limitations, the overall trajectory is clear: privacy-preserving ERP assistance through local, self-improving agents should be feasible, and the autonomous learning loop, crawl documentation, read the codebase, explore the software, generate tasks following the directed acyclic graph of business logic dependencies, generate database, interact with the live system, filter successes, retrain, provides a framework for continuous improvement without exposing confidential enterprise data. As open-source foundation models grow more capable, the methods explored here should become increasingly effective. Further research with larger hardware, larger models, and substantially bigger training and evaluation datasets would be necessary to determine whether the reported accuracy figures generalize beyond the limited experimental conditions of this work.

6.1 Future Work

6.1.1 Combining autonomous learning and lessons from Game-Playing AI

Recent advances in game-playing AI offer valuable insights for building CUA (computer using agent) or software-using agents. These systems have demonstrated remarkable success in mastering complex, dynamic environments through self-play and reinforcement learning.

AlphaZero achieved superhuman performance in chess, shogi, and Go within 24 hours of training, starting from random play with no domain knowledge except the game rules. The key innovation is *tabula rasa* self-play: the agent plays millions of games against itself, using Monte Carlo Tree Search (MCTS) guided by a neural network that outputs both policy (action probabilities) and value (position evaluation). This approach eliminated the need for hand-crafted heuristics and allowed the system to discover novel strategies unconstrained by human biases. For software automation, similar self-play mechanisms could enable agents to discover optimal interaction patterns by repeatedly attempting tasks and learning from outcomes.

AlphaStar extended these principles to StarCraft II, a real-time strategy game with imperfect information, continuous action spaces, and long-horizon planning. The system combined imitation learning from human gameplay with multi-agent reinforcement learning in a structured league. The *AlphaStar League* maintained diverse populations of agents with different objectives, some trained to beat specific opponents, others to explore particular strategies. This diversity prevented convergence to narrow solutions and encouraged robust generalization. For ERP automation, a similar league structure could train agents with varying specializations (e.g., document processing, inventory management, financial reporting) that collectively cover the full range of enterprise workflows. Their expertise can then be distilled into a single final model, which selects the most effective solution through a consensus or voting mechanism among the specialized agents.

Stockfish with NNUE demonstrates an alternative paradigm: combining traditional search algo-

gorithms with neural network evaluation. NNUE (Efficiently Updatable Neural Network) provides fast position evaluation that can be incrementally updated as pieces move, enabling millions of evaluations per second on standard CPUs. This hybrid approach, deep search with learned evaluation, achieved stronger performance than pure neural network approaches in chess. For software agents, similar architectures could combine systematic exploration of interface states with learned heuristics for action selection.

Skill Rating Systems Skill rating systems such as Elo and MMR (Matchmaking Rating), used for instance by LeetCode and Codeforces to evaluate developers and LLMs, provide robust mechanisms for continuously estimating agent competence within large multi-agent populations. Rather than relying on static benchmarks, these ratings evolve through direct performance comparisons, allowing the system to identify which agents consistently outperform others across diverse tasks.

These game AI techniques suggest a promising research direction: treating software interfaces as game environments where agents can learn through self-play, maintain diverse strategy populations, and progressively improve through structured competition and curriculum learning. An additional avenue worth exploring is the use of a smaller, highly optimized model during training to efficiently explore strategies and stabilize learning at a fraction of the computational cost, followed by distilling the acquired knowledge into a larger transformer-based model for improved generalization and deployment performance.

6.1.2 Discriminator-Based Evaluation for Complex Software

For highly complex creative software such as Blender, Photoshop, or CAD systems, evaluating agent performance becomes significantly more challenging than in structured ERP tasks. Unlike form-filling operations where success can be verified by comparing field values, creative tasks involve subjective quality assessments and open-ended outputs. In these scenarios, a *discriminator model*, analogous to those used in Generative Adversarial Networks (GANs) [56] and diffusion models [57], could serve as a learned reward signal.

The discriminator would be trained to distinguish between expert-quality outputs and agent-generated outputs, providing a continuous quality score that guides agent improvement. This approach mirrors the generator-discriminator dynamic in GANs: as the agent (generator) improves, the discriminator must become more sophisticated to identify remaining quality gaps, driving iterative refinement. For software automation competitions or benchmarks, discriminators could replace or augment human judges, enabling scalable evaluation of agent submissions.

In diffusion models, discriminator guidance [58] has proven effective for steering generation toward high-quality outputs. Similarly, a discriminator-guided agent could learn to prefer action sequences that lead to outputs indistinguishable from expert work. This would be particularly valuable for tasks where ground truth is ambiguous, such as “make this 3D model look more realistic” or “improve the layout of this document”, where a trained discriminator captures implicit quality criteria that are difficult to specify explicitly.

An open question in this setting is whether agents should be trained to operate the creative software itself through sequential actions, or instead to directly generate the final artifacts (e.g., producing a complete Blender scene), bypassing the tool interface altogether.

6.1.3 NPC-Driven Synthetic Data Generation

An alternative approach to generating realistic enterprise data is to deploy AI-driven non-player character (NPC) agents that simulate business interactions and populate ERP systems through synthetic transactions and workflows. Project Sid [59] demonstrated that over 1,000 autonomous NPC agents can sustain a functioning civilization in Minecraft, complete with emergent economy, culture, and governance, suggesting that similar multi-agent simulations could be adapted to enterprise environments. Rather than relying on real client data, which is scarce and privacy-sensitive, NPC agents could autonomously enact realistic business scenarios (creating customers, placing orders, processing invoices, and managing inventory), thereby producing a continuous stream of ground-truth training data. This idea builds on established work in multi-agent business simulation, where process mining has been applied to analyze the transactional logs generated by simulated enterprise actors [60]. More recently, LLM-empowered agent-based modeling has demonstrated that large language models can drive believable simulations of complex social and economic systems [61]. Combining these approaches with the self-improving training methods presented in this thesis could create a virtuous cycle: NPC agents populate the ERP with diverse, realistic data; the trainable model practices on this data through E-GRFT or SYS; and the resulting improved model in turn generates higher-quality interactions, progressively enriching the training environment without any human annotation.

More broadly, combining skill rating systems such as MMR with the autonomous self-improving learning techniques explored in this thesis could yield highly capable models. Agents ranked by MMR could compete in structured tournaments, with the best-performing agents generating training data for the next generation. This creates a virtuous cycle: competitive evaluation surfaces weaknesses, autonomous learning addresses them, and improved agents raise the bar for subsequent rounds, while comprehensive continuous evaluation monitors and prevents catastrophic forgetting. Such an approach could accelerate progress toward general-purpose software-using agents far beyond what static benchmarks alone can achieve.

Bibliography

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.
- [3] Andrés Marafioti, Quentin Bonnet, Victor Picard, Thomas Wolf, Victor Sanh, and Yonatan Shen. “SmolVLM: Redefining Small and Efficient Multimodal Models”. In: *arXiv preprint* (2025).
- [4] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. “Finetuned Language Models are Zero-Shot Learners”. In: *arXiv preprint arXiv:2109.01652* (2022).
- [5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [7] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [8] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2024.
- [9] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint arXiv:2106.09685* (2021).
- [10] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. “DocVQA: A Dataset for VQA on Document Images”. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2021, pp. 2200–2209.
- [11] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. “ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning”. In: *Findings of the Association for Computational Linguistics: ACL 2022*. 2022, pp. 2263–2279.
- [12] Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. “Endless Terminals: Scaling RL Environments for Terminal Agents”. In: *arXiv preprint arXiv:2601.16443* (2026).

- [13] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.
- [14] Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, Wenhai Wang, Yu Qiao, Xizhou Zhu, and Jifeng Dai. “ZeroGUI: Automating Online GUI Learning at Zero Human Cost”. In: *arXiv preprint arXiv:2505.23762* (2025).
- [15] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. “Grandmaster level in StarCraft II using multi-agent reinforcement learning”. In: *Nature* 575.7782 (2019), pp. 350–354.
- [16] Shiva Maharaj, Nick Polson, and Alex Turk. “Chess AI: Competing Paradigms for Machine Intelligence”. In: *arXiv preprint arXiv:2109.11602* (2022).
- [17] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. “STaR: Bootstrapping Reasoning With Reasoning”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 15476–15488.
- [18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.
- [19] Aaron Parisi, Yao Zhao, and Noah Fiedel. “TALM: Tool Augmented Language Models”. In: *arXiv preprint arXiv:2205.12255* (2022).
- [20] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *International Conference on Learning Representations*. Spotlight. 2024.
- [21] Anthropic. *Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>. Open standard for connecting AI assistants to external data sources and tools. Specification: <https://modelcontextprotocol.io>. Introduction: <https://modelcontextprotocol.io/docs/getting-started/intro>. 2024.
- [22] Rakesh Gangwar. *ERPNext MCP Server*. <https://github.com/rakeshgangwar/erpnext-mcp-server>. MCP server for ERPNext integration. Accessed: 2026-02-18. 2024.
- [23] buildswithpaul. *Frappe Assistant Core*. https://github.com/buildswithpaul/Frappe_Assistant_Core. AI assistant core for Frappe/ERPNext. Accessed: 2026-02-18. 2024.
- [24] Jeremiah Lowin. *FastMCP: The Fast, Pythonic Way to Build MCP Servers*. <https://github.com/jlowin/fastmcp>. Documentation: <https://gofastmcp.com/getting-started/welcome>. Accessed: 2026-02-18. 2024.
- [25] Connor Tam, Reihaneh Rabbany, Prajjwal Bhargava, and Michael Liut Saad Khan. “StructuredRAG: JSON Response Formatting with Large Language Models”. In: *arXiv preprint arXiv:2408.11061* (2024).
- [26] Yichen Wang et al. “StructEval: Benchmarking LLMs’ Capabilities to Generate Structural Outputs”. In: *arXiv preprint arXiv:2505.20139* (2025).
- [27] Elias Niederwieser, Daniel Siegle, and Dominik Matt. “AI-Driven ERP Systems: Integrating Large Language Models for Enhanced Customer Interaction and Operational Efficiency”. In: *Zeitschrift für wirtschaftlichen Fabrikbetrieb* 120.s1 (2025), pp. 112–117.
- [28] Hongyang Yang, Likun Lin, Yang She, Xinyu Liao, Jiaoyang Wang, Runjia Zhang, Yuquan Mo, and Christina Dan Wang. “FinRobot: Generative Business Process AI Agents for Enterprise Resource Planning in Finance”. In: *arXiv preprint arXiv:2506.01423* (2025).
- [29] Amin Beheshti, Jian Yang, Quan Z Sheng, Boualem Benatallah, Fabio Casati, Schahram Dustdar, Hamid Reza Motahari Nezhad, Xuyun Zhang, and Shan Xue. “ProcessGPT: Transforming Business Process Management with Generative Artificial Intelligence”. In: *IEEE International Conference on Web Services (ICWS)*. IEEE. 2023, pp. 731–739.

- [30] Fangyu Lei et al. “Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows”. In: *arXiv preprint arXiv:2411.07763* (2024).
- [31] Peter Luo et al. “BEAVER: An Enterprise Benchmark for Text-to-SQL”. In: *arXiv preprint arXiv:2409.02038* (2024).
- [32] Frappe Technologies. *ERPNext: Open Source ERP*. <https://frappe.io/erpnext>. Source code: <https://github.com/frappe/erpnext>. Accessed: 2026-02-18. 2024.
- [33] sugyan. *claude-code-webui: Web UI for Claude Code*. <https://github.com/sugyan/claude-code-webui>. Accessed: 2026-02-18. 2025.
- [34] Yigal Attali and Maya Bar-Hillel. “Guess Where: The Position of Correct Answers in Multiple-Choice Test Items as a Psychometric Variable”. In: *Journal of Educational Measurement* 40.2 (2003), pp. 109–128.
- [35] LessWrong. *Your LLM Judge May Be Biased*. <https://www.lesswrong.com/posts/S4aGGF2cWi5dHtJab/your-llm-judge-may-be-biased>. Accessed: 2026-02-18. 2024.
- [36] Yuliang Liu, Biao Li, Jiebo Yang, et al. “OCRBench: On the Hidden Mystery of OCR in Large Multimodal Models”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024.
- [37] Chin-Yew Lin. “ROUGE: A Package for Automatic Evaluation of Summaries”. In: *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. Association for Computational Linguistics, 2004, pp. 74–81.
- [38] Qiying Yu et al. “DAPO: An Open-Source LLM Reinforcement Learning System”. In: *arXiv preprint arXiv:2503.14476* (2025).
- [39] YuYue Zhang et al. “VAPO: Efficient and Reliable Reinforcement Learning for Advanced Reasoning Tasks”. In: *arXiv preprint arXiv:2504.05118* (2025).
- [40] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tur, Gokhan Tur, and Heng Ji. “ToolRL: Reward is All Tool Learning Needs”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025.
- [41] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint arXiv:2307.09288* (2023).
- [42] Li-Cheng Lan, Andrew Bai, Minhao Cheng, Cho-Jui Hsieh, and Tianyi Zhou. “Exploring Expert Failures Improves LLM Agent Tuning”. In: *arXiv preprint arXiv:2504.13145* (2025).
- [43] Thomas Carta, Clément Romic, Thomas Wolf, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. “Grounding Large Language Models in Interactive Environments with Online Reinforcement Learning”. In: *International Conference on Machine Learning (ICML)*. Vol. 202. 2023, pp. 3676–3713. URL: <https://proceedings.mlr.press/v202/carta23a.html>.
- [44] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. “RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning”. In: *arXiv preprint arXiv:2410.02089* (2025).
- [45] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. “API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023, pp. 3102–3116.
- [46] Minqi Jiang, Andrei Lupu, and Yoram Bachrach. “Bootstrapping Task Spaces for Self-Improvement”. In: *arXiv preprint arXiv:2509.04575* (2025).
- [47] Maxim Gumin. *Wave Function Collapse Algorithm*. <https://github.com/mxgmn/WaveFunctionCollapse>. Accessed: 2025. 2016.

- [48] Wikipedia contributors. *Topological sorting* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Topological_sorting. Accessed: 2026-02-18. 2024.
- [49] BigBinary. *Resolve Foreign Key Constraint Conflict While Copying Data Using Topological Sort*. <https://www.bigbinary.com/blog/resolve-foreign-key-constraint-conflict-while-copying-data-using-topological-sort>. Accessed: 2026-02-18. 2023.
- [50] SchemaSorter Contributors. *SchemaSorter: Topological Sorting of SQL CREATE TABLE Statements*. <https://pypi.org/project/schemasorter/>. Python library using Kahn's algorithm for FK-based table ordering. Accessed: 2026-02-18. 2023.
- [51] Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. "Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning". In: *arXiv preprint arXiv:2511.16043* (2025).
- [52] Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. "WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning". In: *International Conference on Learning Representations (ICLR)*. 2025.
- [53] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. "DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024.
- [54] Lakshya Gupta, Litao Li, Yizhe Liu, Sriram Ganapathi Subramanian, Kaheer Suleman, Zichen Zhang, Haoye Lu, and Sumit Pasupalak. "World of Workflows: a Benchmark for Bringing World Models to Enterprise Systems". In: *arXiv preprint arXiv:2601.22130* (2026).
- [55] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. "Memento: Fine-tuning LLM Agents without Fine-tuning LLMs". In: *arXiv preprint arXiv:2508.16153* (2025).
- [56] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.
- [57] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.
- [58] Dongjun Kim, Yeongmin Kim, Se Jung Kwon, Wanmo Kang, and Il-Chul Moon. "Refining Generative Process with Discriminator Guidance in Score-based Diffusion Models". In: *International Conference on Machine Learning (ICML)*. 2023.
- [59] Fundamental Research Labs (Altera). *Project Sid: 1000 AI NPC Agents Simulating a Civilization in Minecraft*. <https://www.youtube.com/watch?v=2tbaCnOK190>. Video demonstration of 1000+ autonomous agents with emergent economy, culture, and government. Accessed: 2026-02-18. 2024.
- [60] Sohei Ito, Dominik Vymětal, Roman Šperka, and Michal Halaška. "Process Mining of a Multi-Agent Business Simulator". In: *Computational and Mathematical Organization Theory* 24.4 (2018), pp. 500–531. DOI: 10.1007/s10588-018-9268-6.
- [61] Chen Gao, Xiaochong Lan, Nian Li, Yuan Yuan, Jingtao Ding, Zhilun Zhou, Fengli Xu, and Yong Li. "Large Language Models Empowered Agent-Based Modeling and Simulation: A Survey and Perspectives". In: *Humanities and Social Sciences Communications* 11.1 (2024), p. 1259. DOI: 10.1057/s41599-024-03611-3.

Appendix A

Qualitative Examples

This appendix presents representative model outputs on evaluation benchmarks and domain-specific tasks. The examples illustrate typical failure modes of the base model and, where applicable, the effect of fine-tuning.

A.1 Standard Benchmark Outputs

The following examples show the base SmolVLM2-256M model's responses on public benchmarks. They illustrate that the model can recognize text and interpret documents at a basic level, but makes errors on tasks requiring precise counting or logical reasoning.

Example 1: OCRBench – Text Recognition



Question: what is written in the image?

Base Model Response: The text on the paper says "Clintie".

Ground Truth: CENTRE

Takeaway: The base model hallucinates a plausible but incorrect word, illustrating OCR fragility on stylized text.

Example 2: OCRBench – Text Recognition



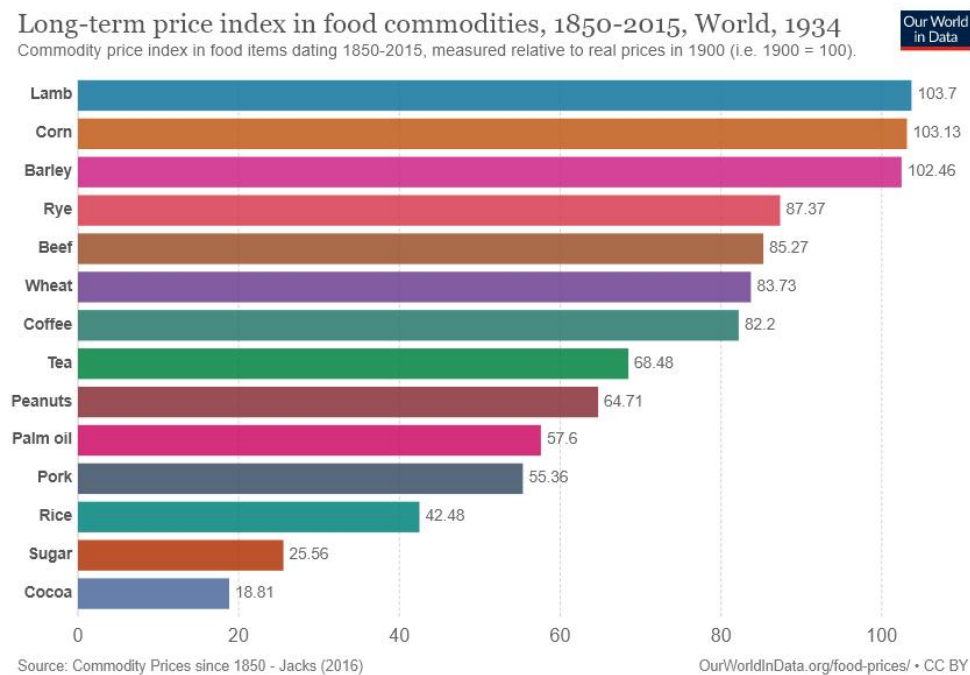
Question: what is written in the image?

Base Model Response: The text "Friend" is visible in white color on a black background.

Ground Truth: FRIEND

Takeaway: Correct recognition. The model handles high-contrast, standard fonts reliably.

Example 3: ChartQA – Data Visualization



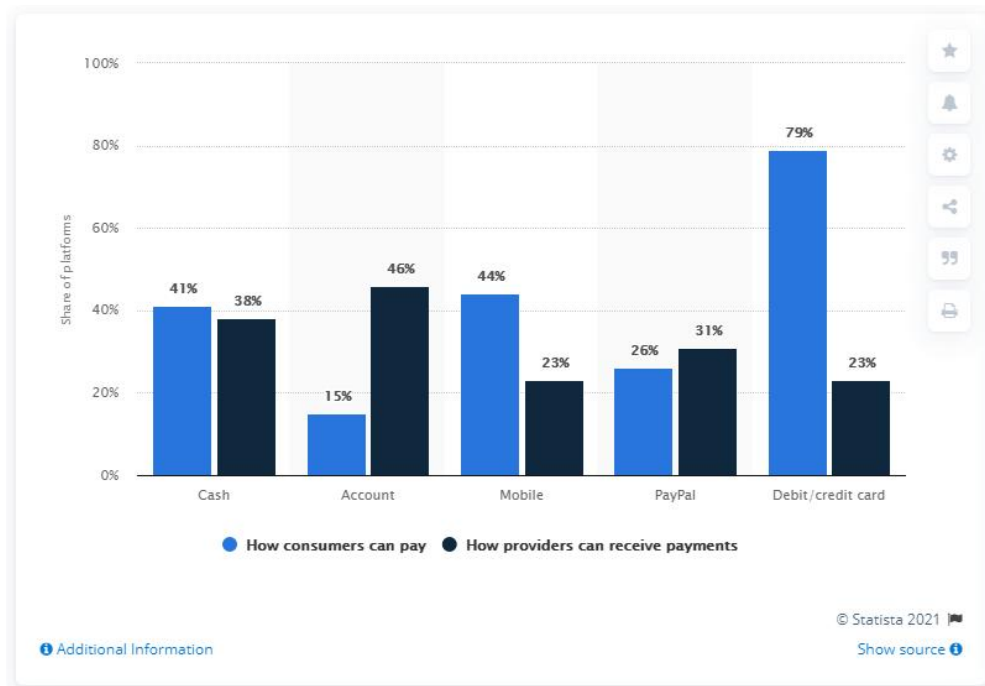
Question: How many food item is shown in the bar graph?

Base Model Response: 16

Ground Truth: 14

Takeaway: The model miscounts items in a dense chart, a common failure mode for small VLMs on quantitative visual reasoning.

Example 4: ChartQA – Data Visualization



Question: Which payment method shows the smallest difference between how consumers can pay and how providers can receive payments?

Base Model Response: PayPal showed a 23 percent difference in customer willingness to pay for services compared with provider's ability to collect payments.

Ground Truth: Cash

Takeaway: The model generates a plausible-sounding but incorrect answer, failing at comparative visual reasoning.

Example 5: DocVQA – Document Understanding

Confidential

RJRT PR APPROVAL

DATE: 1/8/93
SUBJECT: LA gds
PROPOSED RELEASE DATE: for response
FOR RELEASE TO:
CONTACT: P. CARTER

ROUTE TO:

Name	Initials	Date
Peggy Carter	PC	1/8/93
Maura Payne	mt	
David Fishel	DF	
Tom Griscom	TG	
Diane Barrows	DB	
Ed Blackmer	EB	
Tom Rucker	TR	

Return to Peggy Carter, PR, 16 Reynolds Building

5142 3977

Source: <https://www.industrydocuments.ucsf.edu/docs/xnbl0037>

Question: what is the date mentioned in this letter?

Base Model Response: 1/8/93

Ground Truth: 1/8/93

Takeaway: Correct. Factual extraction from structured documents is a relative strength of the base model.

Example 6: DocVQA – Document Understanding

INTER-OFFICE MEMORANDUM

Great Western United Corporation

TO: Blake McFeely
FROM: Barb MacDonald
SUBJECT: Overview - Labor 1971

DATE: 7/6/71

Blake:

Per your request the following is a brief description of labor as it is characterized in recent literature. The following information is presented in the briefest form possible and may be a bit unclear. If so please call and I will elaborate for you.

The subject captions are intended to relate the current union opinion or status in each area. The conclusions attempt to set forth the relationship between the current hard-line bargaining which dominates the labor scene, and the subjects covered on labor related articles over the past year. I have an extensive bibliography with notations as to the best and most pertinent articles should you wish to see it.

NATIONAL POLITICS: Unions are striving to obtain the voting support of the young (between 18 and 21) and the minority voters, as well as the blue collar, middle aged. Union leadership is attempting to control the selection of the Democratic party presidential candidate in an effort to unseat Nixon and anti-labor Congressmen in the 1972 election.

UNION POLITICS: Union leadership is generally old, between 55 and 60 years of age, and clings tightly to the status quo and the ideals of unionism as they appeared in the 1930's. No new blood of equal competence appears eager to take on the general cause of unionism as it exists currently.

MEMBERSHIP: Membership is declining, especially in the areas of aerospace, auto, and agricultural equipment manufacturing. In several unions unemployed members are not required to pay dues, although they are still included in the membership rolls. Loss of dues from such members has caused serious financial problems resulting in a cutback of national union staff.

Rank and file determination to remain segregated for as long as possible has lead to a very low percentage of minority members, and a very slow admission rate.

The average union member is a suburbanite, property holder, and tax payer interested in air pollution and tax relief, not the old line concerns with right to work laws and broadening the picket privileges. Alienation of the worker from his job and the union causes has resulted in little more than self concern on the part

Source: <https://www.industrydocuments.ucsf.edu/docs/ssvx0227>

Question: Which Corporation is mentioned on the letterhead?

Base Model Response: The Great Western United Corporation is mentioned on the letterhead.

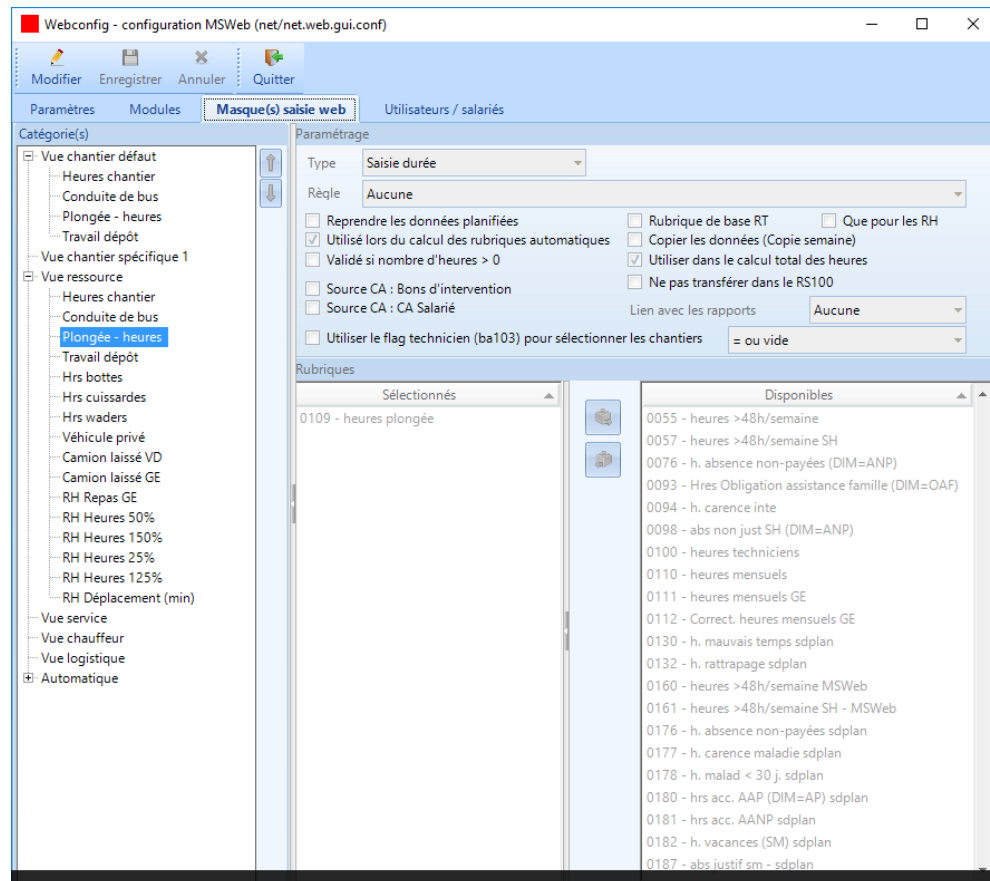
Ground Truth: GreatWestern United Corporation

Takeaway: Semantically correct despite minor formatting difference, showing adequate document comprehension.

A.2 Domain-Specific ERP Outputs

The following examples show model behavior on Mediasoft MS.ERP-specific tasks. They compare the base model, the fine-tuned model, and (for DPO data) the chosen and rejected reference responses, illustrating the effect of domain-specific training.

Example 7: MCQ – ERP Understanding (fine-tuned model)



Question: Quelle est la relation fonctionnelle entre la sélection d'une « Catégorie(s) » dans le panneau de gauche et le contenu du panneau « Paramétrage » à droite ?

- A. Le panneau de « Paramétrage » affiche des options génériques qui restent inchangées, quelle que soit la catégorie sélectionnée à gauche.
- B. La sélection d'une catégorie à gauche ouvre une nouvelle fenêtre de configuration distincte, remplaçant la vue actuelle.
- C. La sélection d'une catégorie dans le panneau de gauche, comme « Plongée - heures », détermine dynamiquement les options de configuration spécifiques affichées dans le panneau « Paramétrage » de droite.

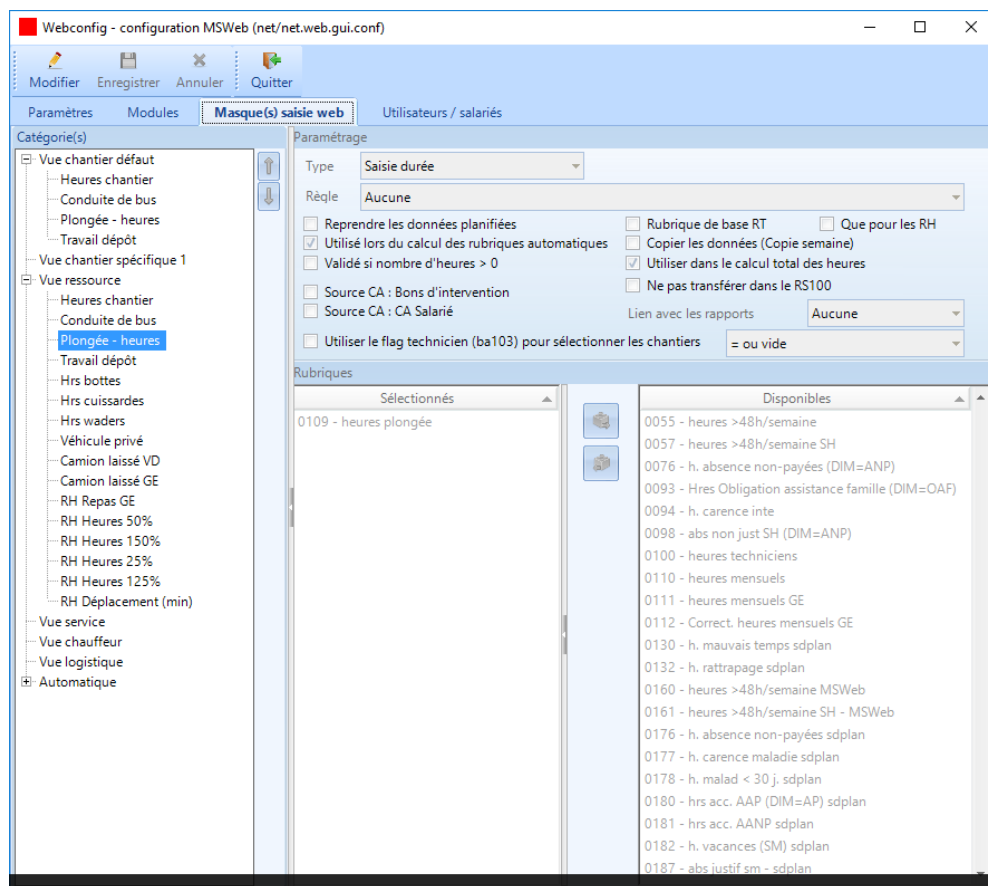
- D. Le panneau de gauche sert uniquement à organiser les vues sans impact direct sur les paramètres affichés à droite.

Correct Answer: B Trained model predicted: B

Takeaway: After brief fine-tuning on ERP documentation, the model correctly answers domain-specific MCQs about interface behavior.

DPO Training Data: Chosen vs. Rejected Responses

The following examples show the chosen (detailed) and rejected (superficial) responses used for DPO training, alongside the base and trained model outputs. They illustrate how the base model degenerates into repetitive token lists, while the trained model produces more structured, though still imperfect, enumerations.



Question 1: Interface Description

« Que montre cette image et quels sont les éléments d'interface principaux visibles ? »

Chosen Response:

L'image présente une interface de configuration d'un logiciel « MSWeb », identifiée par le titre « Webconfig - configuration MSWeb ». On y voit une barre d'outils en haut avec des actions standard comme « Modifier », « Enregistrer », « Annuler » et « Quitter ». Une navigation par onglets est visible, avec « Masque(s) saisie web » comme onglet actif. Le panneau de gauche contient une arborescence « Catégorie(s) » listant des éléments comme « Vue chantier défaut » et « Vue ressource », sous laquelle « Plongée - heures » est sélectionné. Le panneau de droite, « Paramétrage », affiche les détails de configuration de la catégorie sélectionnée, incluant des champs de sélection pour le « Type » (« Saisie durée ») et la « Règle » (« Aucune »), ainsi qu'une série de cases à cocher pour diverses options. Une section « Rubriques » permet de gérer des listes d'éléments « Sélectionnés » (comme « 0109 - heures plongée ») et « Disponibles », avec des boutons de transfert entre les deux listes.

Rejected Response:

L'image montre un écran de configuration avec des boutons et des listes. On peut changer des paramètres pour des rubriques d'heures. Il y a aussi des catégories à gauche.

Base Model Response:

« Modifier, Enregistrer, Annuler, Quitter, Paramètres, Modèles, Modèles saisis web, Répondre les données planifiées, Répondre les données planifiées, Répondre les données planifiées... »

Trained Model Response:

« Modifier, Enregistrer, Annuler, Quitter, Utilisateurs / salaires, Paramètres, Modèles, Masque(s) saisie web, Catégorie(s), Paramétrage, Type, Réglé, Conduite de bus, Plonge - heures, Travail dépôt, Hrs bottes, Hrs cuisards, Hrs waders, Véhicule privé, Sélectionnés, Camion laissé VD, Camion laissé GE, RH Repas GE, RH Heures 50%, RH Heures 150%, RH Heures 25%, RH Déplacement (min), Vue service, Vue chaussure, Vue logistique. »

Takeaway: The base model collapses into repetitive output. The trained model enumerates more interface elements correctly but still produces a flat list rather than a structured description, showing that fine-tuning improved recognition but not response quality.

Question 2: Configuration Identification

« Comment un utilisateur de Mediasoft ERP peut-il identifier la section de configuration active et quels types de paramètres sont présentés dans cette vue 'Masque(s) saisie web' ? »

Chosen Response:

L'utilisateur peut identifier la section de configuration active grâce à l'onglet « Masque(s) saisie web » qui est visuellement mis en évidence par un cadre en pointillé. Dans cette vue,

les types de paramètres présentés dans le panneau de droite « Paramétrage » incluent des options générales configurables via des listes déroulantes comme le « Type » de saisie (« Saisie durée ») et la « Règle » (« Aucune »), des indicateurs d'activation via des cases à cocher (par exemple, « Utilisé lors du calcul des rubriques automatiques » ou « Utiliser dans le calcul total des heures »), des options de liaison avec les rapports et la sélection de chantiers via un flag technicien. Il y a également une gestion de « Rubriques », avec des listes d'éléments « Sélectionnés » et « Disponibles » qui peuvent être transférés.

Rejected Response:

La section active est l'onglet qui est encadré. Les paramètres montrent des options d'heures et des cases à cocher pour choisir des réglages. Il y a des listes pour des rubriques.

Base Model Response:

« Vue de configuration active. »

Trained Model Response:

« Les paramètres sont présentés dans la vue 'Masque(s) saisie web' sont des champs de configuration. »

Takeaway: The base model gives a three-word non-answer. The trained model attempts a relevant response but remains superficial compared to the chosen reference, confirming that 256M parameters and limited training data are insufficient for generating detailed ERP descriptions.

Declaration on the Use of AI Tools

In accordance with academic transparency, the author declares that generative AI tools were used during the preparation of this thesis. Specifically, the following AI-assisted coding and writing tools were employed:

- **Claude Code** (Anthropic): used for code development, LaTeX editing, dataset generation and literature search assistance.
- **Codex / ChatGPT** (OpenAI): used for dataset generation and LaTeX editing.
- **Gemini and Gemini CLI** (Google): used for dataset generation.
- **Nova Pro** (Amazon): used for dataset generation.

These tools were used as assistants to accelerate development and improve productivity.